vFORTH 1.8

ZX Spectrum Next version

also
ZX Microdrive, MGT, 128K back-ports annotations

1990-2026 Matteo Vitturi

...oOo...

Introduction
&
Technical Information

...oOo...

Build 2026-11-07

1 Foreword

This document introduces a Forth implementation suitable to run on *Sinclair ZX Spectrum Next* as a "**dot command**" and as usual "Loader + Code" program. Back-ports to ZX Microdrive and MGT is also summarized.

Please, I invite you to read at least the first 10-12 pages of this document to get an essential overview about this Forth.

This is more a technical reference than an introductory book to Forth: To learn Forth language and for a good reference I suggest the book "Starting FORTH" - Leo Brodie (1981, Forth Inc.) FORTH, Inc. kindly makes it available free of charge on its own website. The first edition is old enough to stick on 16-bits integer numbers. In this perspective, almost all Forth source described in Leo Brodie's book are available within this system in **Screens** from **#800** upward (see §**2.2.2**).

This is a working and functional piece of software, but it's not bug free and there are many things still to do to cope with all new *Sinclair ZX Spectrum Next*'s functionalities – Repository at https://github.com/mattsteeldue/vforth-next.

This is a *quasi* standard Forth based on my previous work **vForth version 1.2** available somewhere in the Internet (https://sites.google.com/view/vforth/vforth1-3). Among the differences is that now it uses a single dedicated file on **SD** to provide a **`BLOCK`** / **Screen** storage facility, while older versions use ZX Microdrive **cartridges** and DISCiPLE **diskettes** (https://github.com/mattsteeldue/vforth).

From this **versions 1.8**, vForth has standard **`DOES>`** inner behavior making **`<BUILDS`** deprecated, but kept for backward compatibility. Among others, this Forth has a facility, called PERSISTENCE (§ **6.7**), that allows you to save your current session and restore it later. Since previous **version 1.7** split the dictionary structure in two parts, a *name-space* and a *code-space*, by exploiting a few 8K RAM pages fitted at **MMU7** providing a more efficient memory usage and more memory available for actual code. This also means there is no more fear in creating *long-named definitions*, because the definition's name uses *heap memory* and not *main memory*. For *heap-memory* capabilities see § **6.4**. Since previous **versions 1.6**, runtime is faster than any previous version, thanks to the idea of dedicating more Z80 registers to keep the internal status of Forth's *virtual machine* and a smaller *Inner-Interpreter*, see § **3.8.1**. Since **version 1.52**, the behavior of **`VARIABLE`** is *standard* and doesn't need an initial value: this means that some older syntax – for instance good for White Lightning – may not work properly as it leaves some extra data on the stack that isn't used by **`VARIABLE`** anymore. From **version 1.51**, this Forth has with two flavors: *Direct-Threaded* or *Indirect-Threaded* code, but versions > 1.7 are Direct-Threaded only, and offer some more speed at the cost of a little more memory allocation for each colon-definition. See § **3.8.1** for some technical detail.

Previous versions are available as zip files at https://github.com/mattsteeldue/vforth-next/tree/master/download/older.

1.1 MIT License

Copyright (c) 1990-2026 Matteo Vitturi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

...---...

This very same MIT License can be viewed from within vForth environment via **`VIEW LICENSE.MD`** provided that **`VIEW`** is made available via **`NEEDS VIEW`**.

1.2 Typos, bugs and suggestions

I, the author am not a native English speaker and you, very likely, will find grammatical errors. In this case, it would be kindly appreciated if you could drop me a line with any suggestion and/or correction and/or bug report at *matteo* -underscore- *vitturi@yahoo.com.* You can often reach me on Facebook.

1.3 Acknowledgment

Many thanks to Rob Probin for his invaluable suggestions and insights.
Special thanks goes to Roland Herrera who helped to edit one of the first issues of this documentation.
Special mention to Albert van der Horst for his 8080 inspirational Assembler library.
Huge thanks to Mike Dailly, most pictures are captured from his #CSpect emulator; all debug sessions rely on it.

1.4 Document structure

Chapter 2 describes how to install, activate and get acquainted with the Forth environment, explaining some basic options and utilities, such as chosing case-sensitive, editing blocks (screens) and feeding your source code in order to create new definitions. It also describes how to create a standalone executable via **ZAP**.
Chapter 3 lists some useful libraries and utilities you can use while in Forth that (memory is a premium) can be imported in your session using **NEEDS**. Here are some details:
- **3.1**: before anything else, you need a way to write and edit your own Forth code: the Editors chapter covers the tools vForth offers for that -- the screen-oriented **Full Screen Editor** and **LED** for real files, plus the historical Line Editor kept only for compatibility along with an old fashion **BLOCK** oriented search-and-locate utility, especially for Microdrive/MGT.
- **3.2**, **3.3**, **3.4** introduce **GRAPHICS** library to manage various **Modes** and **Layers** along with **Color** and **Attributes** definitions, with pixel addressing and line or circle drawing, etc. The libraries give access to the larger Layer 2 mode **320 w x 256 h** pixels, 256 colors total, one color per pixel that's not directly accessible from BASIC, but in Forth we can mess with everything in the system freely.
- **3.5** illustrates how to handle the file-system and various NextZXOS features.
- **3.6** introduces the **MOUSE** facility relying upon the **Interrupt Service Routine** library (§ **3.7**): see the demo demo/color-picker.f for a nice example of interrupt-driven mouse cursor movement.
- **3.8** explains the inner parts of Forth introducing the **SEE** "debugger utility".
- **3.9** shows how to exploit the Standard-ROM **FLOATING** point calculator.
- **3.10** introduces the **ASSEMBLER** vocabulary, an alternative way to code Z80. There are a few examples.
- **3.11** explains how to interact with the Raspberry Pi Zero accelerator (RPi0).
- **3.12** shows how to exploit Hardware Tilemap (80x32 text mode).

Chapter 4 gives some deeper insight and technical information about registers and number formalism.
Chapter 5 is a straight list of error messages stored in the first few Screens.
Chapter 6 provides detailed information of **Forth Dictionary** where each definition is presented in a formal way:
- **6.1** is the "core" dictionary, a long list where most definitions are available at COLD start,
- **6.2** introduces the optional set of definitions that provides the **Case-Of structure**.
- **6.4** describes the **Heap Memory Facility** to access the large quantity of memory available from **Version 1.7** Forth's dictionary itself relies upon this Heap Memory Facility.
- **6.5** introduces the **Testing Suite**, to show that vForth *wants to comply* to modern Standard.
- **6.6** for other minor utilities.
- **6.7** explains **PERSISTENCE** utility.

Chapter 7 briefly shows the memory map and 8k pages used or equivalent BANKs.

1.5 Legend

Much effort was put to adhere to the following conventions:

Courier New	used whenever a Forth definition is referenced or to indicate some typed-in source code.
Calibri Bold	to highlight something important, like filenames, messages, or key-strokes.
Calibri	is used elsewhere

All definitions explained in the dictionary pages are introduced in the form:

EXAMPLE **a1 n1 ... --- a2 n2 ...**

where "**a1 n1 ...**" represents the Stack status before **EXAMPLE** is executed, and "**a2 n2 ...**" represents the Stack status after **EXAMPLE** is executed. Special behavior, such as **IMMEDIATE** definitions are explained properly.

a	memory address	16 bits
b	byte, small unsigned integer	8 bits
c	character	8 bits, but often only lower 7 are significant.
d	signed double precision integer	32 bits
fh	file-handle (NextZXOS)	8 bits
fp	floating point number (see **§3.9**)	32 bits
ha	heap-pointer address (see **§6.4** and **>FAR**)	16 bits.
n	signed integer	16 bits
u	unsigned integer	16 bits
ud	unsigned double precision integer	32 bits
f	flag: a number evaluated as a boolean	16 bits
ff	false flag: zero	16 bits
tf	true flag: non-zero	16 bits
nfa	name field address	16 bits
lfa	link field address	16 bits
cfa	code field address	16 bits
pfa	parameters field address	16 bits
xt	execution token	16 bits

cccc	character string or definition-name available in the vocabulary
...	a list of definitions
TOS	top of calculator stack

2 Getting started

2.1 Installation

This paragraph shows how to do a fresh installation, to upgrade from a previous build see also § 2.5.

The latest version of this software can be downloaded from GitHub repository as .zip file at

https://github.com/mattsteeldue/vforth-next/tree/master/download

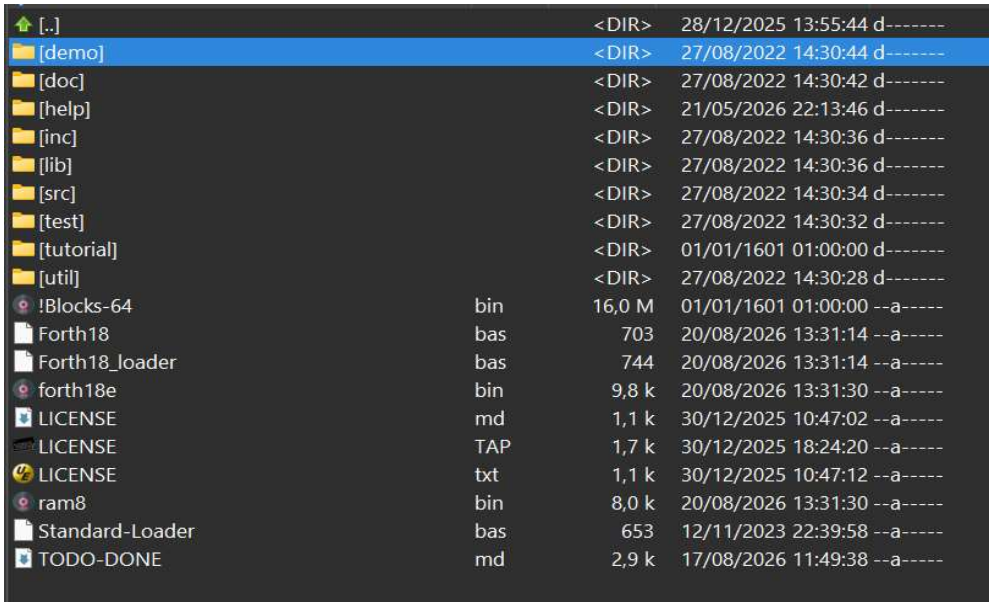
The same executable programs, docs and library are available in the same repository:

https://github.com/mattsteeldue/vforth-next/tree/master/SD/tools/vforth

The zip file contains two subdirectories **/dot** and **/tools/vForth**.

The **/dot** directory contains the **vforth** *dot-command* that has to be copied to **C:/dot/** directory inside your Next's SD card, the **/tools/vForth** directory contains the standard installation and has to be copied to **C:/tools/vForth** of SD so the following sub-directory hierarchy will appear:

demo/ some useful demos and tutorials available via **`INCLUDE`**
doc/ contains help text files and this very same document.
help/ quick online reference files used as interactive help, via **`NEEDS HELP`** and **`HELP cccc`**
inc/ contains source files of single definitions available via **`NEEDS cccc`**.
lib/ same as **inc/** but these source files are a collection of several definitions that forms a "library utility"
src/ among others, the source file of this Forth System.
test/ contains an adaptation of John Hayes' Test Suite to let this Forth more *standard*.
tutorial/ tutorial via **`NEEDS TUTORIAL`** and **`n TUTORIAL`**
util/ with some Perl script to manage with !Blocks-64.bin file I collect over the time.



If you wish to use a different directory instead of **C:/tools/vforth**, you would need to modify the paths in the above Basic programs *and* the manually patch the path+filename specified inside the binary files... or recompile the whole thing.

2.1.1 dot-command version

To properly work, the dot-command version relies on the **standard installation part** described in paragraph 2.1: Along with the standard **/tools/vForth** directory, the repository has a **/dot** directory that contains **`vforth`** binary dot-command that has to be manually copied to directory **/dot**.

N.B. : Executing **`.vforth`** changes current directory to **/tools/vforth** and screen mode to LAYER 1,2. Screen mode is restored on regular quit to Basic, but the current directory is not restored, maybe in the future I will be able to fix this.

N.B.B. : Not *everything works fine* yet in dot-version, for example the **`MOUSE`** facility works or may not work depending of bugs I'm still not able to catch and snatch.

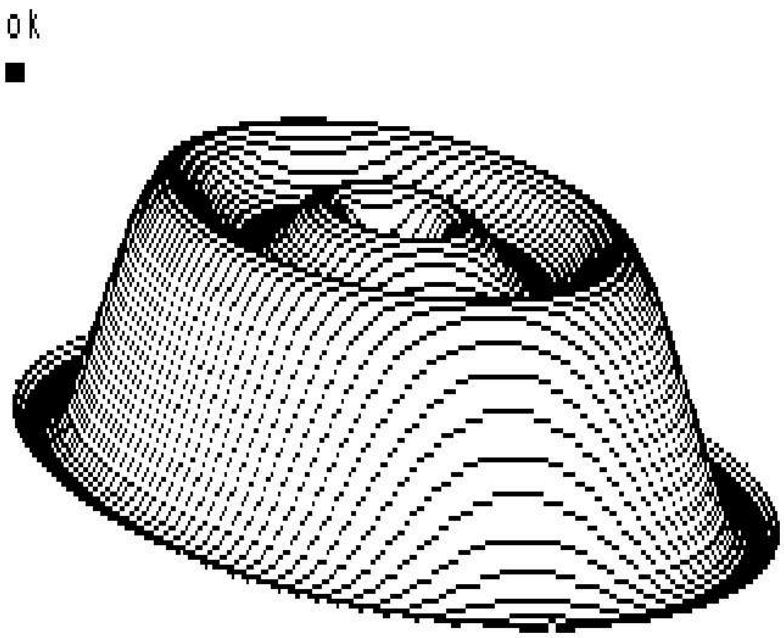
The **dot version** accepts an optional filename parameter that is taken as the source file to be immediately executed, if omitted the default filename executed is **./lib/autoexec-dot.f** source-file that should just load Screen# 11.

If the file doesn't exist, vForth will complete the start-up phase burping something at screen and stopping with the message “F_GETLINE pos error”.

For example:

```
.vforth demo/fedora.f
```

after a couple of minutes of busy loading, it produces the following result



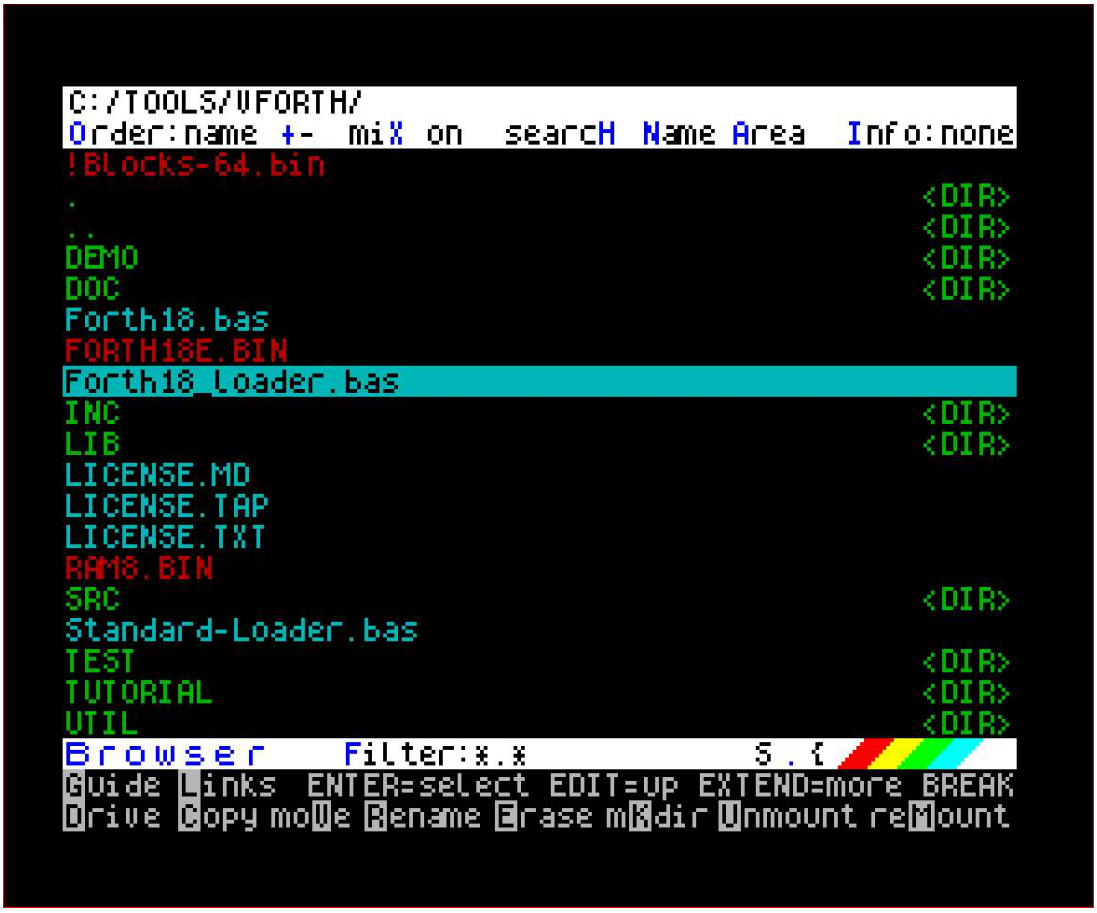
2.2 Activation / Deactivation and tutorial

The Forth System can be activated in two ways:

- running dot-command **.vforth** from command line or NextBasic
- running the Basic program **C:/tools/vforth/forth18_loader.bas**.
  You can activate the Non-dot version via Browser and selecting it, then hitting [ENTER].

Both should be equivalent, but the dot-command loads at address $2000 and has some OS dependencies.

To terminate a Forth session exiting to Basic you can use **BYE**.



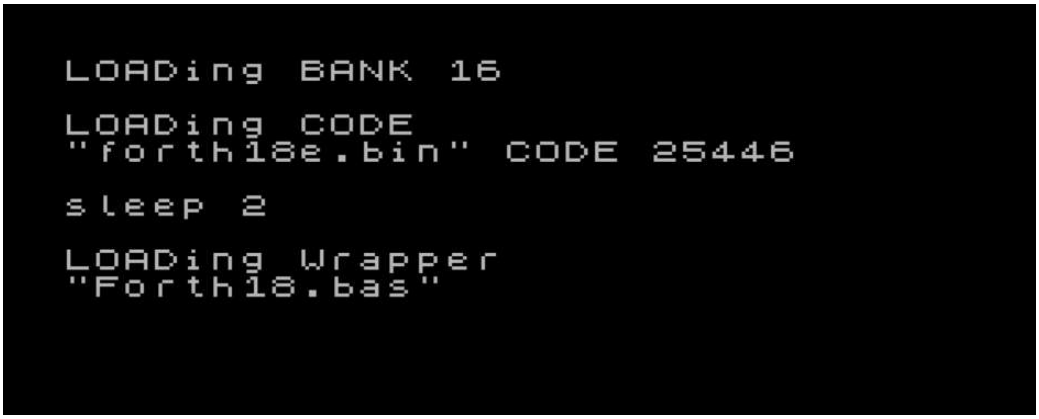
As a personal taste, I prefer the darker color scheme and I always use from **BASIC** the following two commands to have it

```
.editprefs    --scheme dark
.browseprefs --scheme dark
```

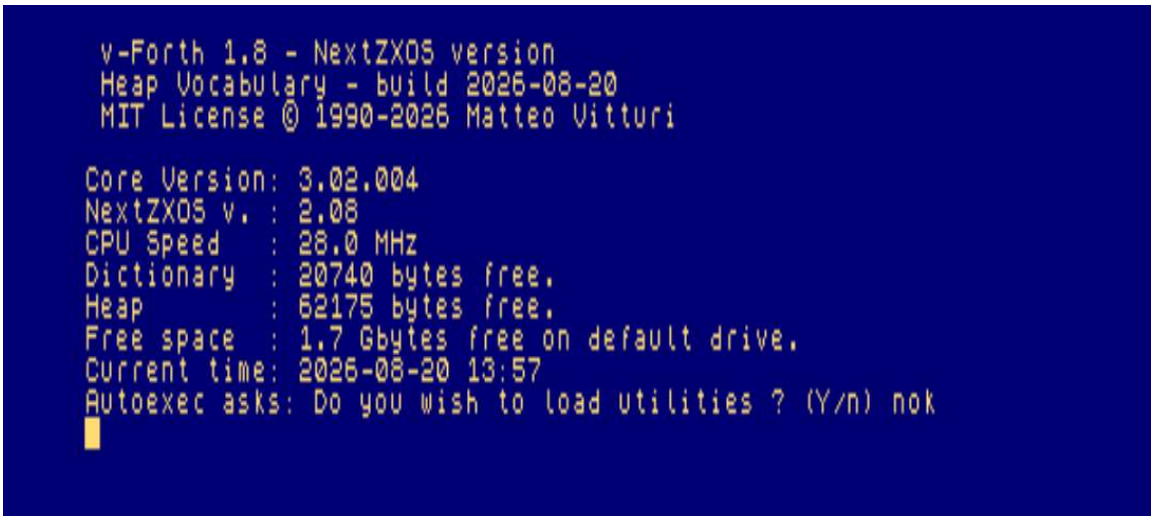
and to have the internal real-time clock aligned:

```
.nxtp time.nxtel.org 12300 -z=CET
```

The Basic loader **forth18_loader.bas** frees upper memory setting RAMTOP to a very low address 25345 and usually loads **ram8.bin** to BANK 16, then **forth18e.bin** to few bytes above RAMTOP and then it loads a smaller Basic launcher **forth18.bas** that you can customize for your purposes. See section §7 for a **memory map**.



Then, a Splash screen displays the Version Number and the Build Date followed by some technical system information that are collected by the **Auto-Start** sequence of **Screen # 11**: within a few seconds the system will ask if you would like to "**`Run Scr # 11 autoexec`**": the only way to refuse is by using the **[N] key**. However, it's a good idea to allow Forth to continue to LOAD **Screen # 11** which in turn loads a few useful utilities making available, among the other definitions, two particular definitions: **`EDIT`** the "**Screen Editor**" and **`SEE`** the "**Debugger Inspector**". This phase is executed only at *first* startup, but you can run it again using **`AUTOEXEC`**. Dictionary space is a premium, this is why of this choice.



In general, in any source-file available in these directories to be imported, I always put some "print" statement just to show the user what is going on or what is being loaded, this is because often, whenever nothing's moving nor blinking on the screen, a person is induced to think the machine just crashed.

The Basic launcher **forth18.bas** usually auto-starts the first time at line `20`, so you usually won't notice, but just in case you `STOP` or the Forth system encounters a ROM Error that forces it to suddenly return to Basic, you have two main choices:

a. type RUN at Basic prompt : This does a **`WARM`** Forth-start, preserving your previous work and buffers status.

b. type RUN 20 at Basic prompt: This does a **`COLD`** start, that should reset all as if you had just loaded from SD card.

In both case, file-handles are not released and you have to manually close them using **`F_CLOSE`** providing the file-handler number. For example **`BLK-FH`** contains the file-handle open to **!Blocks-64.bin** file.

2.2.1 Online Reference – Interactive help

vForth includes a built-in reference system that displays a short description of any word directly on the ZX Spectrum Next screen. It is intended as a quick memory aid at the interactive prompt, not as a substitute for this manual.

Loading. The reference system is not part of the core and must be loaded once per session:

```
NEEDS HELP
```

Usage. After loading, type `HELP` followed by the name of the word you want to look up:

```
HELP DUP
HELP FM/MOD
HELP FLOATING
```

`HELP` searches the **help/** directory on the SD card for a file whose name corresponds to the requested word (with FAT-illegal characters mapped automatically, as described in section 6.1). If the file is found, its content is displayed on screen. If no file exists for that word, nothing is printed. Format of help pages. Each help page is a plain-text file of at most 63 characters per line and 20 lines — sized to fit the 64-column display without wrapping. The first line shows the word name and its stack effect; the second line is blank; the remaining lines give a brief description. For example:

```
  DUP   n -- n n

Duplicate top of stack.
```

Words from libraries. Words defined in **inc/** or **lib/** are not available until the corresponding `NEEDS` has been executed. Their help pages include a reminder on the last line:

```
  Available after NEEDS <wordname>
```

Words built into the core carry no such line — they are always present.

Coverage. Help pages are provided for the complete core vocabulary, all **inc/** single-word definitions, and the major **lib/** modules. The pages are stored in the **help/** directory of the vForth installation on the SD card.

See also: `SEE`, `WORDS`, `DUMP`

2.2.2 Tutorial

Beyond the interactive help, vForth ships with a complete set of guided tutorials. Two complementary tracks are available: a modern one, made of source files in the **tutorial/** directory, and a classic one, made of Screens in the block file, transcribed from the introductory book "Starting FORTH" by Leo Brodie.

The **tutorial/** directory contains several numbered lessons, one file each (NNN-topic.f), meant to be read and tried in sequence. Tutorials 001-029 cover the Forth language itself: stack manipulation, defining words, control flow, memory, strings, `CREATE` / `DOES>`, compilation, vocabularies, floating point, the assembler, and the Block / Screen system with its editor. Tutorials 030-059 cover the ZX Spectrum Next hardware: ULA and Layer 2 graphics, sprites, beeper and AY sound, keyboard and mouse, timing and interrupts, Next registers, MMU paging, file I/O, Copper, UART and more.

Loading.
The tutorial launcher is not part of the core and must be loaded once per session:

```
NEEDS TUTORIAL
```

Usage.
After loading, pass the lesson number to `TUTORIAL`:

```
3 TUTORIAL
```

loads and interprets tutorial/003-output.f. Each tutorial prints a short banner, defines a few small demonstration words to try at the prompt, and shows its examples as comments in the form

```
\  42 .      => 42
```

where => introduces the expected output. To list a tutorial source on screen without interpreting it, use VIEW instead:

```
3 VIEW
```

The [EDIT] key pauses the listing.

Unloading.
Every tutorial begins with MARKER NEWTASK: typing NEWTASK removes, if any, the whole current tutorial from the dictionary, so you can move to the next lesson without wasting dictionary space. The name is fixed and the same in all tutorials. The only exception is tutorial 024 (floating point): there NEWTASK restores integer number interpretation via NO-FLOATING, but does not unload the floating-point vocabulary itself.

The Brodie Screens (800-905). Screens # 800 to # 905 of the default !Blocks-64.bin file contain a transcription of the examples of "Starting FORTH" by Leo Brodie, chapters 1 to 11, in plain standard Forth. Use LIST to read a screen and LOAD to interpret it:

```
800 LIST
800 LOAD
```

Consecutive screens are chained with --> (in vForth there is a THRU definition only via NEEDS). The book itself is not bundled with vForth for copyright reasons, but FORTH, Inc. kindly makes it available free of charge on its own website:

https://www.forth.com/wp-content/uploads/2018/01/Starting-FORTH.pdf

The two tracks run in parallel and are independent: they share the concepts but not the definitions -- the tutorials are written in idiomatic vForth, while the Screens stay close to the book. The header of each tutorial 001-029 carries a cross-reference line pointing to the corresponding Starting FORTH chapter and screen range, so the same topic can be studied from both sides.

See also: HELP (2.2.1), EDIT, LIST, LOAD, MARKER

Table 1 — Language tutorials (000-029)

000	000-help.f	Get familiar with help.
001	001-stack-basics.f	Introduction to the vForth calculator stack, basic arithmetic, and output.
002	002-stack-ops.f	Stack manipulation: rearranging, copying, and discarding cells.
003	003-output.f	Text output: characters, strings, numbers, and numeric bases.
004	004-numeric-bases.f	Numeric bases: BASE, global switching, per-number prefixes, double-precision.
005	005-defining-words.f	Building the dictionary: colon-definitions, constants, variables, values.
006	006-control-flow.f	Conditionals and unbounded loops: IF, BEGIN, and their variants.
007	007-loops.f	Counted loops: DO, LOOP, +LOOP, ?DO, LEAVE, I.
008	008-memory.f	Direct memory access: fetching, storing, and allocating memory.
009	009-strings.f	String and memory operations: moving, filling, and comparing byte arrays.
010	010-create-does.f	Defining words: CREATE, DOES>, and the family pattern.
011	011-bit-ops.f	Bit-wise operations: AND OR XOR INVERT LSHIFT RSHIFT SPLIT FLIP.
012	012-return-stack.f	The return stack: >R, R>, R@.
013	013-case.f	Multi-branch dispatch: CASE OF ENDOF ENDCASE. Vectorized fast indexed dispatch.
014	014-pictured-output.f	Formatted numeric output: <# # #S HOLD SIGN #>.
015	015-double-arith.f	32-bit (double-precision) integer arithmetic.
016	016-input.f	Keyboard input: KEY, ACCEPT, CURS.
017	017-defer-is.f	Deferred words: DEFER, IS, ACTION-OF.
018	018-vocabularies.f	Vocabularies: VOCABULARY, DEFINITIONS, FORTH, CONTEXT, CURRENT.
019	019-compilation.f	Compilation internals: STATE, [ ], IMMEDIATE, [COMPILE], ,
020	020-standard.f	Compilation internals: STATE, [ ], IMMEDIATE, COMPILE,, POSTPONE.
021	021-evaluate.f	Text interpretation: EVALUATE, WORD, FIND, NUMBER.
022	022-introspection.f	Dictionary introspection: WORDS, SEE, WHERE, '.
023	023-structures.f	Data structures: +FIELD, ENUM, ENUMERATED, 2CONSTANT.
024	024-floating-point.f	Floating-point support: exploit standard-rom routines
025	025-memory-advanced.f	Advanced memory: heap, HALLOT, ALIGNED, UNUSED, ROOM, PAD.
026	026-catch-throw.f	Exception handling: CATCH, THROW, ABORT.
027	027-assembler.f	ASSEMBLER vocabulary: writing CODE words in Z80/Z80N assembly.
028	028-blocks.f	BLOCK: native Forth mass-storage, LIST, LOAD.
029	029-edit.f	EDIT: the full-screen block editor.

Table 2 — ZX Spectrum Next hardware tutorials (030-059)

030	030-ula-display.f	ULA display attributes: ink, paper, border, bright, flash, cursor positioning.
031	031-screen-control.f	Screen control: INVERSE, OVER, PAGE, and direct attribute memory.

```
032 | 032-timing.f                | Timing: ms delays and frame-synchronised animation.
033 | 033-beeper.f                | Beeper sound: BLEEP, BLEEP-CALC, BEEP-PITCH, and BEEP.
034 | 034-ay-sound.f              | AY-3-8910 sound chip: registers, chip selection, and tone output.
035 | 035-keyboard.f              | Keyboard input: KEY, WAIT-KEY, ?ESCAPE and a simple menu demo.
036 | 036-graphics-ula.f          | ULA pixel graphics: PLOT, DRAW-LINE, and DRAW-CIRCLE.
037 | 037-layer2.f                | Layer 2 graphics: 256x192 pixels with 256 palette colors.
038 | 038-graphics-advanced.f     | Advanced GRAPHICS.f: LAYER modes, PAINT, XPLOT, and POINT.
039 | 039-sprites.f               | Hardware sprites: setup, pattern upload, and positioning.
040 | 040-next-registers.f        | ZX Next hardware registers (NextRegs): REG!, REG@, SPEED!.
041 | 041-mmu.f                   | MMU and memory banking: mapping 8K pages into the Z80 address space.
042 | 042-file-io.f               | File I/O via NextZXOS: LOAD-BYTES, SAVE-BYTES, PAD" and SAVE.
043 | 043-filesystem.f            | Filesystem navigation: CD, PWD, and directory listings.
044 | 044-mouse.f                 | Mouse input via the Kempston mouse interface and sprite cursor.
045 | 045-copper.f                | TThe ZX Next copper unit: scanline-timed display effects.
046 | 046-bmp-load.f              | Loading BMP images into Layer 2 memory with BMP-LOAD.
047 | 047-uart.f                  | UART serial communication
048 | 048-rpi0.f                  | Raspberry Pi Zero bridge: RPI0, TERM, and the # meta-command.
049 | 049-interrupts.f            | Interrupt service routines (ISR) with lib/INTERRUPTS.f.
050 | 050-afxframe.f              | AY music playback with AFXFRAME and the interrupt handler.
051 | 051-keyboard-matrix.f       | Direct keyboard-matrix scanning: KEY-DOWN?, KEY-SCAN, GET-KEY and redefinable game keys
(KEYBOARD.f).
052 | 052-modular-graphics.f      | Modular graphics: load just ONE display mode with LAYERxx-GRAPHICS.
053 | 053-more-sprites.f          | Hardware sprites, intermediate: struct, loading patterns from file, animation.
054 | 054-dma.f                   | DMA stub demo (DMA.f is not developed yet).
055 | 055-afx-sound-board.f       | Background AFX sound: fire-and-forget effects while the keyboard stays live.
056 | 056-layer2-palette.f        | Layer 2 palette: palette-swap, 9-bit colors, priority, Global Transparency $14.
057 | 057-dot-commands.f          | Building standalone NextZXOS ".dot" commands with vForth's ASSEMBLER.
058 | 058-layer3-tilemap.f        | The Tilemap hardware layer (Layer 3): 40x32/80x32 cells, LAYER3 and TILE80.
059 | 059-standalone-executables.f | Freezing a running session into a standalone executable: ZAP and ZAP".
```

Table 3 - Advanced topics tutorials (060-063)

```
060 | 060-idiomatic-traps.f       | A field guide to Forth/vForth traps: mistakes that compile and run silently.
061 | 061-locals.f                | Named local variables: the { ... } bracket form and what they cost.
062 | 062-error-messages.f        | Numbered error messages: how ?ERROR, ERROR and MESSAGE work together.
063 | 063-blocks-as-assets.f      | BLOCK as a binary-asset store: LOAD2BLOCK and the AFX sound library.
064 | 064-scaled-integer-math.f   |
065 | 065-fedora-silhoutette.f    |
066 | 066-kempston-joystick.f     |
```

2.2.3 Editing with Visual Studio Code

vForth source files are plain text and can be written with any editor. For **Visual Studio Code** an extension is available, distributed as the file vforth-0.1.2.vsix in the tools/ directory. It is not part of vForth itself and it does not run on the ZX Spectrum Next: it works on the vForth directory tree on your PC, and, if you use the CSpect emulator, on its SD card image.

Installation. VS Code 1.75 or later is required: Open the Extensions view, choose "Install from VSIX..." in the "..." menu and select the file; or, from a terminal in the tools/ directory:

```
code --install-extension vforth-0.1.0.vsix
```

then reload the window. A newer .vsix is installed the same way, it does not update by itself.

Language support. Open the vForth directory (the one holding src/F18e.f, inc/, lib/ and help/) with File > Open Folder; the extension recognises it and treats every .f file as vForth source. It provides:

- syntax colouring, which tells core words, library words (those defined in inc/ and lib/) and words defined in the file itself apart;
- diagnostics: a library word used before the `NEEDS` that loads it is flagged together with the `NEEDS` line to add; a `NEEDS` or `INCLUDE` that cannot be resolved and any non-ASCII character outside comments are flagged too;
- hovering a word shows its help/ page, the file where it is defined and the `NEEDS` that loads it, the same page that HELP would show;
- go to definition, which on the argument of `NEEDS` or `INCLUDE` opens the file, and an outline of the definitions of the current file.

The check follows the real order of loading, including the files loaded by the loaded files, so it reflects what `NEEDS` would find on the machine.

Settings. Language support needs no setting when the vForth directory is the one opened. The tools that act on the SD card image need the following, found in Settings under "vForth: SD image":

- vforth.root — the vForth directory; empty means auto-detect
- vforth.sdImage — the CSpect SD card image file (.img)

- vforth.hdfmonkeyPath full path of hdfmonkey, or just hdfmonkey if it is on `PATH`
- vforth.sdDestPrefix the directory of the SD image where the vForthtree is mirrored, e.g. /tools/vforth

The image must be the .img file itself, not !Blocks-64.bin nor any file inside it. The tools use hdfmonkey, a small program that reads and writes the FAT filesystem of the image directly; it is available at https://github.com/simonowen/hdfmonkey.

Pushing a file. The command "vForth: Push file to SD image", from the command palette (Ctrl+Shift+P), copies the file being edited into the SD card image, in the position given by its path relative to the vForth directory, preceded by vforth.sdDestPrefix. Because hdfmonkey does not need to lock the image, this works while CSpect is running, and a running vForth sees the new file at once: it is enough to type `NEEDS` or `INCLUDE` again. No `REMOUNT` is required. Nothing is pushed when a file is saved, and a file under a directory that is not normally deployed to the SD card (dev, doc, dot, emu, forum, project, prompts, tools, version) is pushed only after confirmation.

Editing a Screen. The command "vForth: Open Screen #" asks for a Screen number and opens it, taken from !Blocks-64.bin in the SD card image, as a text document of 16 lines of 64 columns. A ruler at column 64 and a border under line 16 mark its limits. Ctrl+S checks that every line is at most 64 characters, that all characters are 7-bit ASCII and that there is no NUL; only then the 1024 bytes are written back and the rest of the block file is left untouched. Otherwise the line and column of the fault are reported and nothing is written.

Editing a Block. The command "vForth: Open Block # (hex)" opens a single Block (512 bytes) as raw bytes in the Microsoft "Hex Editor" extension, which VS Code offers to install the first time. It is meant for Blocks that do not contain source text, such as graphics, `PERSISTENCE` snapshots or the message table. The size must remain exactly 512 bytes.

Moving between Screens and Blocks. With a Screen or a Block open, **Ctrl+Shift+F8** opens the next one and **Ctrl+Shift+F7** the previous one, replacing the current tab.

Each open or save of a Screen or a Block reads or writes the whole block file through hdfmonkey, which takes a fraction of a second. If a Block is written from inside CSpect during that short time, the change could be lost.

2.3 Case-insensitive and Case-sensitive option

By default, the Forth interpreter is *case-insensitive*, so you can type your commands using lower-case or upper-case or a mix of them with no difference. To enable or disable the case-sensitive interpretation you can use **CASEON** and **CASEOFF** definitions.

The case-insensitive option applies to the Interpreter's dictionary search and to string comparison done by **COMPARE**, and so by any definition built on it, such as **LOCATE, GREP, BSEARCH** and **SEARCH.** A new definition-name retains the exact case it was coded with.

2.4 Block / Screen system

This Forth System comes with a file named **!Blocks-64.bin** that provides the simplest mass-storage system usually available in old-days Forth systems. The complete text-dump of !Blocks-64.bin file is available in **./doc/txt/** directory.

To choose a different file you must the **USE** definition (§ 6.1), for example:

```
USE /tools/filename.bin
```

A **BLOCK** is identified by its *number*, an integer between 1 and 32,767, and can be temporarily be kept in a **BUFFER** identified by the same *number* in a memory area that can be addressed, inspected and (persistently) modified. The dedicated memory area that lies between **FIRST** and **LIMIT**. In this system there are 7 buffers (for a few years it used to be 6 buffers, but we used one more buffer to solve a bug).

Using **UPDATE**, the latest referred **BUFFER** is marked for re-write to file at the moment the system will request the same memory area for a different **BLOCK** or when **FLUSH** is used.

On this Forth system, two **BLOCKs** forms a **Screen** that can be edited using the "Full Screen Editor" utility available after you type: **NEEDS EDIT** (see §**3.1.1**).

Each **BLOCK** has 512 bytes and each **Screen** is 1 KByte so it can store text in 16 lines and 64 columns. A **BLOCK** can be used as a *virtual-memory* area where you can persistently store *anything*. For instance, you can think a **BLOCK** as an 256-elements *integer array* with persistence, but before doing it read the **HEAP** facility here below (§**6.4**). The default file included **!Blocks-64.bin** contains *years* of code and data I collected before the system was able to use external source files via **INCLUDE**. Screens from 99 upward contain the ASSEMBLER vocabulary, the same that can be imported via **NEEDS ASSEMBLER**; screens 800 upwards contain almost all Forth source described in the introductory book "Starting FORTH " by Leo Brodie; screens 2200 upward are filled of binary data imported from **.afx** sound files; then I used screens 4000 upwards as a very large array to keep track of prime numbers...

2.4.1 Block File Format

The file provided as default (**!Blocks-64.bin**) is 16 MBytes long and it is a simple ordered concatenation of all blocks available, so that block # 1 starts at file-offset 0 and ends at file-offset 511, block # 2 starts at offset 512 and ends at offset 1023, and so on. Block # 0 doesn't exist, and the greatest number available is then 32,767 (16,383 **Screens**).

A **Block-number** is always twice the corresponding **Screen-number**, for example **Screen # 11** uses block # **22** and **23**. You may argue that this way **BLOCK** *number* 1 is never accessible, and in fact it is reserved for internal purposes.

Remember the rule: **"Block # is always twice the Screen # (and the next one belongs to the same Screen)"**

```
Screen# N = BLOCK 2N and BLOCK 2N+1
```

You can use your own file, but that the first few Screens are reserved for error messages and such things. In particular, Screens between 4 and 8 contain the standard messages, listed in §**5**. If you don't have them there, the system will become quite foolish, since any **ERROR** or **MESSAGE** would display something gibberish or blank.

2.5 Upgrading keeping your own code and data

I'm used to release new builds to provide bug-fixes or new features I'm working on. Each build comes with the latest version of *my own* **!Blocks-64.bin** file. This provides **BLOCK** / **Screen** storage facility, a simple and easy way to save *your own* code and data across sessions and usually you want to modify such a file via **EDIT** (§**3.1.1**).

This means that, upgrading, you usually want *and can keep* your own **!Blocks-64.bin** file that you modified from a previous release.

To extract the first **N** blocks from a **!Blocks-64.bin** file and produce a human readable text file, you can use a Perl utility **blocks2txt.pl**. See documentation file **doc/txt/!Blocks-64.bin_**yyyymmdd**.txt**.

You can use **putscr.pl** utility to store a group of blocks back to **!Blocks-64.bin** file, starting from a text file with the same format produced by **blocks2txt.pl**. This is useful in case you just want to restore a few blocks but retain your own **!Blocks-64.bin** file. For example, if you inadvertently had corrupted the standard message blocks (see Screens from #4 to #8) you can restore them with the command

```
$> perl  putscr.pl  Standard_Messages.txt  !Blocks-64.bin  4
```

in this case, the text file Standard_Messages.txt was extracted from the output of the first utility executed beforehand.

Also, from within vForth, there are a few utility definitions that allow you to move blocks around:

- **load2block.f** to import a single block from file
- **load2scr.f** to import two contiguous blocks that forms a Screen.
- **screen-to-file.f** to export a screen to file.
- **screen-from-file.f** to import a screen from file.

2.6 Character visualization size

In this Forth implementation I prefer LAYER 1,2 display mode to allow 64 character per line: this is quite necessary in order to be able to display a whole 1024 characters Screen in a single go.

If you prefer LAYER 1,1 you can add a line 61 in **forth18.bas** wrapper as follow

```
61 LAYER 1,1: PRINT CHR$ 30; CHR$ 4;
```

to switch to LAYER 1,1 and condensed character set. The result is quite poor in my opinion.

You can also change **LAYER** mode using some Layer-related definitions available after **NEEDS GRAPHICS** or **NEEDS LAYERS**.

2.7 Source feeding and output spooling

Before entering Forth, the Basic launcher is allowed to open text-files via OPEN#, for instance

```
OPEN# 13,"o>out.txt"
```

that can be later selected for output in Forth via **13 SELECT** to collect any output you send to this output-channel. To restore sending output to video there is an easy **VIDEO** definition that simply does **2 SELECT**.

Two specific definitions allow you to include (and compile) source from any file i.e. **INCLUDE** and **NEEDS.**

For example,

```
INCLUDE demo/chomp-chomp.f
```

or

```
NEEDS GRAPHICS
```

Moreover, you may edit almost any source text-file using **LED** – which uses the whole 2MB of RAM – via

```
NEEDS LED
```

See § 3.5 “LED – The Large EDitor”for more details.

2.8 Creation of standalone executable

Available only for the non dot-version, the **ZAP** and **ZAP"** definitions (see §6.1) turn a vForth game or program into a standalone executable: a frozen copy of the whole vForth system that, once loaded, starts straight into one chosen definition, with no ok prompt and no editor, and goes back to BASIC when that definition ends.

ZAP writes the memory images as binary files: **cccc-core.bin** (from **ORIGIN** to **HERE**), **cccc-user.bin** (the user variables and the block buffers, from R0 to $E000) and one file for each 16K bank of the heap that **HP** has reached: **cccc-heap.bin** for bank 16, **cccc-heap1.bin** for bank 17, and so on up to bank 19. They are loaded by a BASIC program made from "Standard-Loader.bas": MERGE it, write the name in its DATA line 7990, and GO TO 9999 to save it as **cccc.bas** next to the

binary files. The loader reads from the images themselves where the user area goes and how many heap banks there are, so it never needs to be edited when vForth is rebuilt. The directory **demo/chomp-chomp** holds a complete example: game.bas and the files produced by **ZAP GAME**.

ZAP" writes instead a single **.nex** file, which is started from the NextZXOS Browser without any BASIC loader: it holds banks 5, 2 and 0 and the heap banks HP has reached. **demo/chomp-chomp/chomp-chomp.nex** is the same game made this way.

NEEDS ZAP (or **NEEDS ZAP"**) must be the last thing loaded before **ZAP:** the image is the dictionary as it was just before it, so neither **ZAP** itself nor anything defined after it ends up in the executable. The image is saved with **WARNING** set to 0, because a standalone program has no block file: an error shows just its number, **msg#n**, and the program goes back to **BASIC** through **BYE,** as it does when its definition ends. A program that calls **QUIT** instead stops at the ok prompt of the standalone image. The session in which **ZAP** is given is left exactly as it was, so you can keep working and **ZAP** again. Tutorial 059 describes the whole procedure.

2.9 Numeric literals interpretation

Current **BASE** determines how numbers are displayed or more broadly sent to output, and how they are interpreted from keyboard prompt or during compilation. The definitions **HEX** , **DECIMAL** and **BINARY** respectively change current **BASE** to hexadecimal (base sixteen), decimal (base ten) and binary (base two). The **NUMBER** interpreter accepts an optional prefix “$”, “#” or “%” to temporarily modify the **BASE** for the number being interpreted, “$” for hexadecimal, “#” for decimal, “%” for binary. The side effect is that $ and % alone are somehow equivalent to zero as long as they aren't defined by you; that's not the case of **#** since it is a definition on its own. Octal (base eight) hasn't a prefix, but see **OCTAL** definition. **BINARY** and **OCTAL** are available via **NEEDS.**

The “unary-sign” character is interpreted after any base-prefix “$”, “#” or “%”, so that **#-33** is good, but **-#33** is wrong.

Double-precision Integer numbers interpretation accepts any of the following five punctuation marks: **, . / - :**

For example:

```
120,000     23:59:59    1/23/45    3.14159
```

are all legit double-precision integer.

2.10 Back-port to Microdrive and MGT and 128K

Version 1.6 – that is quite the same as current version, but without **HEAP** facility – has been back-ported to **ZX Microdrive** and **MGT** and can run in 48K or 128K real hardware. It can run within an emulator such as **Fuse** Emulator (see the following repository for any reference: https://github.com/mattsteeldue/vforth).

Due to obvious hardware limitations, there is no **NEEDS** definition to import semantics, instead you have to use **LOAD** from the appropriate Block, and this is why the first 150 Blocks or so are reserved to keep the many utilities loaded during **AUTOEXEC** (see Screen# 13) such as Line-Editor, Full-screen Editor, SEE Debugger, Case-Of structure, Search utility, Graphics, Interrupt Handler, Assembler vocabulary.

For example, to run one of the demos, you have to type **600 LOAD** instead of **INCLUDE demo/chomp-chomp.f**

To run this Forth system within an emulator, you have to pick one that supports ZX Microdrive and/or DISCiPLE disk drive, such **Fuse** (https://fuse-emulator.sourceforge.net) that works well under Windows and Linux.

There are two zip files available:

- **vForth16m_8Microdrives_20260101.zip** that contains **eight** .mdr files.
- **vForth16m_DISCiPLE_20260101.zip** that contains **two** .img files.

suitable to be used within Fuse Emulator. I still own a ZX Interface1 and a single ZX Microdrive unit and a dozen cartriges, but my real hardware has not been tested since years... but see next section for a possible real hardware stage.

2.10.1 ZX Microdrive version using Fuse Emulator

This version uses 8 ZX Microdrive units chained together to offer **1778** blocks half-KB each (889 Kbytes total). Emulator such as FUSE provides such 8 ZX Microdrive units: unit number one is used to keep the system loader and other utilities, while the remaining seven units are used to store Blocks. This Forth system uses a low-level direct access to sectors, so that the "!Blocks" text-file appears to vForth system as a single file spread across seven cartridges.

To run under Windows you can use Fuse and to spare some time at start-up, you can specify the switches to enable ZX Interface 1 and insert eight Microdrive cartridges. N.B. The caret (^) is used to split the command in many lines.

```
start fuse.exe ^
      --interface1 ^
      --microdrive-file M1.MDR   ^
      --microdrive-2-file M2.MDR ^
      --microdrive-3-file M3.MDR ^
      --microdrive-4-file M4.MDR ^
      --microdrive-5-file M5.MDR ^
      --microdrive-6-file M6.MDR ^
      --microdrive-7-file M7.MDR ^
      --microdrive-8-file M8.MDR
```

Once the Spectrum shows the copyright message, you are able to type the classic **RUN** to load the "run" Basic loader.

This Forth system was born and run on my 48K for years, but to effectively run on real hardware with a single Microdrive Unit, you need to use "**run_HW**" Basic loader instead of the usual "**run**" loader. That loader prompts you to switch cartridges, removing the "Programs" cartridge and inserting the "Blocks" one, and awaits a key-press. To achieve such result, the "Blocks" cartridge must be prepared beforehand using the Basic program "Tap2Mdr.bas" (available in M1.MDR cartridge-file) that reads from tape file !Blocks7.TAP four string-array to be transferred to a single text file that fills the whole cartridge. Usually, such a transfer program stops with "Microdrive full" message after 160 or 170 blocks, depending on the real capacity of a cartridge. At this point the real-hardware single-unit system is ready to run. In particular, the TAP file content was produced by "Mdr2Tap.bas" Basic program that exploits – in Basic – the same technique to achieve a "Random R/W Access" from/to a single text file "!Blocks" present in all seven cartridge-files M2.MDR ... M8.MDR

2.10.2 DISCiPLE version using Fuse Emulator

This version needs two disks unit, the first for DOS system and vForth itself, and the second for data storage to offer **1560** Blocks / Screens (780 KBytes). Again, Fuse emulator works fine.

To spare some time you can specify the suitable switches

```
start fuse.exe ^
      --disciple ^
      --discipledisk Forth1.IMG
```

To start v-Forth system, you have to load the Basic loader, usually **LOAD P6** would be fine. But, I'm not aware of a switch Fuse provides to insert the second floppy disk image at start-up, and you have to select Forth2.IMG via usual Menu bar. If you don't insert the second disk image you'll get an error message "NO DISK in drive" and must redo from start.

2.11 128K memory and later models

Microdrive and MGT versions can work in 128K hardware too, but at startup you aren't able to access upper memory pages. 128K hardware provides a way to fit at $C000 one of eight 16K RAM pages, but attention must be paid since pages #2 and #5 are always fitted at $8000 and $4000 respectively, and it's good not to tamper with these two pages. At startup, page #0 is fitted.

To enable access to RAM pages fitted at address $C000 you have to re-locate a few things using definitions available in Screen# 220. Once you type **#220 LOAD** two new definitions will be available, namely **UP** and **DOWN: DOWN** moves important system parts below $C000 and **UP** restore everything above.

	UP	**DOWN**
LIMIT @ U.	FF58	C000
FIRST @ U.	F340	B9F4
R0 @ U.	F2F0	B398
TIB @ U.	F250	B2F8
S0 @ U.	F250	B2F8

Once "moved down" important parts of the system, the upper 16K RAM page at $C000 is free and you can use **BANK!** to fit any page. **BANK!** is available after you type **#88 LOAD**. The value in system variable address $5B5C (decimal 23388) on the ZX Spectrum 128K and later models stores the last value written to the memory paging I/O port $7FFD. **BANK@** definition provides simple access to such location. **BANK!** is defined in machine-code as follows and it preserve interrupt status (enabled/disabled)

```
      ldar
      di
      ex   af,af'
      exx
      pop  bc
      ld   hl,5B5C
      ld   a,(hl)
      and  $F8
      or   c
      ld   bc,$7FFD
      out  (c),a
      ld   (hl),a
      exx
      ex   af,af
      jp   po,end
      ei
      jp   (ix)
```

For example:

```
220 LOAD DOWN          \ now 16 pages can be fitted at C000
88 LOAD                \ make available BANK@ and BANK!
BANK@ .                \ should display current page i.e. 0
0 BANK! 3 $C000 !      \ store 3 at $C000 current page 0
1 BANK! 4 $C000 !      \ store 4 at $C000 current page 1
BANK@ .                \ should display current page i.e. 1
```

```
0 BANK! $C000 ?       \ displays 3, i.e. the value stored in page 0.
1 BANK! $C000 ?       \ displays 4, i.e. the value stored in page 1.
```

2.12 Definitions grouped by category

Here is my personal classification of most definitions available in this system.

2.12.1 Comments

Block oriented	Line oriented	No-operation
(...)	\ ...	NOOP

2.12.2 Stack manipulation

Broadly speaking, a *Calculator-Stack* entry is a 16-bits number (§ 4.2) i.e. a **CELL**, while a Double-precision Integer value is a 32-bits number (§ 4.3) which needs two CELLS in the Calculator-Stack, the higher significant part on top of stack. *Return-Stack* is used on entering-exiting phase of a definition *and also* to keep track of **DO** – **LOOP** index and limit. *Floating-Point-Stack* is the standard ZX Spectrum floating-point Calculator Stack that is accessible after loading the Floating-Point-Option (§ 3.9). See "Technical specifications" (§ Errore: sorgente del riferimento non trovata) for more details.

Single Cell	Double cells	Return Stack	Stack Inspection	Floating point Stack
DUP OVER DROP SWAP ROT -ROT NIP TUCK PICK ROLL ?DUP and -DUP	2DUP 2OVER 2DROP 2SWAP 2ROT	>R R> R@ I' DUP>R R>DROP	DEPTH .S	>F F> FOP

2.12.3 Comparison

Comparison involves the top element or the two top elements available on the Calculator Stack. Double-precision Integers (§ 4.3), formed by two elements each, involves twice the elements, obviously.

Zero comparison	Signed	Unsigned	Double-precision
0= 0< 0> NOT	= < > <> MIN MAX	U<	D0= D< D= DU<

2.12.4 Output

Any output is sent to video by default, but the actual device depends on which Stream is chosen via **SELECT**.

Single-integer definition Stack Value	Double-precision integer stack value and Floating point Stack	String	Other
. .R ? U.	D. D.R F.	." .(.C SPACE SPACES EMIT EMITC INVV TRUV MARK MESSAGE TYPE	SPLASH CLS CR DEVICE SELECT

2.12.5 Integer Arithmetics

Normally all definitions act upon 16-bits (signed or unsigned) integers (§ 4.2).
Definitions that act upon 32-bits integers (§ 4.3) have names that begin with **D** for *double-precision*.
Mixed definitions, that involve both 16-bits and 32-bits integers begin with **M** for *mixed*.

Arithmetics	Signed / Unsigned	Double-Precision	Constants
+ - * / /MOD MOD 2/ 2*	+- ABS NEGATE UM/MOD UM*	D+ D- DNEGATE DABS D+- S>D	0 1 2 3 -1 PI

Mixed	Bitwise	Increment/Decre-ment	Other
M+ M* M/ M/MOD */ */MOD	AND OR XOR NOT RSHIFT LSHIFT FLIP INVERT SPLIT UPPER	1+ 2+ CELL+ 1- 2- CELL-	FM/MOD SM/MOD RANDOM RND RANDOMIZE DSQRT

2.12.6 Memory

Normally all definitions act upon 16-bits (signed or unsigned) integers (§ 4.2).
Definitions that act upon 32-bits integers (§ 4.3) have names that begin with **2** for *two-cells*.

Store & Fetch	Memory chunks	Pointers & Variables	8K RAM Paging
! @ 2! 2@ C! C@ +! TOGGLE TO (used with VALUE) +TO	FILL ERASE BLANK CMOVE CMOVE> PAD BUFFER BLOCK CELL CELLS ALIGNED DUMP	RP@ RP! SP@ SP! S0 R0 USE PREV	S" HEAP FAR POINTER H" +" +C >FAR <FAR HEAP-INIT HEAP-DONE SKIP-PAGE

2.12.7 Flow control

Counted Loop	Unconted Loop	Conditionals	System related	Exception handling
DO ?DO LOOP +LOOP LEAVE I I' J K WITHIN	BEGIN WHILE REPEAT UNTIL or END AGAIN BACK RECURSE UNLOOP	IF THEN or ENDIF ELSE CASE ENDCASE OF ENDOF EXEC: **-?EXECUTE**	BYE AUTOEXEC COLD WARM ABORT CALL# EXECUTE QUIT BASIC	CATCH THROW ERROR ABORT ABORT"

2.12.8 I/O and Hardware

I/O Ports	HW Registers	Keyboard	Other
P! P@	REG! REG@ MMU7! MMU7@	?TERMINAL KEY CURS WAIT-KEY	LOAD2BLOCK LOAD-BYTES SAVE-BYTES

2.12.9 Definition related

Creators	Status & Variables	Compilation / Interpretation	Dictionary Allocation
: :NONAME CREATE VARIABLE CONSTANT CODE EXIT DOES> <BUILDS USER VALUE	?COMP ?CSP ?ERROR ?EXEC ?LOADING ?PAIRS ?STACK STATE CSP !CSP	COMPILE [COMPILE] [CHAR] ['] POSTOPNE [ ] EVALUATE INTERPRET ;	ALLOT , C, ." .(COMPILE, LITERAL DLITERAL

2.12.10 BLOCK / Screen related

Block & Buffer	Input	Block-file primitives	Variables & Constants
.LINE BLOCK EMPTY-BUFFERS FLUSH INDEX LIST UPDATE OPEN< SCREEN-TO-FILE SCREEN-FROM-FILE USE	LOAD WHERE --> QUERY ACCEPT ENCLOSE CHAR EXPECT WHERE LOCATE GREP BSEARCH	BLK-INIT BLK-READ BLK-SEEK BLK-WRITE	TIB FIRST LIMIT SOURCE-ID BLK >IN OUT SCR OFFSET BLK-FH BLK-FNAME #SEC #BUFF SPAN B/BUF B/SCR C/L

2.12.11 Numbers & strings

Number to string	Base	Interpretation	Variables
<# # #S #> SIGN HOLD	BASE HEX DECIMAL BINARY OCTAL	NUMBER (NUMBER) (SGN)	NMODE HLD DPL FLD PLACE EXP

2.12.12 Dictionary related

Input Stream	Vocabulary manipulation	Definition data	Variables & Constants
' -FIND INCLUDE MARKER NEEDS CASEOFF CASEON WORDS	FORTH DEFINITIONS ASSEMBLER SMUDGE RENAME FORGET ALIGN ID.	CFA PFA NFA LFA <NAME >BODY TRAVERSE .WORD	WIDTH WARNING FENCE DP VOC-LINK CONTEXT CURRENT BL

2.12.13 Editor

Screen oriented	Line oriented	Memory	File oriented
EDIT B N L SAVE WIPE	H D RE INS S E	.PAD P -MOVE TEXT LINE UNUSED ROOM	LED LED-EDIT LED-SAVE LED-FILE

2.12.14 NextZXOS

File hook	Directory hook	+3DOS hook	Other
F_SEEK F_CLOSE F_SYNC F_FGETPOS F_READ F_WRITE F_OPEN F_FSTAT TOUCH UNLINK	F_OPENDIR F_READDIR DIR PWD CD	M_P3DOS	R# LP HANDLER ZAP VIEW

2.13 Known bugs, workarounds, and improvement needed

INCLUDE The **INCLUDE**d source text file *must end with an empty line*, otherwise the system crashes usually showing some vertical grid. **NEEDS** suffers the same bug since it uses **INCLUDE**.
Fixed in build 2026-10-19: the **INCLUDE**d source text file no longer needs to end with an empty line.

NEEDS In case of interpretation/compilation error, the file/handle remains open and you have to manually close it using something like **2 F_CLOSE DROP** and you cannot use **REMOUNT** until then.
The same problem arises using other disk related definitions.

LOAD In some cases, interpretation of long structure via **LOAD** cannot cope with **BLOCK**s boundaries within the same Screen. This means, for example, that you cannot start an **ENUMERATED** structure in the first **BLOCK** of a Screen and continue it in the next one.

A 'nul' character (0x00) inside a Screen is completely invisible, but is the cause of many programmer headache because it provokes a sudden stop during **LOAD**. Within a Screen, you can locate such characters using **EDIT** (§ 3.1.1) that shows the ASCII code of character at cursor position.



In addition, loading from Screen# 0 crashes the system.

OPEN< At the moment, this definition can be used only in interpretation mode.

LED Pressing [BREAK] will stop any I/O operations: if not correctly used, may produce data-loss.

MOUSE Using dot-version, mouse facility may not work properly.

3 Utilities

3.1 The Editors

Before anything else, you need a way to write and edit your own Forth code. The most immediate and direct way to do so in vForth is still the classic Block/Screen facility: type a definition straight into a Screen, **LOAD** it, see it work (or fail and then you have to use **WHERE)** on the spot, edit, reload. Screens# 20 to 99 are deliberately left empty for exactly this purpose -- your own utilities and experiments -- so pick one and start typing. Whatever else this chapter introduces -- **LED** for real files, or the historical Line Editor -- there is no escaping the **BLOCK**/Screen editor itself: sooner or later, everyone ends up back in **EDIT**. Also gathered here, for convenience, are the Block Search and Locate utilities -- **LOCATE, GREP, BSEARCH** -- handy companions once your Screens start piling up and you need to find your way back through them.

3.1.1 The Full Screen Editor Utility – Screen oriented

The **EDIT** definition is available after you type: **NEEDS EDIT** (or in the old way **190 LOAD** if the source it is still there and you didn't reused these Screens).

On this Forth system, as in many others, a Screen has 1,024 bytes of data spread in 16 lines, 64 bytes each.

This "Full Screen Editor Utility" is invoked using the **EDIT** definition that enters a simple page-editor that allows modifying the *current Screen*, i.e. the one contained in **SCR** variable. During **EDIT**, you are allowed move to the next Screen or to the previous one using the commands explained below.

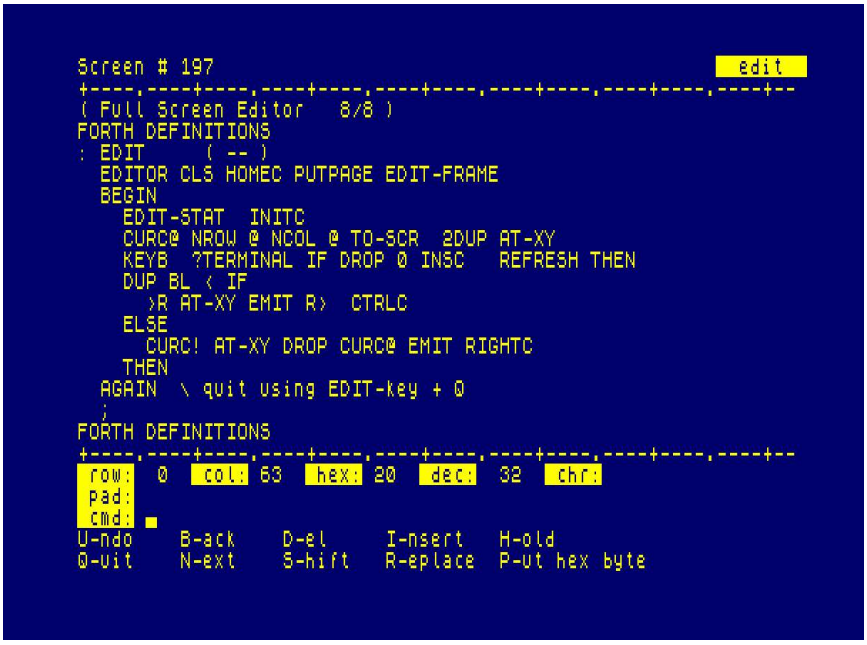
Remember: to *quit* the **EDIT** phase, you have to use [**Edit**] key followed by **[Q]** key, in a way that mimics Unix **vi** editor.

This editor works only if the display-mode allows 64 character per line at least.

EDIT **---**

For example, to select, show and edit **Screen # 197** you can type:

```
197  LIST          ( to set 197 the "current screen" )
EDIT               ( to enter the editor on "current screen" )
```



This picture shows a header reporting the Screen number and a line-ruler followed by 16 lines that make up the Screen itself. A flashing cursor is visible at home position: The cursor has two flashing mode to distinguish **CAPS LOCK** enabled – the higher flashing cursor – (▉) or disabled – the half flashing cursor – (▄) .

The cursor keys, or the equivalent **[Shift]** key + **5 / 6 / 7 / 8** keys or using standard PC keyboard, allow the flashing cursor to be moved across the screen to point the current position inside the Screen, so text can be typed at any position on the Screen.

Current cursor positions (**row** number and **col**umn number) are shown on the bottom status bar along with current character, **dec**imal ASCII code and **hex**adecimal code of it.

Pad line shows the current **PAD** content. Line oriented commands handle and work with **PAD**. See the "Line Editor" chapter (§3.1.3). When **PAD** contains garbage, the whole screen may become corrupted: in this case you can type **[Edit]** + **H** to copy the current line to **PAD** that should fix that issue.

After the **[Edit]** key (**Shift + 1** using standard PC keyboard) the Editor recognizes the subsequent single or double key-stroke commands:

> **[Edit] , Q** : **Q**uit **EDIT** Utility
> **[Edit] , U** : **U**ndo, that is re-read current screen from disk ignoring any modification done since last **FLUSH**. This feature is quite important, since it does for the current Screen buffers what **EMPTY-BUFFERS** in general.
> **[Edit] , H** : take (or **H**old) current line content and keep it in **PAD**
> **[Edit] , R** : **R**eplace current line with the current **PAD** content.
> **[Edit] , S** : make **S**pace at current cursor position shifting lower lines down; last line will be lost.
> **[Edit] , D** : **D**elete current line shifting up lower line, but a copy is copied to **PAD** before deletion, like **H**
> **[Edit] , I** : **I**nsert at current cursor line position the content of **PAD**: it does commands **S** and **R**.
> **[Edit] , N** : go to **N**ext screen
> **[Edit] , B** : go **B**ack to previous screen
> **[Edit] , P** : accepts *two hexadecimal digits* representing a byte and **P**ut it at cursor position. This way, non-printable characters, i.e. ASCII code between 0 and 31 ($00 - $1F) or graphics characters (above $7F), can be stored inside a Screen, but care must be paid to avoid corrupting the display because most of them are *control characters* and some of them are interpreted during display. Characters with ASCII code between $80 and $FF can be stored in a Screen, but they are emitted to video translated to the corresponding codes between $00 and $7F.

Any other key has no meaning and return the flashing cursor back to its position.

[Delete] (that is **Caps-Shift + ZERO**) removes a character at current cursor position, shifting left the rest of the line.

[Break] (that is **Caps-Shift + SPACE**) inserts a space at current cursor position, shifting right the rest of the line.

[Caps-Lock] (that is **Caps-Shift + 2)** accounts for a keystroke, but it is interpreted by the system to change the Caps-Lock state switching between enabled (▉) and disabled (▄).

Beware, any modification you make – even for mistake – immediately affects the underlying buffer, so if you mess things too much so that **[Edit] + U** is not enough, there is only a way to recover it, to quit before it's too late and invoke the definition **EMPTY-BUFFERS** to blank all buffers without flushing them to disk.

This "Full Screen Editor" is a work-in-progress and can be improved if needed either acting on Screens between 190 and 197 or editing **./lib/edit.f** source file.

SCREEN-FROM-FILE n ---

Available after **NEEDS SCREEN-FROM-FILE**. Import a single Screen from file.

Used in the form

```
n SCREEN-FROM-FILE filename.f
```

SCREEN-TO-FILE n ---

Available after **NEEDS SCREEN-TO-FILE**. Export a single Screen to file.

Used in the form

```
n SCREEN-TO-FILE filename.f
```

For your and the whole system safety, a "NextZXOS Write error" is issued if the specified file already exists: if you really need to, you have to `UNLINK` it in advance.

3.1.2 LED – the Large file Editor

Source text-files can be edited directly within vForth environment using **LED** – the Large file EDitor – that handles text files up to 17.568 rows, 85 characters each. The **LED** definition is available after you type: **NEEDS LED** .

LED asks NextZXOS for as many 8K-pages are needed to keep the text file in RAM, and an "Out-of-memory" error is issued if a page is not available, for instance because you already used some BANK command from Basic.

Along with **LED** you often need **DIR**.

After you type **LED** you enter a simple full-screen editor that can modify current file one screen at a time. While within **LED**, you are allowed move to next page or to previous page using the command explained below.

Remember: to ***quit*** **LED** editor, you have to use [**Edit**] key followed by [**Q**] key.

Remember: to ***save*** the file, you have to use [**Edit**] key followed by [**W**] key.

LED uses **BUFFER**s and interactions between **LED** and **EDIT** may produce inexplicable "No such block" errors. Just give an **EMPTY-BUFFERS** to restore a pristine **BUFFER**s state. This editor works only while the display-mode is **LAYER 1,2**.

LED --- cccc

Available after **NEEDS LED**. Used in the form

```
LED cccc
```

opens specified file `cccc` and enters **LED** editor. For example **LED lib/dir.f**
This editor inherits many of its commands and behavior from the previously described **EDIT** editor except that is has 85 characters per line instead of 64. See previous paragraph (§3.1.1) for details..

Along **LED** command, some more sub-commands are available to better handle a text-file.

LED-EDIT ---

Used in the form

```
LED-EDIT
```

re-enters the **LED** editor after you quit it to continue editing the same file you previously opened that should still be in upper 8k RAM pages, provided you haven't corrupted its content in some way.

LED-SAVE ---

Used in the form

```
LED-SAVE
```

saves back the file you previously open in **LED** editor, using the current filename you already specified using **LED** or **LED-FILE**. The same function is invoked during **LED** editing via [EDIT]-W.

LED-FILE --- cccc

Used in the form

```
LED-FILE cccc
```

modify the filename that **LED-SAVE** will write to. This allow to save to a different filename.

TOUCH --- cccc

Used in the form

```
TOUCH cccc
```

Accept the followin string cccc as a filename and update file-timestamp. If the file doesn't exist, it's created at zero length. Available after `NEEDS TOUCH`.

UNLINK --- cccc

Used in the form

```
UNLINK cccc
```

Accept the following string as a filename and remove it from disk, Beware, there is no way to recovery.
~~At this moment, you must specify the drive e.g. **unlink c:dummy.f** and despite it could be deemed a bug, I keep this behavior to improve security and avoid unwanted destructive operations.~~ Available after `NEEDS UNLINK`.

3.1.3 Line Editor

The following definitions are available after you include the **EDITOR** vocabulary via **NEEDS EDITOR** or **90 LOAD** using the old fashion way. Most logic shown in this section is used by **LED** "The Large file Editor".

The Line Editor has a dozen definitions that can operate on a single line of a given Screen and helps inspect things around.

An edit session normally starts with a **LIST** on the desired Screen, this sets SCR user variable to the passed Screen number. **LIST** is a definition already available in the "core" dictionary. To clear a Screen I foreseen a **BCLEAR** definition, but I left it commented somewehere for now, deeming it too dangerous for my tastes; instead I usually use **BCOPY** from an actually empty Screen. You may type **NEEDS BCOPY.**

The definition **FLUSH** flushes to disk any modification you've done on any Screen. Beware, a Screen is re-written to disk as soon as the **BUFFER** containing it are modified.

EMPTY-BUFFERS is another vital definition: it empties all buffers. It is very useful if you mistakenly overwrite or spoil a Screen during an edit operation, with it, you have the chance to "rollback" the things before anything is written to disk.

To write a line from scratch or to overwrite line, you can use P to "put" the following text to the given line on current screen. For example:

```
1000 LIST
0 P \ One thousand screens
L
```

This sequence selects Screen# 1,000 and put a text "\ One thousand screens" on the first line of it. The definition L repeats the LIST of current screen.

To move or copy a line around, you can use H to "hold in PAD" a given line on current screen, you can change Screen if you wish, then you can complete this **copy-and-paste** operation with INS to "insert" or RE to "replace" the line you copied in advance with H. None of above definitions, but H, modify PAD content, so you can repeat the operation. There is also a way to **cut-and-paste** a line using D to "delete and copy to PAD" instead of H.

See also **BLOCK**, **BUFFER**, **INDEX**, **L/SCR**, **LIST**, **LOAD**, **MESSAGE**, **PAD**, **SCR**, **STRM**, **TIB.**

This is a quick reference of involved memory areas and definitions that work on them.

Text Input Buffer (keyboard) **TIB**	Parsing Operation	 **PAD**	Edit Operations	One **BLOCK BUFFER**	Blanking Operations
	TEXT →		**← H RE →**		**← E**
			← D INS →		**← S**
			P →		

-MOVE a n ---

"Line move". It moves a line, **C/L** bytes length, from address a to the line n of current screen, then it does an **UPDATE**. Current Screen is the one kept by **SCR**.

.PAD ---

"Show PAD". It prints the current **PAD** content assuming it contains a counted-string.

B ---

"Back" one Screen. This definition set to previous Screen by decreasing **SCR** and prints it using **LIST**.

D n ---

"Delete" a row. It deletes line n of current Screen (the one indicated by **SCR**), the following lines are moved up and the last one will be blanked. D executes H so that it can be followed by an **INS** to perform a line move.

BCOPY n1 n2 ---

"Block-Copy" utility that copies Screen n1 to Screen n2. **SCR** will contain n2. This definition's standard name is **COPY**, but I deemed too close to the Basic keyword **COPY**.

E n ---

"Erase" a row. This definition fills line n with spaces. It does **UPDATE**.

H n ---

"Hold" a row in **PAD**. This definition put line n of current Screen to PAD without altering the block on disk. Current Screen is the one kept in **SCR**.

INS n ---

"Insert" from **PAD**. This definition inserts line n using text in **PAD**. The original line n and the following ones are moved down and the last is lost.

L ---

"List" current Screen. This definition does **SCR @ LIST**.

LINE n --- a

Leaves the address a of line n of current screen, the one kept in **SCR**. Such a screen is currently held in a buffer.

N ---

"Next" Screen. This definition sets to next Screen by increasing **SCR** and prints it using **LIST**.

P n ---

"Put" a line. This definition accepts the following text (delimited by a tilde character ~) as the text of line n of current

Screen. Text is taken from **TIB** and sent to the current Screen

RE **n** **---**
"Replace". This definition takes text currently in **PAD** and put it to line n.

S **n** **---**
"Space" one row. This definition frees line n moving the following lines down by one. The last line is lost

SAVE **---**
Does **UPDATE** and **FLUSH** saving this Screen and all previously modified Screens back to disk.

ROOM **---**
This definition shows the room available in the dictionary, that is the difference between **SP@** and **PAD** addresses.

TEXT **c** **---**
This definition accepts the following text and stores it to **PAD**. c is a text delimiter. **TEXT** does not go beyond a 0x00 [null] ASCII.

UNUSED **---** **n**
It returns the number of byte available in dictionary.

WHERE **n1** **n2** **---**
Usually executed after an error has been reported during a **LOAD** phase. Maybe, this definition should be included in the "core" dictionary. n1 is the value of **>IN** and n2 the value of **BLK** as were left by **ERROR**. This way, **WHERE** shows on screen the block number, the line number, the very same line highlighting in "inverse video" the definition that caused the error. When invoked after an error during the loading via **NEEDS** or **INCLUDE**, then the result is poor, because it always reports the row #0 of block #1 due to the way these two definitions are coded.

WIPE **---**
Set content of current Screen to blanks.

3.1.4 Block Search and Locate Utility

This group of definitions allow you to look for text within the Screens / Blocks and are available after you type alternatively:

NEEDS LOCATE or
NEEDS GREP or
NEEDS BSEARCH or

All of them compare text through **COMPARE**, which is part of the core dictionary since build 2026-10-28 (see par. 6.1).

LOCATE ---

Used in the form

LOCATE cccc

this definition examines all Screens between 1 and 1000 looking for the definition of **cccc** showing the Screen where the first occurrence is found, then it makes it the "current screen", just like **LIST** for example:

LOCATE EDIT

takes a few seconds to search in which Screen **COMPARE** is defined, and if found it shows the Screen using **LIST**.

```
Scr# 196
  0 ( Full Screen Editor   7/7 )
  1 : EDIT     ( -- )
  2   CLS HOMEC PUTPAGE EDIT-FRAME
  3   BEGIN
  4     EDIT-STAT  INITC
  5     CURC@ NROW @ NCOL @ TO-SCR  2DUP AT-XY
  6     KEY  ?TERMINAL IF DROP 0 INSC   REFRESH THEN
  7     DUP BL < IF
  8       >R AT-XY EMIT R>  CTRLC
  9     ELSE
 10       CURC! AT-XY DROP CURC@ EMIT RIGHTC
 11     THEN
 12   AGAIN  \ quit using EDIT-key + 0
 13   ;
 14
 15
```

GREP ---

Used in the form

GREP cccc

this definition examines all Screens between 1 and 2000 looking for any occurrence of string **cccc** showing them in a table form, for example

GREP EDIT

will take some more time to complete and gives something like the following



BSEARCH **n1 n2** **---**

Used in the form

n1 n2 BSEARCH cccc

this definition examines all Screens between n1 and n2 looking for any occurrence of definition **cccc** showing them in a table form. This definition is used by **GREP** that in fact is defined as **1 2000 BSEARCH** .

3.2 Graphics mode and Layer facility

The following definitions are available after you type **NEEDS GRAPHICS**.
To forget this library from dictionary you can type **NO-GRAPHICS** or **FORGET GRAPHICS**.
This library is still work-in-progress because I'm improving it every so often.

This library give access to the larger Layer 2 mode 320 w x 256 h pixels, 256 colors total, one color per pixel.

The ZX Spectrum Next's machine can handle several Graphic-Modes and vForth is able to use almos all of them.

In all the following definitions, the x-coordinate is the vertical distance from the top-left corner of the grid, the y-coordinate is the horizontal distance from the top-left corner of the grid.

A few demos are avaliable to demonstrate the capabilities of each Layer 2 mode.

```
demo/brot.f
demo/color-picker.f
```

LAYER! **n** **---**

This definition changes Graphic-Mode. The parameter `n` can be one of the following values:

- **00** to switch to **Layer 0** - Standard Spectrum (ULA) mode, 256 w x 192 h pixels, 8 colors total (2 intensities), 32 x 24 cells, 2 colors per cell. Equivalent to Basic's LAYER 0.
- **10** to switch to **Layer 1,0** - LoRes (Enhanced ULA) mode, 128 w x 96 h pixels, 256 colors total, 1 color per pixel. Equivalent to Basic's LAYER 1,0.
- **11** to switch to **Layer 1,1** – Standard Res (Enhanced ULA) mode, 256 w x 192 h pixels, 256 colors total, 32 x 24 cells, 2 colors per cell. Equivalent to Basic's LAYER 1,1.
- **12** to switch to **Layer 1,2** – Timex HiRes (Enhanced ULA) mode, 512 w x 192 h pixels, 256 colors total, only 2 colors on whole screen. Equivalent to Basic's LAYER 1,2.
- **13** to switch to **Layer 1,3** – Timex HiColour (Enhanced ULA) mode, 256 w x 192 h pixels, 256 colors total, 32 x 192 cells, 2 colors per cell. Equivalent to Basic's LAYER 1,3.
- **20** to switch to **Layer 2** – 256 w x 192 h pixels, 256 colors total, one color per pixel. Equivalent to Basic's LAYER 2,1. For greater resolution 320 w x 256 h, see **LAYER2+** and **LAYER22** below.

To ease of use, this definition accepts `n` which can be expressed both in **DECIMAL** or in **HEX**, the same way, and since there is no ambiguity, the following two lines gives the same result

```
HEX     12 LAYER!
DECIMAL 12 LAYER!
```

This primitive definition *just* changes Graphics-Mode without any other side effect. Instead, the hereby definitions **LAYER0**, **LAYER10**, **LAYER11**, **LAYER12**, **LAYER13** and **LAYER20** also modify the overall behavior of some other graphics definitions.

For completeness, these LAYER related words came alogn **GRAPHICS** library, but for simplicity the same are available as standalone words via **NEEDS**, in which case they don't modify the overall behavior.

The following table shows the peculiarity of each of six Graphics-Mode when applicable

	L0	L11	L12	L13	L10	L2
Char-Size	8	4	8	4	4	4
V-RANGE	0C0	0C0	0C0	0C0	060	0C0
H-RANGE	100	100	200	100	080	100
PIXELADD	L0	L0	L12	L0	L10	L2
POINT	L0	L0	L0	L0	L1	L1
PLOT	L0	L0	L0	L0	L1	L2
XPLOT	L0	L0	L0	L0	L1	L2
PIXELATT	L0	L0	na	L13	na	L2
XY-RATIO	1	1	2/	1	1	1
EDGE	=	=	=	=	L1	L1

LAYER0 ---

Set screen mode to Standard ULA, legacy ZX Spectrum mode, also set the characters to 4 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution.

LAYER10 ---

Set screen mode to LoRes mode, set the characters to 4 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution.

LAYER11 ---

Set screen mode to Standard Res (Enhanced ULA) mode, set the characters to 4 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution.

LAYER12 ---

Set screen mode to Timex HiRes (Enhanced ULA) mode, set the characters to 8 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution, for example, the correct aspect-ratio for **CIRCLE** is enforced.

LAYER13 ---

Set screen mode to Timex HiColour (Enhanced ULA) mode, set the characters to 4 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution.

LAYER2 ---

Set screen mode to Layer 2, set the characters to 4 pixel wide, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution.

LAYER2+ **---**

Experimental : Set screen mode to Layer 2 with larger resolution 320 w x 256 h pixels, 256 colors total, one color per pixel, and modify the overall behavior of all graphics definitions to work with this specific pixel resolution. The coordinates appear swapped. See LAYER22.

IDE_MODE! **n** **---**

This is a primitive definition to switch Graphic-Mode via NextZXOS using **M_P3DOS** call $01D5
It's called by **LAYER!** that prepares n in a suitable way. This definition is also available via NEEDS **IDE_MODE!**

```
LAYER0      :     $0000
LAYER10     :     $0100
LAYER11     :     $0101
LAYER12     :     $0102
LAYER13     :     $0103
LAYER2      :     $0200
```

IDE_MODE@ **--- hl de bc a**

This is a primitive definition to query current Graphic-Mode via NextZXOS using M_P3DOS call $01D5.
Usual result are:

```
LAYER       :      hl    de    bc    a  ( a in binary )
_____________________________________________________

LAYER0      :     $1620 $000F $0800 $00 ( 00 00 00 00 )
LAYER10     :     $0C20 $FF00 $0400 $01 ( 00 00 00 01 )
LAYER11     :     $1840 $000F $0400 $05 ( 00 00 01 01 )
LAYER12     :     $1840 $0000 $0800 $09 ( 00 00 10 01 )
LAYER13     :     $1840 $0038 $0400 $0D ( 00 00 11 01 )
LAYER2      :     $1840 $0038 $0400 $02 ( 00 00 00 10 )
```

3.2.1 Low-level definitions

The following definitions **PIXELADD**, **PIXELATT**, **PLOT** and **POINT** are coded in assembler for maximum performance.

ATTRIB **---** **n**

Value that specifies the byte used as color-attribute in all subsequent graphics command. To modify it you have to use the **VALUE - TO** semantics

```
DECIMAL  216  TO  ATTRIB
```

The value of **ATTRIB** is saved across any Layer switch so that each Graphic-Mode keeps its own value.

PIXELADD **x y** **---** **n**

Depenging on current Graphic-Mode, determine address of a pixel and fit MMU7 8K page if needed.

PIXELATT **x y** **---** **n**

Depenging on current Graphic-Mode, determine address of a pixel and fit MMU7 8K page if needed.

PLOT **x y** **---**

Depenging on current Graphic-Mode, plot a pixel using current **ATTRIB**.

POINT **x y** **---** **c**

Depenging on current Graphic-Mode, it returns the attribute value of a pixel.
For Layer 1,0 and Layer 2 modes this is simply the sequence **PIXELADD C@**.
For Layer 0 Layer 1,1 Layer 1,2 and Layer 1,3 modes, this definition returns a true-flag if the pixel is set or a false-flag if the pixel is unset.

3.2.2 High-level definitions

DRAW-CIRCLE x y r ---

Draw a circle with center at `x y` and radius `r` using the current **ATTRIB** color and Graphic-Mode. As stated above, x-coordinate is the vertical and y-coordinate is horizontal. This definition does not use

DRAW-LINE x0 y0 x1 y1 ---

Draw a line from `x1 y1` to `x0 y0` using the current **ATTRIB** color and Graphic-Mode. As stated above, x-coordinate is the vertical and y-coordinate is horizontal.

PLOT x y ---

Draw a pixel at `x y` using the current **ATTRIB** color and Graphic-Mode. As stated above, x-coordinate is the vertical and y-coordinate is horizontal.

PAINT x y ---

Flood fill via Scanline Span Fill (Smith 1979). An explicit seed stack keeps the return-stack depth constant, so large areas cannot overflow the return stack as recursion would. The seed stack lives in a dedicated 8K page of expansion RAM, reserved from NextZXOS.

3.3 Colors & Attributes

Here is a set of definition that invoke the Standard-ROM routines to change the screen colors. All these definitions end with a dot (.) to spacify that it works via `EMIT` and to avoid confusion with other definitions.

.BORDER **b** **---**
Immediately set the current BORDER color. It uses ROM routine $2297 via **CALL#**.

.BRIGHT **b** **---**
Depending on on current Graphics Mode, set the current BRIGHT attribute for any subsequent output operations.

.FLASH **b** **---**
Depending on on current Graphics Mode, set the current BRIGHT attribute for any subsequent output operations.

.INK **b** **---**
Depending on on current Graphics Mode, set the current INK color for any subsequent output operations.

.INVERSE **b** **---**
Depending on on current Graphics Mode, set the current INVERSE attribute for any subsequent output operations.

.OVER **b** **---**
Depending on on current Graphics Mode, set the current OVER attribute for any subsequent output operations.

.PAPER **b** **---**
Depending on on current Graphics Mode, set the current PAPER color for any subsequent output operations.

3.4 LAYERxx-GRAPHICS - the modular alternative to GRAPHICS

Everything described in the previous paragraphs is also available in a modular form, split into one file per Graphic-Mode plus a shared core:

```
GRAPHICS-COMMON   – color constants, LAYER!, coordinate check, the vectored
                    (DEFER) primitives PIXELADD / PIXELATT / PLOT / XPLOT /
                    POINT / EDGE / XY-RATIO, the LAYER: defining word, and
                    the layer-independent high-level words DRAW-LINE,
                    CIRCLE, PAINT, .INK, .PAPER, .BORDER and the rest of
                    the color-attribute set.

LAYER0-GRAPHICS   – Layer 0, Standard Spectrum (ULA) mode
LAYER10-GRAPHICS  – Layer 1,0, LoRes (Enhanced ULA) mode
LAYER11-GRAPHICS  – Layer 1,1, Standard Res (Enhanced ULA) mode
LAYER12-GRAPHICS  – Layer 1,2, Timex HiRes (Enhanced ULA) mode
LAYER13-GRAPHICS  – Layer 1,3, Timex HiColour (Enhanced ULA) mode
LAYER2-GRAPHICS   – Layer 2, 256 w x 192 h pixels, 256 colors
LAYER22-GRAPHICS  – Layer 2 high resolution, 320 w x 256 h pixels, 256 colors
LAYER24-GRAPHICS  – Layer 2 high resolution, 640 w x 256 h pixels, 16 colors
```

Each `LAYERxx-GRAPHICS` file automatically pulls in `GRAPHICS-COMMON` and immediately activates its mode, so a program only ever loads the modes it actually uses:

```
NEEDS LAYER2-GRAPHICS       \ pulls in GRAPHICS-COMMON, switches to Layer 2
```

This is the key difference from the monolithic `GRAPHICS` library since `NEEDS GRAPHICS` loads every mode and every low-level definition for all six classic Graphic-Modes in one block, whereas `NEEDS LAYERxx-GRAPHICS` loads only the one mode named, plus the shared core. On a memory-constrained system like our is, this can save a substantial amount of dictionary space; it also keeps a program's `NEEDS` list self-documenting about which modes it actually relies on.

Once loaded, every high-level word behaves exactly as documented in the above paragraphs: `PLOT`, `POINT`, `PIXELADD`, `PIXELATT`, `DRAW-LINE`, `CIRCLE`, `PAINT`, `ATTRIB`, `.INK`, `.PAPER`, `.BORDER`, `.BRIGHT`, `.FLASH`, `.INVERSE`, `.OVER` and the `LAYER!` mode selector are all identical, because they are the same shared definitions in `GRAPHICS-COMMON` - only the low-level `PIXELADD`/`PLOT`/`POINT`/`XPLOT`/ `EDGE`/`XY-RATIO` implementations, and the resolution (`V-RANGE`/`H-RANGE`), differ per mode. A program is free to load more than one `LAYERxx-GRAPHICS` module and switch between them at run time with `LAYER0`, `LAYER10`, `LAYER11`, `LAYER12`, `LAYER13` or `LAYER2` - the same defining words described in 3.2, now living one per file instead of all together.

To switch to Layer 2 **320x256** or Layer 2 **640x256** (see below), use the `LAYER22` or `LAYER24` word defined by the corresponding module; these two modes are not reachable through the numeric `LAYER!` selector, since they require an extra NextReg $70 resolution switch on top of the base Layer 2 mode 20.

Unloading a single mode does not disturb the others or the shared core, provided the correct forgetting order is respected:

```
NO-LAYER2-GRAPHICS          \ or FORGET LAYER2-GRAPHICS
                            \ removes only the Layer 2 mode

NO-GRAPHICS-COMMON          \ or FORGET GRAPHICS-COMMON
                            \ removes the shared core (and with it every
                            \ LAYERxx-GRAPHICS module built on top of it)
```

`LAYER22-GRAPHICS` and `LAYER24-GRAPHICS` – high resolution Layer 2

These two modules extend Layer 2 beyond the classic 256 w x 192 h, one-colour- per-pixel mode, using 80K of RAM (five 16K banks = ten 8K MMU7 pages) laid out in vertical bands of 32 `(LAYER22)` or 32 byte- `(LAYER24)` columns each:

- `LAYER22-GRAPHICS`: 320 w x 256 h pixels, 256 colors, one colour per pixel.
- `LAYER24-GRAPHICS`: 640 w x 256 h pixels, 16 colors, 4 bits per pixel (two pixels packed per byte). Byte layout is identical to LAYER22; only the pixel addressing halves the horizontal coordinate and PLOT/POINT/XPLOT work on a single nibble. The "4" in the name recalls 4bpp / 16 colours.

In both modes the coordinate convention of the rest of this library is preserved: x is vertical (0..255), y is horizontal (0..319 for LAYER22, 0..639 for LAYER24); PLOT/POINT/... take (x y) exactly as in every other Graphic-Mode.

Warning: both modes assume the five 16K banks starting at the Layer 2 active bank (NextReg $12) are free. NextZXOS allocates only three banks (48K) for the standard 256x192 Layer 2, so the two extra banks must not be in use by anything else (BASIC BANK allocations, other libraries, ...).

Warning: `LAYER24-GRAPHICS` is experimental. It has been verified in the headless emulator but shows an unexpected 256-pixel horizontal shift under CSpect; it has not yet been verified on real hardware. `LAYER22-GRAPHICS` does not show this issue.

3.5 Filesystem

3.5.1 Filesystem navigation words

vForth's interactive filesystem-navigation words fall into two families that share a common purpose -- moving around and listing the NextZXOS filesystem from the prompt -- but rest on two independent, unrelated NextZXOS services. **CD, PWD, MAKEDIR** and **REMOVEDIR** are thin wrappers around the single core primitive **IDE_PATH** (lib/IDE_PATH.f), which drives the **M_P3DOS** $01B1 service; all four share **IDE_PATH**'s (**PARSE-PATH**) helper to turn the word following them on the input line into a $FF-terminated pathspec. **DIR** (lib/DIR.f) and **WILDCARD** (inc/wildcard.f) instead drive the **F_OPENDIR/F_READDIR** pair, which expects NUL-terminated names and applies its own wildcard filtering at the NextZXOS level. The two families share no code -- not even the pathname-parsing helper, since one needs a $FF terminator and the other a $00 -- and are grouped here only because, together, they form the complete filesystem-navigation toolkit a user reaches for at the prompt: **PWD/CD** to see and change location, **DIR/WILDCARD** to list its contents, **MAKEDIR/REMOVEDIR** to shape it.

CD **---**

Available after **NEEDS CD**. Change current directory. Used in the form

```
CD  xxx
```

Warning #1 : changing to a different directory impede any subsequent use of **NEEDS** until current directory is restored to its default `c:/tools/vforth`.
Warning # 2 : be careful when using hexadecimal base, since CD may be interpreted as a hex literal.

DIR **---**

Available after **NEEDS DIR**. Used in the form

```
DIR  xxx
```

it displays the content of directory **c:xxx**
For long output, the scrolling can be stopped by holding pressed the [EDIT] key.

DOSVER **---**

Print current NextZXOS version. Available after `NEEDS DOSVER`.

MAKEDIR **---**

Available after **NEEDS MAKEDIR**. Used in the form

```
MAKEDIR  xxx
```

create a new directory **c:xxx**
E.g. to create a new “example” directory **MAKEDIR example**

PWD **---**

Available after `NEEDS PWD`. Prints current directory.

REMOVEDIR **---**

Available after **NEEDS REMOVEDIR**. Used in the form

```
REMOVEDIR  xxx
```

removes directory **c:xxx**
E.g. to remove the previously created directory, **REMOVEDIR example**

WILDCARD ---
Available after **NEEDS WILDCARD**. Set the filter wildcard for any subsequent **DIR** operation. Used in the form

```
WILDCARD  xxx
```

Wildcard accepts ? and * as "wildcard" characters: ? pairs a single character; * pairs any number of characters.
Giving **WILDCARD** alone resets to default *.*

3.5.2 Loading and saving bytes

LOAD-BYTES a n ---
Available after NEEDS LOAD-BYTES. Load n bytes to address a reading from filename which spec is held in PAD. This is just a wrapper around low-level definitions and errors are issued in case of I/O problems. PAD content is volatile. Typical usage is:

```
PAD" test.bin"      ( the space is needed but is not part of filename )
<address> <size> LOAD-BYTES
```

See also: SAVE-BYTES

LOAD2BLOCK n ---
Available after NEEDS LOAD2BLOCK. Load up to 448 bytes to specified BLOCK number n reading from filename which spec is held in PAD. This is just a wrapper around low-level definitions and errors are issued in case of I/O problems. PAD content is volatile. WARNING: In this implementation a Screen occupies two BLOCKs so for example **Screen# 440** is BLOCK **880**. Typical usage is:

```
PAD" test.bin"      ( the space is needed but is not part of filename )
<blocknumber> LOAD2BLOCK
```

See also: LOAD-BYTES

SAVE-BYTES a n ---
Available after NEEDS SAVE-BYTES. Saves n bytes from address a writing filename which spec is held in PAD. It creates new a new file, error if it already exists. PAD content is volatile.
Typical usage is:

```
PAD" test.bin"      ( the space is needed but is not part of filename )
<address> <size> SAVE-BYTES
```

See also: LOAD-BYTES

SAVE-BYTES-FORCE a n ---
Same as SAVE-BYTES, but overwrites the file if it already exists.

3.5.3 Low-level definitions

vForth gives Forth programs direct, first-class access to the real filesystem exposed by NextZXOS. The **F_xxx** words — **F_OPEN, F_CLOSE, F_READ, F_WRITE, F_SEEK, F_OPENDIR, F_READDIR, F_MKDIR, F_RMDIR, F_UNLINK, F_RENAME** and the rest — are thin **CODE** wrappers around the NextZXOS API, invoked through the same RST $08 dispatch NextBASIC itself uses. Every operation works on ordinary files and directories on the SD card or disk, addressed by their usual pathname, and returns a handle exactly as NextZXOS does.

This is deliberately independent of vForth's own **BLOCK/Screen** persistence, which lives entirely inside the single ! Blocks-64.bin file. A small set of interactive convenience words — **CD, DIR, PWD, TOUCH, UNLINK** — build on the **F_xxx** primitives to give the Ok prompt shell-like file management; **LED,** the Large file Editor, uses the same handles to edit any text file directly, without going through a Block.

F_CHDIR a --- f

Given a null-terminated path `a`, changes the working directory of the default drive ('*') to it, unless a includes a drive letter. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A9 service number. This definition is available after `NEEDS F_CHDIR`.

F_CHMOD a1 attr mask -- f

Given a null-terminated filespec `a1`, modify its attributes. `attr` is the bitmap of attribute values, `mask` is the bitmap of which attributes to change (1=change, 0=leave alone). Bitmasks are any combination of:

```
A_WRITE     %00000001
A_READ      %10000000
A_RDWR      %10000001
A_HIDDEN    %00000010
A_SYSTEM    %00000100
A_ARCH      %00100000
```

return 0 on success, True flag on error (F_CHMOD via RST 08 hook $AF)

F_CLOSE fh --- f

NextZXOS primitive: it closes a file handle `fh` previously opened with `F_OPEN` or `F_OPENDIR`. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $9B service number.

F_FGETPOS fh --- d f

NextZXOS primitive: given an open file handle `fh` returns the position `d`. Flag `f` is 0 for OK.

F_FTRUNCATE fh d --- f

Given an open file handle `fh` and a new size `d` (double), truncates or extends the file to exactly d bytes; extended parts are zero-filled unless `M_SETCAPS` has disabled it. The file position is unaffected. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A2 service number. This definition is available after `NEEDS F_FTRUNCATE`.

F_GETCWD buf --- f

Given a buffer address `buf`, stores the current working directory of the default drive as a null-terminated path. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A8 service number. This definition is available after `NEEDS F_GETCWD`.

F_GETLINE a n1 fh --- n2

Given a filehandle **fh**, reads the next line (terminated with $0D or $0A) into the buffer at address a, which must be n1 bytes long (at least 3), and returns n2 as the length of the line plus one (the terminator), or 0 at the end of the file. At most n1 - 2 characters are read: the line is left at a padded with blanks, and the last byte of the buffer is set to 0x00. A longer line is split, and the next call returns the rest. A last line without its terminator is read like any other. This definition is used by F_INCLUDE.

F_INCLUDE fh ---

Given an open file-handle **fh**, this definition includes the source from file. This definition is used by INCLUDE and NEEDS.

F_MKDIR a --- f

Given a null-terminated path a, creates a directory. Drive defaults to '*' (current drive) unless a includes a drive letter. Flag f is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $AA service number. This definition is available after NEEDS F_MKDIR.

F_OPEN a1 a2 b --- fh f

NextZXOS primitive: it opens a file using a filespec given at address a1 and returns filehandle number n, n1 is "mode" as specified in "NextZXOS and esxDOS APIs" standard documentation. Integer fh is the filehandle. Flag f is 0 for OK
It uses an RST 8 call followed by $9A service number. See also F_CLOSE.
Parameters in details as per standard documentation *NextZXOS and esxDOS APIs (Updated 24 May 2023*):

a1 (filespec) is a null-terminated string, such as produced by **,"** or **S"** definitions.
a2 is address to an 8-byte header data used in some cases.
b is access mode-byte, that is a combination of any/all of:

esx_mode_read	$01 request read access
esx_mode_write	$02 request write access
esx_mode_use_header	$40 read/write +3DOS header

plus one of:

esx_mode_open_exist	$00 only open existing file
esx_mode_open_creat	$08 open existing or create file
esx_mode_creat_noexist	$04 create new file, error if exists
esx_mode_creat_trunc	$0C create new file, delete existing

F_OPENDIR a --- fh f

NextZXOS primitive: given a z-string address, open a file-handle to the directory. Integer fh is the filehandle. Flag f is 0 for OK.

F_READ a u1 fh --- u2 f

NextZXOS primitive: it reads at most u1 bytes from file handle fh and stores them at address a. Returns u2 as the actual bytes read. Flag f is 0 for OK. It uses RST 8 call followed by $9D service number.

F_READDIR a1 a2 fh --- u f

NextZXOS primitive: Given a pad address a1, a filter z-string a2 and a file-handle fh obtained via F_OPENDIR fetch the next available entry of the directory. Return 1 as ok or 0 to signal end of data then return 0 on success, True flag on error

F_RENAME src dest --- f

Given two null-terminated filespecs `src` and `dest`, renames/moves the file. Only the base file is renamed, not any associated +3DOS metadata file (use M_P3DOS's DOS_RENAME call for that). Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $B0 service number. This definition is available after `NEEDS F_RENAME`.

F_REWINDDIR fh --- f

Given an open directory handle `fh`, rewinds its position back to the start of the directory. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A7 service number. This definition is available after `NEEDS F_REWINDDIR`.

F_RMDIR a --- f

Given a null-terminated path `a`, removes a directory. Drive defaults to '*' (current drive) unless `a` includes a drive letter. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $AB service number. This definition is available after `NEEDS F_RMDIR`.

F_SEEKDIR hf d --- f

Given an open directory handle `hf` and a double-cell offset `d` as previously returned by `F_TELLDIR`, sets the current directory position. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A6 service number. This definition is available after `NEEDS F_SEEKDIR`.

F_TELLDIR hf --- d f

Given an open directory handle `h`, returns its current position as a double-cell offset `d`, usable with `F_SEEKDIR`. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $A5 service number. This definition is available after `NEEDS F_TELLDIR`.

F_TRUNCATE a d --- f

Given a null-terminated filespec `a` (unopened file) and a new size d (double), truncates or extends the file to exactly `d` bytes; extended parts are zero-filled unless `M_SETCAPS` has disabled it. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $AE service number. This definition is available after `NEEDS F_TRUNCATE`.

F_RMDIR a --- f

Given a null-terminated path `a`, removes a directory. Drive defaults to '*' (current drive) unless `a` includes a drive letter. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $AB service number. This definition is available after `NEEDS F_RMDIR`.

F_SEEK d fh --- f

NextZXOS primitive: it seeks position `d` at open file given by filehandle `fh`. It uses an RST 8 call followed by $9F service number. Flag `f` is 0 for OK.

F_SYNC fh --- f

NextZXOS primitive: it syncs to disk open file given by filehandle `fh`. It uses an RST 8 call followed by $9C service number. Flag `f` is 0 for OK.

F_UNLINK a --- f

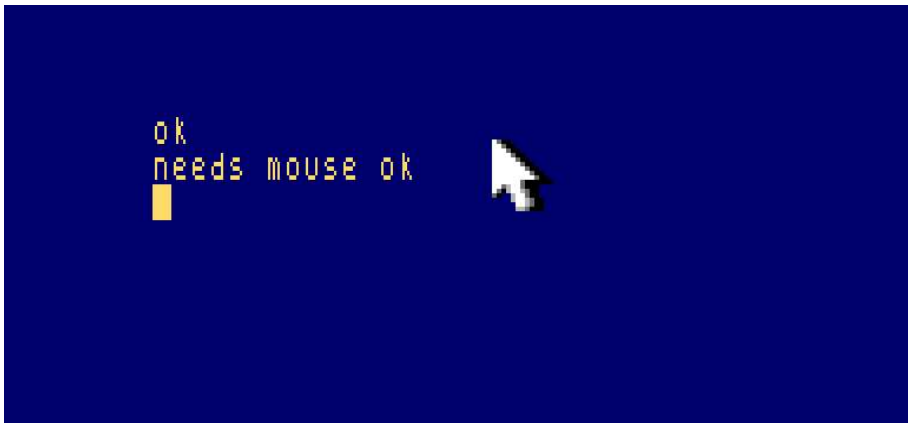
Given a null-terminated filespec `a`, deletes the file. Drive defaults to '*' (current drive) unless a includes a drive letter. Only the base file is deleted, not any associated +3DOS metadata file. Flag `f` is 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $AD service number. This definition is available after `NEEDS F_UNLINK`. See also `UNLINK`, a friendlier wrapper that parses a filename from the input stream.

M_DOSVERSION --- ver language-code

Returns the NextZXOS API version ver (BCD, e.g. $0208 for v2.08) and a packed language-code pair lang (e.g. the two characters of "en" or "es"); decode both with SPLIT. This is just a wrapper around NextZXOS API via RST 8 call followed by $88 service number. This definition is available after `NEEDS M_DOSVERSION`. See also `DOSVER`, which prints the version directly.

3.6 Mouse

An *interrupt-driven mouse facility* is made available via **NEEDS MOUSE** (which requires **INTERRUPTS** described in Chapter 3.7).



Sprite #0 is the white shadowed-arrow: its definition is given directly in Forth code, as follows (this, by the way, shows how easily Forth language can perform such a task):

```
HEX  14 REG@ CONSTANT E3 \ Global Transparency Colour
:  "  00  C,  ;  \ Black
:  |  6D  C,  ;  \ Dark-Grey
:  v  B6  C,  ;  \ Light-Gray
:  M  FF  C,  ;  \ White
:  _  E3  C,  ;  \ Transparency

\ Semi-graphical mouse-face definition 10x8-pixels arrow
CREATE MOUSE-FACE
\ 0 1 2 3 4 5 6 7 8 9 A B C D E F \
\ ------------------------------ \
  M | " " _ _ _ _ _ _ _ _ _ _ _ _ \  0
  M M | " " _ _ _ _ _ _ _ _ _ _ _ \  1
  M M M | " " _ _ _ _ _ _ _ _ _ _ \  2
  M M M M | " " _ _ _ _ _ _ _ _ _ \  3
  M M M M M | " " _ _ _ _ _ _ _ _ \  4
  M M M M M M | " " _ _ _ _ _ _ _ \  5
  M M M M M M M | " " _ _ _ _ _ _ \  6
  M M M M M M M M | " " _ _ _ _ _ \  7
  M M M M M M M M M | " " _ _ _ _ \  8
  M M M M M M | " " " " " " _ _ _ \  9
  M M | v M M | " " " _ _ _ _ _ _ \  A
  M | " v M M v | " " _ _ _ _ _ _ \  B
  | " _ _ v M M | " " _ _ _ _ _ _ \  C
  _ _ _ _ v M M v | " " _ _ _ _ _ \  D
  _ _ _ _ _ M M v | " " _ _ _ _ _ \  E
  _ _ _ _ _ v v | " " " _ _ _ _ _ \  F
```

The hardware is polled every 20 ms (during interrupt) and data is processed as follows:

1. Three bytes are collected as “raw-data” from three hardware ports:
   - **$FFDF** (Kempston mouse Y vertical) stored in **MOUSE-RX** variable (result is multiplied by 256).
   - **$FBDF** (Kempston mouse X horizontal) stored in **MOUSE-RY** variable (result multiplied by 256).
   - **$FADF** (Kempston mouse Wheel andButtons) stored in **MOUSE-RS**
     - three lower bits are decoded as follow:
       bit 0: right button, zero when pressed, one when released.
       bit 1: left button, zero when pressed, one when released.
       bit 2: wheel click-down event, zero when pressed, one when released.
     - the four higher bits contain the current wheel position among sixteen different cyclical position coded between 0 and 15.

2. Current raw-data (at time t) is compared against the previous ones (at time t-1) revealing signed deltas:
   - MOUSE-DX ← $MOUSE\text{-}RX_{t} - MOUSE\text{-}RX_{t-1}$
   - MOUSE-DY ← $MOUSE\text{-}RY_{t} - MOUSE\text{-}RY_{t-1}$
   - MOUSE-DS ← $MOUSE\text{-}RS_{t} - MOUSE\text{-}RS_{t-1}$

3. If delta x or y (MOUSE-DX , MOUSE-DY) are non-zero, compute new mouse position and move the corresponding sprite, clipping within screen range:
   - **MOUSE-X** : vertical distance from top-left corner
   - **MOUSE-Y** : horizontal distance from top-left corner

4. If delta-buttons (MOUSE-DS) is non-zero, the following event-flags are provided in **MOUSE-S**:
   $0000 : no event
   $0001 : right button click-down
   $0002 : left button click-down
   $0004 : wheel button click-down
   $0010 : wheel forward direction
   $0100 : right button click-up
   $0200 : left button click-up
   $0400 : wheel button click-up
   $1000 : wheel backward direction
   Events are OR-ed together and kept persistent in **MOUSE-S** until they're consumed via **MOUSE** that reset it to zero.

The mouse position and status can be inspected using the following definitions:

MOUSE-XY --- n1 n2

Returns the current mouse position in pixel

n1 : vertical distance from top-left corner in pixel. Range is 0 – 319.
n2 : horizontal distance from top-left corner in pixel. . Range is 0 – 255.

?MOUSE --- f

Returns a true-flag if there is a mouse-click event.

MOUSE --- n

Collects and consumes the latest “persistent” click-events values and reset **MOUSE-S** to zero. The event(s) is (are) reported as shown in point 4 above.

NO-MOUSE ---

Disable interrupts and remove mouse facility restoring the system as it was before **NEEDS MOUSE** was given.

BYE ---

The original definition is kept, but a new **BYE** definition hides the mouse-arrow and disable interrupts before exiting. Re-entering Forth, the mouse will stay disable. You have to type **ISR-ON 1 MOUSE!** To re-enable it.

3.7 Interrupt Service Routine

After you type **NEEDS INTERRUPTS** a few new definitions will be loaded in memory along with some low-level definitions that allow setting-up an Interrupt-Driven definition: The ISR must be a single definition suitably defined. This is a standard IM 2 interrupt routine implementation. In the future, I hope to be able to exploit the new Next's IM 2 interrupt vector mode. Programming an Interrupt Service Routine using Forth itself is tricky and if not correctly coded, it can impair the system or cause a system-crash. As said, this library still does not exploit the new ZX Spectrum Next interrupt vector, but hope this will be soon implemented: this means that the all these definitions listed here below will go under a deep overhaul when I'll code it, but my aim is to keep backward compatibility as much as possible.

A Z80's maskable interrupt occurs every 20 ms (50 times per second), and when it occurs, a CALL to a specific routine is performed. In vForth we use IM 2 interrupt mode by preparing a 257 bytes vector-table at $6200, filled with $63, so that the interrupt service routine is located at $6363 to jump to the suitable code – i.e. the definition **ISR-SUB** – that makes possible to a Forth definition to be executed as interrupt.

First, **ISR-SUB** performs a RST $38 to fulfill the legacy ISR, then it must save the whole *Forth machine status* by pushing to stack the value of CPU registers and then saving Forth's Return-Stack-Pointer and Calculator-Stack-Pointer.
Second, it prepares Forth virtual registers (Calculator-Stack Pointer, Return-Stack-Pointer and Instruction-Pointer) to execute the xt contained in **ISR-XT** variable, then a jump to the Inner-interpreter via JP (IX) is performed. Interrupts stay disabled during the execution of such a xt.

After the xt contained in **ISR-XT** is executed, the **ISR-RET** definition is executed restoring back the machine status by retrieving Calculator-Stack Pointer, Return-Stack-Pointer and Instruction-Pointer and then popping all CPU registers before returning from the interrupt routine and re-enabling interrupts.

It's worth to be noticed that, since an interrupt may occur in the middle of the execution of any part of Forth system, not everything can be performed within an interrupt service routine, and care must be put to avoid critical interference with the main program, such as trying to write system variables or invoking peculiar definitions that are known to modify the code they're going to execute, such as **CASEON**, **CASEOFF**, or memory areas such as most Floating-Point operations.

ISR-OFF **---**
Disable Interrupt Utility by restoring IM 1 and I register to its default $3F value.

ISR-XT **xt** **---**
Variable that contains the **xt** of the definition that will be executed in background at each Interrupt. It is always followed by the execution of **ISR-RET** so that **ISR-XT** can be viewed as the pointer to an anonymous definition that contains two definitions: the *interrupt-service-routine* definition and the *return-from-interrupt* definition.

ISR-ON **---**
Enable *Interrupt Service Utility*: This definition prepares “IM 2 Vector Table” at address $6200-$6300 filling it with all $63 and set Interrupt Mode 2, so that when an Interrupt is issued a CALL to address $6363 is performed.

At address $6363 is a jump to address of **ISR-SUB** body i.e. **[' ISR-SUB >BODY]** and it's used in the form

```
ISR-OFF
' <your-isr-word>  ISR-XT  !
ISR-ON
```

Then, **<your-isr-word>** is executed in background at each Interrupt.

During an Interrupt, Forth uses a separate Calculator Stack (4 bytes below current SP) and a separate Return Stack located

at $6330. Care must be paid to avoid any critical interference with the normal *foreground* Forth execution.

Typical usage is to control some Sprite movement or poll *mouse* and *joystick*, some demos are available.

The following example keeps the display filled with evenly spaced dots in Layer 1,1 or Layer 1,2 modes.

```
: ISR-WORD

      $80 $57FF C!
      $5701 $5700 $0FF CMOVE>
      $5700 $4700 $100 CMOVE
      $4700 $4F00 $100 CMOVE
;
ISR-OFF
' ISR-WORD ISR-XT !
ISR-ON
```

ISR-EI **---**

Low-level “enable interrupt”. It actually executes an EI opcode.

ISR-DI **---**

Low-level “disable interrupt”. It actually executes a DI opcode.

ISR-IM1 **---**

Low-level “interrupt mode 1”. It actually executes an IM 1 opcode. This is the default *mode* for any ZX Spectrum.

INT-IM2 **---**

Low-level “interrupt mode 2”. It actually executes an IM 2 opcode. It relies on a “vector table” located

ISR-SYNC **---**

Low-level “sync to video”. It actually executes an HALT opcode to force the machine to wait until the next interrupt.

SETIREG **b** **---**

Low level Z80 register I setting. It actually executes an LD I, A opcode.

ISR-RET **---**

Low-level “return from interrupt” definition. It restores all registers and returns control to Forth foreground execution.

ISR-SUB **---**

Low-level “interrupt service routine” definition. It saves all registers and gives control to `INT-XT` background definition execution. Interrupt SP is initialized at 4 bytes below current SP. Interrupt RP is initialized at $6330 and allows room for 14 cells.

3.8 Debugger Utility

The following definitions are available after you type **NEEDS SEE** or usually after a regular **AUTOEXEC**.

Also, this section exposes in detail how definitions are stored in dictionary memory. See §4.6 Dictionary memory structure for more reference.

SEE ---

Used in the form

```
SEE cccc
```

it prints how definition `cccc` is defined along with its NFA, LFA, CFA, PFA information.

If `cccc` is a regular colon-definition, the result will show something very close to the original source code the definition was coded from.

For example, the definition **TYPE** is a colon-definition that emits to the current device a **n** bytes-long string stored at address **a**, it's defined as follow:

```
: TYPE   ( a n -- )
    BOUNDS              \ turn  a,n  into  a+n,a  to cope with ?DO
    ?DO                 \ loop between  a  and  a+n  (non-inclusive)
        I C@ EMIT       \ emit one character stored at address a+i
    LOOP ;
```

If you type

```
SEE TYPE
```

the system will show something like this:

```
Nfa: E7D0 84
Lfa: E7D5 7C7 CURS
Cfa: 6F9C 6A23
BOUNDS (?DO) 12 I C@ EMIT (LOOP) -8 EXIT ok
```

The first line shows **TYPE** **N**ame **F**ield **A**ddress (**$E7D0**) followed by **$84** that is the counter byte of a 4-bytes length name. The counter byte always has the most significant bit set, that is **$80** added to **$04** giving **$84**.

The second line is the **L**ink **F**ield **A**ddress (**$E7D5**) which holds a *heap-pointer* **$07C7** to **CURS**'s NFA that in this case is the previous definition in the dictionary.

These two fields (NFA and LFA) are stored above address $E000 (i.e. MMU7) and they are effectively stored in one of the extended 8K-RAM pages used as HEAP.

The third line is the **C**ode **F**ield **A**ddress (**$6F9C**) that contains the actual machine code run, which in this case is a "CALL" to the ENTER routine of every colon-definition, located at **$6A23**.

The fourth line represents the **P**arameter **F**ield **A**ddress and, in this case, is in some way a definition *decompilation* but literals and offsets are shown in "inverse video". For example the number **-8** after **(LOOP)** is the "displacement" to where the Instruction Pointer has to jump to go back to next iteration, that is 4 cells backward. In this example **(?DO)** and **(LOOP)** are the *compiled counterpart* of **?DO** and **LOOP** that in fact normally won't be *compiled by themselves,* instead they control the compilation of some other definitions.

Another example: **NIP** definition that removes the second element of Stack, isn't a colon-definition, but a low-level definition coded in machine-code as follow (this is an example of use of the **ASSEMBLER** built-in vocabulary available via **NEEDS ASSEMBLER**)

```
CODE NIP  ( n1 n2 -- n2 )
        POP       HL|     \  pop  hl
        EX(SP)HL          \  ex   (sp), hl
        Next              \  jp   (ix)
        C;
```

and if you type

```
SEE NIP
```

you'll see

```
Nfa: E2F4 83
Lfa: E2F8 2EB DROP
Cfa: 6906 DDE3
6906  E1 E3 DD E9  03 03 E1 F1  ac]i  aq
690E  E5 F5 E5 DD  E9 0C 03 E1  eue]i  a
```

In this case, since **NIP** is a low-level definition, the PFA part is shown as a hexadecimal dump: it's a real machine-code routine. Again, the first line shows **NIP**'s NFA (**$E2F4** in this case) and **$83**, the count-byte, that indicates a 3-bytes length definition name. The second line is **NIP**'s LFA (**$E2F8**) that contains a *heap-pointer* **02EB** to **DROP**'s NFA, that is the previous definition in dictionary. The third line is **NIP**'s CFA (**$6906**) which content contains the machine-code routine itself. Examining the dump you should be able to locate **E1** for POP HL, **E3** for EX(SP),HL and **DD E9** for JP(IX) to the inner interpreter address that is compiled by **NEXT** Assembler definition (§3.10).
The bytes that follow – **03 03 E1 F1** – are the beginning of the subsequent definition compiled in dictionary (**TUCK** in this case): 03 03 is **TUCK**'s Mirror, i.e. the heap-pointer $0303 to its CFA. Try **$0303 FAR 8 DUMP** to inspect the HEAP: you'll see **TUCK**'s xt $690C followed by **SWAP**'s NFA.

Last example is the definition **IF** a colon-definition that compiles a conditional branching in the program flow, defined as follows:

```
: IF    ( -- a 2 ) \ compile-time
    COMPILE 0BRANCH
    HERE 0 , 2 ; IMMEDIATE
```

with the following output:

```
Nfa: EC69 C2
Lfa: EC6C C60 BACK
Cfa: 8059 6A23
COMPILE  0BRANCH  HERE 0 , 2 EXIT ok
```

In this case, since **IF** is an **IMMEDIATE** definition, the NFA length-byte is **$C2** instead **$82**.

3.8.1 The Inner-interpreter

At the very core level of any Forth system lies the *Inner Interpreter*, that needs a few information to keep the system itself alive, namely the *Instruction Pointer*, the *Calculator Stack Pointer* and the *Return Stack Pointer*.

The latest version of vForth keeps such fundamental status information in some permanently dedicated Z80 registers:

- **BC** The *Instruction Pointer* that points to the current "xt" being executed
- **SP** the *Data Stack Pointer* that points to the "top-of-stack" element
- **DE** the *Return Stack Pointer* used to track sub-routines calls and keep some value of IP.
- **IX** the address of the *Inner Interpreter* routine, aka "NEXT".

Any other register, such as A, F, H, L and all alternate registers A', F', B', C', D', E', H', L', are free to be used by the low-level definitions.

```
NEXT:
    ld      a, (bc) ; fetch the xt pointed by Instruction Pointer (bc).
    inc     bc
    ld      l, a
    ld      a, (bc)
    inc     bc      ; IP is incremented to point the following xt.
    ld      h, a
    jp      (hl)    ; jump to xt address also known as "CFA"
```

```
CFA:                ; "CFA" contains the actual machine-code to be executed
    call    ENTER   ; and it can be simply a CALL to the piece of code peculiar
                    ; to that kind of definition.
                    ; For example a "colon-definition" has a CALL to ENTER
PFA: ...
```

```
ENTER:
    ex      de, hl  ; put Return Stack Pointer (de) to hl register
    ld      (hl), b ; and save the current value of Instruction Pointer (bc)
    inc     hl
    ld      (hl), c
    inc     hl
    ex      de, hl  ;
    pop     bc      ; Instruction Pointer now contains PFA
    jp      (ix)    ; jump back to NEXT
```

DUMP **a u ---**

Performs a "dump display" of a memory area from address `a` for `u` bytes or until **[Break]** is pressed. The value of `u` is always rounded to the nearest greater multiple of 8.

Visualization is always in hexadecimal, current base is maintained. For example:

```
DECIMAL 448 60 DUMP
```

will print the Standard ROM content starting from address 448 ($01C0) for 64 bytes, i.e. the nearest greater multiple of 8 and keeps **DECIMAL** as the current **BASE**.

```
01C0  4C 49 53 D4  4C 45 D4 50  LISTLETP
01C8  41 55 53 C5  4E 45 58 D4  AUSENEXT
01D0  50 4F 4B C5  50 52 49 4E  POKEPRIN
01D8  D4 50 4C 4F  D4 52 55 CE  TPLOTRUN
01E0  53 41 56 C5  52 41 4E 44  SAVERAND
01E8  4F 4D 49 5A  C5 49 C6 43  OMIZEIFC
01F0  4C D3 44 52  41 D7 43 4C  LSDRAWCL
01F8  45 41 D2 52  45 54 55 52  EARRETUR
```

if required outside SEE utility, you have to import via **NEEDS DUMP**.

.WORD **a ---**

Given a CFA, this definition prints the **ID.** It is used by **SEE** to perform some "decompilation".
if required outside SEE utility, you have to import via **NEEDS .WORD**.

.S **---**

Prints the current content of Calculator Stack without destroying its content.
For example, supposing to start with an empty stack,

```
0 1 2 3 .S
```

will print

```
0 1 2 3 ok
```

if required outside SEE utility, you have to import via **NEEDS .S**.

DEPTH **--- n**

It leaves the depth of the Calculator Stack before it was executed. For example, supposing to start with an empty stack,

```
0 1 2 DEPTH .
```

will print

```
3 ok
```

if required outside SEE utility, you have to import via **NEEDS DEPTH**.

3.9 Floating-Point Option

This is an simple Floating-Point Option Library that exploits the native standard ZX Spectrum Floating-Point capabilities, with some limitations and differences. This library is not yet compatible with DOT-command version.

To load this Floating-point Option Library you have to use **NEEDS FLOATING** .

To perform any floating-point operations you first need to push one or two numbers onto *Spectrum's calculator stack* using **>W** definition. then you need to call the floating-point calculator using **FOP** definition (that calls RST $28 service routine). Finally, you have to pop the result from Spectrum's calculator stack using **W>** definition.

For example, to define a definition that returns the value of *pi* you can code something like this:

```
: PI
  [  1 0 >W 36 FOP  \ atan(1)
     4 0 >W 04 FOP  \ *4
     W>  ]  DLITERAL
;
```

A floating point in Spectrum's calculator stack takes 5 bytes, instead in Forth Calculator Stack it takes 4 bytes only i.e. the same as a Double-Precision Integer". This means there is some **precision loss**: Maybe in the future I'll be able to fix this fact. But we can verify for instance what difference is between the integer approximation 355/113 and **pi**

```
355.0 113.0 F/  PI  F-  F.
0.2384e-6 ok
```

The real deviation is **0.26686e-6**, this means there are still six precision digits to count on.

Thinking a floating-number stored in CPU four registers HLDE, the sign is the msb of H, so you can check for sign in the integer-way. 128 plus the exponent is stored in the following 8 bits of HL and the **significand**/**mantissa** is stored the remaining bits of HL and 16 bits of DE. The fifth byte of a standard Spectrum floating-point number is then defaulted to a fixed value.

If a floating-number is an integer between 0 and 65535, then it is kept on stack the same as a double-precision integer. To verify this fact you can type.

```
FLOATING  DECIMAL  65535.0  65537.0  .S
```

that displays

```
65535  0  128  18560
```

where the two single precision integer 65535 and 0 are the representation of integer 65535.0, while the two integers 128 and 18560 are the internal bit-representation of 65537.0

The integer on TOS always keeps the sign information of the floating-number.

Most definitions described in this chapter are defined using **CREATE** ... **DOES>** technique.

Actually, there is little use of floating-number in this Forth system, but to be used during development to build pre-computed tables. For example, Screen# 513, available in "!Blocks-64.bin" file, shows a sine/cosine table that has been built using this library as shows in Screen #511 and #512.

3.9.1 Floating-point option activation and deactivation

To import the floating point library option you must type **NEEDS FLOATING** and then, you can use **FLOATING** to enable the floating-number interpretation and **INTEGER** to disable it and remain within the integers.

FLOATING **---**
Activate floating-point numbers mode. **NMODE** user variable is set to 1.

INTEGER **---**
Deactivate floating-point numbers mode. **NMODE** user variable is set to 0.

NO-FLOATING **---**
Discard from dictionary the Floating-Point option by executing a specific **MARKER** to restore memory and pointers to the state before **NEEDS FLOATING** were invoked.

3.9.2 Floating-point number conversion

To import the floating point library option you must type **NEEDS FLOATING** and then, you can use **FLOATING** to enable the floating-numbe

D>F **d** **---** **fp**
Convert a double-precision integer into a floating-number. See **F>D**.

F>D **fp** **---** **d**
Convert a floating-number into a double-precision integer truncating to the lower integer. It's the opposite of **D>F**.

FLOAT **n** **---** **fp**
Convert a single-precision-integer into a floating-number. See **FIX.**

FIX **fp** **---** **n**
Convert a floating-number a into single-precision-integer. It's the opposite of **FLOAT.**

3.9.3 Representation and constants

F>PAD **fp** **---** **u**
The representation of floating-number fp is stored in **PAD**. The number u is the length of the string.

F.R **fp** **u** **---**
Prints fp on a field of u characters to video or current stream (See **SELECT**).

F. **fp** **---**
Prints fp to video or current stream (See **SELECT**).

1/2 **---** **fp**

Put on TOS the value 0.5.

PI **---** **fp**

Put on TOS the value of pi.

3.9.4 Arithmetics

F- **fp1 fp2** **---** **fp3**

Floating point difference: fp3 := fp1 – fp2

F+ **fp1 fp2** **---** **fp3**

Floating point addition: fp3 := fp1 + fp2

F* **fp1 fp2** **---** **fp3**

Floating point product: fp3 := fp1 * fp2

F/ **fp1 fp2** **---** **fp3**

Floating point division: fp3 := fp1 / fp2

FNEGATE **fp1** **---** **fp2**

Floating point negate, i.e. : fp1 := - fp2

FSGN **fp1** **---** **fp2**

Floating point sign. Fp2 is the sign of fp1.

FABS **fp1** **---** **fp2**

Floating point absolute value

F/MOD **fp1 fp2** **---** **fp3 fp4**

Floating point division and reminder: fp4 is the quotient of fp1 / fp2 and fp3 is the reminder.

F** **fp1 fp2** **---** **fp3**

Floating point power: fp3 := fp1 ^ fp2

FMOD **fp1 fp2** **---** **fp3**

Floating point module: fp3 := fp1 mod fp2

F*/ **fp1 fp2 fp3** **---** **fp4**

Floating point scale operation: fp4 := fp1 * fp2 / fp3 using an intermediate precision of native 5 bytes instead of 4.

F< **fp1 fp2 --- f**
Floating point comparison: f is TRUE if fp1 < fp2, FALSE otherwise.

F> **fp1 fp2 --- fp3**
Floating point comparison: f is TRUE if fp1 > fp2, FALSE otherwise.

F0< **fp1 --- f**
Floating point comparison: f is TRUE if fp1 < 0, FALSE otherwise.

F0> **fp1 --- f**
Floating point comparison: f is TRUE if fp1 > 0, FALSE otherwise.

3.9.5 Log, Exp, Trig

FLN **fp1 --- fp2**
Floating point Natural Logarithm. fp2 := ln(fp1)

FEXP **fp1 --- fp2**
Floating point Exponentation: fp2 := exp(fp1)

FINT **fp1 --- fp2**
Integer truncation. If the floating-number is an integer between 0 and 65535, then it is kept on stack the same as a double-precision integer. **`1.4 FINT`** gives 1.0 but **`-1.4 FINT`** gives -2.0

FSQRT **fp1 --- fp2**
Square root.

FSIN **fp1 --- fp2**
Sine in radians.

FCOS **fp1 --- fp2**
Cosine in radians.

FTAN **fp1 --- fp2**
Tangent in radians

FASIN **fp1 --- fp2**
Arc-sine in radians

FACOS **fp1 --- fp2**
Arc-cosine in radians

FATAN **fp1 --- fp2**
Arc-tangent in radians.

RAD>DEG **fp1 --- fp2**
Convert radians to degrees.

DEG>RAD **fp1 --- fp2**
Convert degrees to radians.

3.9.6 Low-level definitions.

FOP **n ---**
Low-level definition that invokes Floating-Point-Operation `n` .

>W **fp ---**
Takes a floating-point number `d` from Calculator Stack and put to Floating-Pointer Stack.

W> **--- fp**
Takes a floating-point number `d` from Calculator Stack and put to Floating-Pointer Stack.

3.10 ASSEMBLER vocabulary

The definitions provided by **ASSEMBLER** vocabulary are available after you type **100 LOAD** or after you include it via **NEEDS ASSEMBLER**. Then, you can list this vocabulary using **ASSEMBLER WORDS**.

It's a Zilog Z80 adaptation of the Intel 8080 assembler written by Albert van der Horst available at https://github.com/albertvanderhorst/ciasdis.

To create a new definition using this Assembler you have to use **CODE** definition. Compilation **STATE** is not modified, so usually you assemble things staying in interpret **STATE**. This means you have to pay a little more attention when you need to dereference a definition CFA using **'** (tick). Usually, a **CODE** definition should end with **NEXT** that compiles a **JP (IX)** op-code, and finally **C;** which makes the new definition available for subsequent dictionary search.

Between the starting **CODE** and the ending **C;** instructions are given using their specific op-codes followed by as many parameters as needed. The following table describes all type of arguments used by the op-code list below:

rr|	**:**	BC|	DE|	HL|	SP|	and when the case	IX|	IY|	and	AF|
r|	**:**	B|	C|	D|	E|	H|	L|	A|	and	(HL)|	**{ source registers }**
r'|	**:**	B'|	C'|	D'|	E'|	H'|	L'|	A'|	and	(HL)'|	**{ destination registers }**
f|	**:**	NZ|	Z|	NC|	CY|	PO|	PE|	P|	M|	**{ flags used by JP, CALL and RET }**	
f'|	**:**	NZ'|	Z'|	NC'|	CY'|	**{ same as above f| but used by JR }**				
b|	**:**	0|	1|	2|	3|	4|	5|	6|	7|	**{ bit number operand }**	

a|	:	00| 08| 10| 18| 20| 28| 30| 38|
d	:	byte displacement **{ used by JR }**
n	:	byte value (8 bits)
nn	:	integer value (16 bits)
aa	:	address
r	:	Next hardware-register number

COMMAER's are definitions that enforce a syntax checking while assembling op-codes and parameters.

N,	immediate 8 bit value.
NN,	immediate 16 bits value.
AA,	memory address value.
P,	port address value (16 bits) and, in **NEXTREG** op-code, Next's hardware-register number (8 bits).
D,	displacement in relative jump JR.
LH,	used by Next's **PUSHN** to compile big-endian 16-bits argument.

Other peculiar definitions are

CY| and **CY'|**	to specify “carry-flag” to be different from **C|** or **C'|** “register”.
(IX+ and **(IY+**	to begin a “index-register” argument and **)|** to close it.
(IX'+ and **(IY'+**	same as above, but for destination argument.

You can use **(IY+** operand wherever you can use **(IX+** operand.

Some single byte op-code was renamed to have a better near-Z80 notation. To avoid some Forth-Assembler name clash, it is preferred using some peculiar notation, for example **EXAFAF EX(SP)HL EXDEHL** instead of EX AF, AF' or EX (SP),HL or EX DE,HL. Also, we explicitly say A for all arithmetic/logic opcodes, e.g. **ANDA r|** instead of AND r and so on. IX and IY index-register cause most trouble because they add both a prefix and a displacement and because they can be used in conjunction with CB prefix. In this case we use some custom late-compilation definitions to fix things but relaxing some of the syntax check that the Albert's core provided. Z80N extensions are all ED-prefixed, so the follow the same way introducing a new **LH,** commaer to enforce a better syntax check.

3.10.1 Complete list of Assembler Op-Code

Here's the correspondence between Forth and original Z80 mnemonic, Z80N in **red**.

FORTH ASSEMBLER	Z80 MNEMONIC
ADCA (HL)|	ADC A, (HL)
ADCA (IY+ d)|	ADC A, (IY+d)
ADCN n N,	ADC A, n
ADCA r|	ADC A, r
ADCHL rr|	ADC HL, BC/DE/HL/SP
ADDA (HL)|	ADD A, (HL)
ADDA (IY+ d)|	ADD A, (IY+d)
ADDN n N,	ADD A, n
ADDA r|	ADD A, r
ADDHL rr|	ADD HL, BC/DE/HL/SP
ADDHL,A	**ADD HL, A**
ADDDE,A	**ADD DE, A**
ADDBC,A	**ADD BC, A**
ADDHL, nn NN,	**ADD HL, nn**
ADDDE, nn NN,	**ADD DE, nn**
ADDBC, nn NN,	**ADD BC, nn**
ADDIY rr|	ADD IY, BC/DE/IY/SP
ANDA (HL)|	AND (HL)
ANDA (IY+ d)|	AND (IY+d)
ANDN n N,	AND n
ANDA r|	AND r
BIT b| (HL)|	BIT b, (HL)
BIT b| (IY+ d)|	BIT b, (IY+d)
BIT b| r|	BIT b, r
BRLCDE,B	**BRLC DE, B**
BSLADE,B	**BSLA DE, B**
BSRADE,B	**BSRA DE, B**
BSRFDE,B	**BSRF DE, B**
BSRLDE,B	**BSRL DE, B**
CALLF f| aa AA,	CALL Z/NZ/C/NC/PO/PE/P/M, aa
CALL aa AA,	CALL aa
CCF	CCF
CPA (HL)|	CP (HL)
CPA (IY+ d)|	CP (IY+d)
CPN n N,	CP n
CPA r|	CP r
CPD	CPD
CPDR	CPDR
CPI	CPI
CPIR	CPIR
CPL	CPL
DAA	DAA
DEC (HL)'|	DEC (HL)
DEC (IY'+ d)|	DEC (IY+d)
DECX rr|	DEC BC/DE/HL/SP
DECX IX|	DEC IX
DECX IY|	DEC IY
DEC r'|	DEC r
DI	DI
DJNZ d D,	DJNZ d
EI	EI
EX(SP)HL	EX (SP), HL

EX(SP)IY	EX (SP), IY
EXAFAF	EX AF, A'F'
EXDEHL	EX DE, HL
EXX	EXX
HALT	HALT
IM0	IM 0
IM1	IM 1
IM2	IM 2
IN(C) (HL)'|	IN (c)
INA n P,	IN A, (n)
IN(C) r'|	IN r, (c)
INC (HL)'|	INC (HL)
INC (IY'+ d)|	INC (IY+d)
INCX rr|	INC BC/DE/HL/SP
INCX IX|	INC IX
INCX IY|	INC IY
INC r'|	INC r
IND	IND
INDR	INDR
INI	INI
INIR	INIR
JP(C)	**JP (C)**
JPHL	JP (HL)
JPIX	JP (IX)
JPIY	JP (IY)
JPF f| aa AA,	JP Z/NZ/NC/C/PO/PE/P/M, aa
JP aa AA,	JP aa
JRF f'| d D,	JR C/NC/Z/NZ, d
JR d D,	JR d
LD(X)A rr|	LD (BC/DE), A
LD (HL)'| r|	LD (HL), n
LDN (HL)'| n N,	LD (HL), r
LDN (IY'+ d)| n N,	LD (IY+d), n
LD(IY+ d)| r|	LD (IY+d), r
LD()A aa AA,	LD (nn), A
LD()X rr| nn AA,	LD (nn), BC/DE/SP
LD()IY aa AA,	LD (nn), IY
LD()HL aa AA,	LD (nn), HL
LDA(X) rr|	LD A, (BC/DE)
LDA() aa AA,	LD A, (aa)
LDAI	LD A, I
LDAR	LD A, R
LDX rr| nn NN,	LD BC/DE/HL/SP, nn
LDX() rr| nn AA,	LD BC/DE/SP/IY, (aa)
LDHL() aa AA,	LD HL, (aa)
LDIA	LD I, A
LDX IY| nn NN,	LD IY, nn
LDRA	LD R, A
LDSPHL	LD SP, HL
LDSPIX	LD SP, IX
LDSPIY	LD SP, IY
LD r'| (HL)|	LD r, (HL)
LD r'| (IY+ d)|	LD r, (IY+d)
LD r'| r|	LD r, r
LDN r'| n N,	LD r, n
LDD	LDD
LDDR	LDDR
LDDRX	**LDDRX**
LDDX	**LDDX**
LDI	LDI
LDIR	LDIR

LDIRX	**LDIRX**
LDIX	**LDIX**
LDPIRX	**LDPIRX**
LDWS	**LDWS**
MIRRORA	**MIRROR A**
MUL	**MUL**
NEG	NEG
NEXTREGA r P,	**NEXTREG r, A**
NEXTREG r P, n N,	**NEXTREG r, n**
NOP	NOP
ORA (HL)|	OR (HL)
ORA (IY+ d)|	OR (IY+d)
ORN n N,	OR n
ORA r|	OR r
OTDR	OTDR
OTIR	OTIR
OUT(C) (HL)'|	OUT (c), 0
OUT(C) r'|	OUT (c), r
OUTA n P,	OUT (n), A
OUTD	OUTD
OUTI	OUTI
OUTINB	**OUTINB**
PIXELAD	**PIXELAD**
PIXELDN	**PIXELDN**
POP AF|	POP AF
POP rr|	POP BC/DE/HL
POP IX|	POP IX
POP IY|	POP IY
PUSH rr|	PUSH BC/DE/HL/AF
PUSH IX|	PUSH IX
PUSH IY|	PUSH IY
PUSHN nn LH,	**PUSH nn**
RES b| (HL)|	RES b, (HL)
RES b| (IY+ d)|	RES b, (IY+d)
RES b| r|	RES b, r
RES b| r| (IY+ d)|	RES r, b, (IY+d)
RET	RET
RETF f|	RET Z/NZ/C/NC/PO/PE/P/M
RETI	RETI
RETN	RETN
RL (HL)|	RL (HL)
RL (IY+ d)|	RL (IY+d)
RL r|	RL r
RL r| (IY+ d)|	RL r, (IY+d)
RLA	RLA
RLC (HL)|	RLC (HL)
RLC (IY+ d)|	RLC (IY+d)
RLC r|	RLC r
RLC r| (IY+ d)|	RLC r, (IY+d)
RLCA	RLCA
RLD	RLD
RR (HL)|	RR (HL)
RR (IY+ d)|	RR (IY+d)
RR r|	RR r
RR r| (IY+ d)|	RR r, (IY+d)
RRA	RRA
RRC (HL)|	RRC (HL)
RRC (IY+ d)|	RRC (IY+d)
RRC r|	RRC r
RRC r| (IY+ d)|	RRC r,(IY+d)
RRCA	RRCA

```
    RRD                                     RRD
    RST           a|                        RST n
    SBCA       (HL)|                        SBC A, (HL)
    SBCA  (IY+ d )|                         SBC A, (IY+d)
    SBCN          n   N,                    SBC A, n
    SBCA          r|                        SBC A, r
    SBCHL        rr|                        SBC HL, BC/DE/HL/SP
    SCF                                     SCF
    SET           b|      (HL)|             SET b, (HL)
    SET           b| (IY+ d )|              SET b, (IY+d)
    SET           b|         r|            SET b, r
    SET           b|         r|  (IY+ d )| SET r, b, (IX+d)
    SETAE                                   SETAE
    SL1        (HL)|                        SL1 (HL)
    SL1   (IY+ d )|                         SL1 (IY+d)
    SL1           r|                        SL1 r
    SL1           r| (IY+ d )|             SL1 r, (IY+d)
    SLA        (HL)|                        SLA (HL)
    SLA   (IY+ d )|                         SLA (IY+d)
    SLA           r|                        SLA r
    SLA           r| (IY+ d )|             SLA r, (IY+d)
    SRA        (HL)|                        SRA (HL)
    SRA   (IY+ d )|                         SRA (IY+d)
    SRA           r|                        SRA r
    SRA           r| (IY+ d )|             SRA r, (IY+d)
    SRL        (HL)|                        SRL (HL)
    SRL   (IY+ d )|                         SRL (IY+d)
    SRL           r|                        SRL r
    SRL           r| (IY+ d )|             SRL r, (IY+d)
    SUBA       (HL)|                        SUB (HL)
    SUBA  (IY+ d )|                         SUB (IY+d)
    SUBN          n   N,                    SUB n
    SUBA          r|                        SUB r
    SWAPNIB                                 SWAPNIB
    TESTN         n   N,                    TEST n
    XORA       (HL)|                        XOR (HL)
    XORA  (IY+ d )|                         XOR (IY+d)
    XORN          n   N,                    XOR n
    XORA          r|                        XOR r
```

3.10.2 Assembler Example - Checksum

This definition calculates the checksum of addresses between `a` and `a+u` inclusive. The algorithm simply adds each byte (mod 256).

```
NEEDS ASSEMBLER
CODE CHECKSUM ( a u -- b )
      exx               \ needs this to preserve BC and DE
      pop     bc|       \ counter u
      pop     hl|       \ start address a
      xora     a|       \ zero to accumulator
HERE  adda   (hl)|       \ add mod 256
      cpi                \ increment HL, decrement BC
      jpf     pe|  AA,   \ loop back if BC is not zero
      ld      c'|   a|
      push    bc|       \ return result
      exx               \ restore register BC and DE
      NEXT
C;                      \ this performs SMUDGE
```

Up to now, there is no labels, since the same functionality is given by a clever use of `HERE` and stack manipulation, and because routines are kept short, but you are free to define any **`VARIABLE`** or **`CONSTANT`** and use its value with **`AA,`**.

For example:

```
CODE CHECKSUM ( a u -- b )
      exx               \ needs this to preserve BC and DE
      pop     bc|       \ counter u
      pop     hl|       \ start address a
      xora     a|       \ zero to accumulator
HERE CONSTANT LABEL
      adda   (hl)|       \ add mod 256
      cpi                \ increment HL, decrement BC
      jpf     pe| LABEL AA,    \ loop back if BC is not zero
      ld      c'|   a|
      push    bc|       \ return result
      exx               \ restore register BC and DE
      NEXT
C;                      \ this performs SMUDGE
```

To use the new `CHECKSUM` definition you may try

```
DECIMAL 448 60 CHECKSUM .
67 ok
```

3.10.3 Assembler Example - WHITE-NOISE from Egghead 3

This is an example taken from “Egghead 3” that uses the first bytes of ROM as a pseudo-random number list used to produce a white noise. The source is available in Screen# 166 of standard BLOCK file.

Labels seem a little more complicated, but just remember each **HERE** is resolved using **BACK,** without confusion.

Also, this is an example of *forward reference* resolved directly patching the code just compiled: the address to the subroutine is patched by saving the instruction address (the first `HERE`) and then using the sequence `HERE SWAP 1+ !` to patch the previous CALL address.

```
NEEDS ASSEMBLER
DECIMAL
CODE WHITE-NOISE ( -- )
      di
      exx
HERE  call  0000  AA,          \ 0000 is patched here below (*)
      exx
      ei
      next
\
\ subroutine
HERE SWAP 1+ !                 \ patch the previous CALL  (*)
      ldn   e'|   250   NN,
      ldx   hl|   300   NN,
HERE  push  de|                \ <-------... (†) main loop
      ldn   b'|    32   N,
HERE    push      bc|          \ <-------+
        ld  a'|   (hl)|        \         |
        incx      hl|          \         |
        andn      248   N,     \         |
        outa      254   P,     \         |
        ld  a'|   e|           \         |
        cpl                    \         |
HERE      dec   a'|            \ <--+     |
          jrf  nz'| BACK,      \ ---+     |
        pop   bc|              \         |
      djnz  BACK,              \ --------+
      pop   de|
      ld    a'|    e|
      subn  24     N,
      cpn   30     N,
      retf   z|                \ return from routine
      retf  cy|                \ return from routine
      ld    e'|    a|
      cpl
HERE  ldn   b'|    40   N,     \ <-------+
HERE                           \ <--+    |
        djnz  BACK,            \ ---+    |
      dec   a'|                \         |
      jrf  nz'|   BACK,        \ --------+
      jr    BACK,              \ --------> ... (†)
```

To test it you need to set CPU's speed at 3.5 MHz via **SPEED!**.

```
NEEDS SPEED!
0 SPEED! WHITE-NOISE
```

3.10.4 Assembler Example “ms” millisecond delay

This definition waits at least `u` milliseconds, with `u` < 8192, indentation helps to glimpse the various nested loops. The outer loop takes exactly 3,500 T-states so that it can be used to produce milliseconds delays on a standard 3.5 MHz CPU. Some CPU time is spent to compute the correct number of iterations.
Also, this is an example of *forward relative-jump* **HOLDPLACE** resolved via **DISP,**

```
DECIMAL
CODE ms ( u -- )
    exx                             \ preserve BC and DE
    pop     hl|
    ld      a'|    l|
    ora      h|
    jrf     z'| HOLDPLACE           \ skip to end when u is zero (*)

        ldx     bc|  $243B  NN,      \ get current CPU speed
        ldn     a'|     07   N,      \ to calculate the correct
        out(c)  a'|                 \ delay
        inc     b'|
        in(c)   a'|
        andn    3   N,
                                    \ this is a begin-while-repeat loop.
        HERE                        \ keep this address for later (♠)
        jrf     z'| HOLDPLACE       \ condidional forward to (♦)
            addhl   hl|             \ double HL based on CPU speed
            dec     a'|
            jr  SWAP BACK,          \ uncontidional loop back to (♠)
        HERE DISP,                  \ resolve forward jrf (♦)

        HERE                        \ begin of outer loop (♣)
            nop                     \    4 T
            ldn   b'| 204 N,        \    7 T
            HERE                    \        inner loop (♥)
                nop                 \ 3463 T = (4+13)*203 + (4+8)
            djnz  BACK,             \        back to inner loop (♥)
            decx  hl|               \    6 T
            ld    a'|   l|          \    4 T
            ora    h|               \    4 T
        jrf  nz'| BACK,             \   12 T   ( -5 T on final loop )
                                    \ 3500 T total
                                    \ repeat outer loop (♣)

    HERE DISP,                      \ resolve skip-to-end when zero (*)

    exx
    NEXT
C;
```

HOLDPLACE --- a1

ALLOT the next byte as placeholder of a relative-jump displacement. The address a1 points to this placeholder and should be resolved by a subsequent **DISP,** or a derived definition such as **BACK,**
It's defined just as **: HERE 0 D, ;**

DISP, a1 a2 ---

Compute the displacement from a2 to a1 and compile it to address a2 as displacement of a relative-jump op-code.
It's defined just as **: DISP, OVER - 1- SWAP C! ;**
The following snippet implements an IF-THEN-ELSE phrase in Assembler:

```
cpn    $60 N,
jrf  cy'|  HOLDPLACE              \ if lowercase
     subn  $20  N,                \ quick'n'dirty uppercase
HERE DISP,                        \ aka THEN,
```

The following example implements a complete IF-THEN-ELSE phrase that check “carry-flag”:

```
jrf  cy'|  HOLDPLACE              \ IF,
     ldx   hl|    1   NN,
jr   HOLDPLACE  SWAP HERE DISP,   \ ELSE,
     ldx   hl|   -1   NN,
HERE DISP,                        \ THEN,
```

BACK, a1 ---

Compute the displacement from HERE to a1 and compile it to address HERE as displacement of a relative-jump op-code. It's defined just as **: BACK, HOLDPLACE SWAP DISP, ;**

The following example implements a BEGIN-UNTIL loop in Assembler:

```
\ Wait for a standard key-press.
HERE                              \ BEGIN,
    bit      5| (iy+ 1 )|         \ FLAGS (5C3A+1)
jrf     z'| BACK,                 \ UNTIL,
```

3.10.5 Syntax check error messages

During assembly, the following error messages may be reported

#26 Error at postit time.
The previous instruction is incomplete.
For example:

```
ldn    a'|   0  N,
ldn    b'|   0     \  missing N,
nop
```

reports: **`nop? Error at postit time.`** meaning the previous instruction is incomplete.

#27 Unexpected fixup/commaer.
Argument is illegal for this op-code
For example:

```
ina    b'|
nop
```

reports: **`b'|? Unexpected fixup/commaer.`** meaning the argument is wrong.

#29 Commaer data error.
This is a common error issued whenever the wrong kind of operand is used
For example:

```
ldn  a'| $00 n,
ldn   b| $00 n,   \  should be  b'|  instead of  b|
```

reports: **`b'|? Commaer data error.`** meaning the argument is wrong.

#30 Commaer wrong order.
When messing with commaers

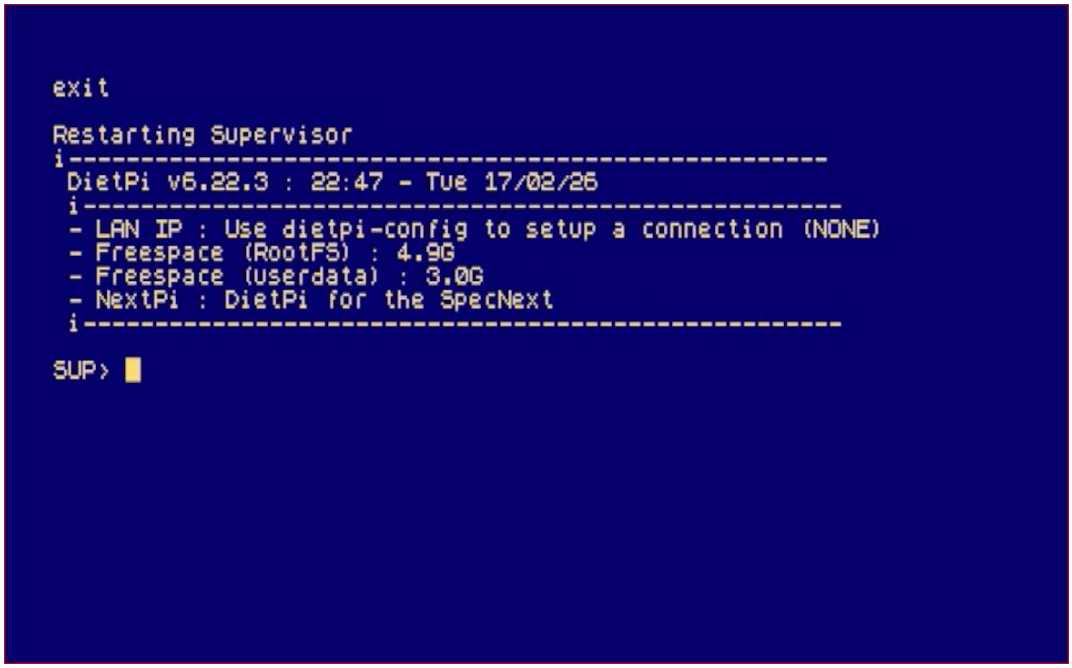
For example:

```
ldn a'| 0 n, p,
```

reports: **`p'|? Commaer wrong order.`** Funny

3.11 Raspberry Pi Zero accelerator

If you have the Accelerated version of the ZX Spectrum Next, that is a Raspberry Pi Zero attached to the motherboard, then you can experiment it via this Forth system. Some useful definitions are available after you type **`NEEDS RPi0`** that put in place a two-way communication stream using **`RPi0`** via UART I/O.



The rationale is as follows. The main loop continuously polls both the Raspberry Pi Zero UART for any incoming data, and the keyboard for user input, any key pressed is immediately transmitted to RPi0, and any byte received from RPi0 is immediately sent to screen, but **[ENTER]** key has a special behavior such that, once "0x0D" is transmitted to RPi0, up to 8192 bytes are fast-read from UART to a RAM buffer and – only then – slowly sent to screen allowing long output hopefully without data-loss.

Other keys works as follows:

[BREAK]	quit to Forth prompt
[CAPS LOCK]	toggles caps lock and sends nothing.
[>=]	transmits 0x03, ETX or ^C that helps to emulate CTRL-C key-press.
[TRUE VIDEO]	transmits 0x04, EOT or ^D that normally produces a "normal exit" from whatever you where in.
[INV VIDEO]	transmits 0x05, ENQ or ^E
[EDIT]	transmits 0x07, BEL or ^G
[DELETE]	transmits 0x08, BS or ^H and it is the normal back-space key.
[←]	transmits 0x08, BS or ^H same as [DELETE]
[→]	transmits 0x09, LF or ^I. This is equivalent to TAB key-stroke useful to complete a command.
[↓]	transmits 0x0A, BS or ^J
[↑]	transmits 0x0B, VT or ^K
[EXTENDED]	transmits 0x0E, S0 or ^N
[GRAPH]	transmits 0x0F, SI or ^O
[<>]	transmits 0x18, CAN or ^X that helps to emulate CTRL-X key-press.
[<=]	transmits 0x1A, SUB or ^Z that helps to emulate CTRL-Z key-press.
[AT]	transmits 0x1B ESC or ^[that helps to emulate escape sequences

Remaining ASCII characters **`~ | \ [ ] { }`** are produced via SYMBOL-SHIFT as usual.

Typing **RPi0** Raspberry Pi Zero's UART is first correctly configured and when you get the prompt "**SUP>**" you know you're logged in.

If you hit [BREAK] you immediately quit back to Forth prompt leaving RPi0 in whatever state it is, which usually you can re-enter again using just **TERM** instead of the whole route of **RPi0**.

Once you're "connected" to RPi0, you can send a line of text back to Forth to interpreter via **#** character that it's usually used as a comment within Linux . This feature is disabled or enabled setting **UART-META** variable to zero or non-zero.

Using **RPi0** you can reach with a large amount of space and computation power using any scripting language available (Python, Perl, Lua).

Some more definitions allows you to keep in Screens your script source and transmit them to RPi0: To make your scripts persistent across a RPi0 reboots, you should use /NextPi/nextpi directory, for example.

3.11.1 RPi0 high-level communication

Here is how to interact with RPi0 command prompt once it is initialized via **RPi0-INIT**. The following high-level definitions must be imported using **NEEDS**.

ASK **---** **n**

Accept the following text from current input stream and send it to RPi0 terminal followed by a $0D character to start whatever command is sent, then collect to **PAD** up to 1024 bytes received from RPi0 in reply, then returning **n** as the length of such a reply.

For example, at prompt, you can ask which Bash-shell version RPi0 is currently running, typing from keyboard:

```
ASK  echo $BASH_VERSION  ◄┘
DISPLAY ◄┘

4.4.12(1)-release
```

Notice, in this case you have to hit [ENTER] after the first line to let know `ASK` definition where the command ends, and in another source line let Forth accept `DISPLAY` to display the reply from RPi0.

DISPLAY **n** **---**

Counterpart of **ASK**, this definition emits n characters-long string currently stored at **PAD**.
The above example given for **ASK** definition can be moved into a file or a Screen, and later used via **INCLUDE** or **LOAD**.
The following example shows how to set-up date and time:

```
Scr# 899
  0 ( Example for ASK-DISPLAY )
  1  NEEDS ASK
  2  NEEDS DISPLAY
  3  ASK  date -s "2026-01-15 17:05"
  4  DISPLAY
```

Then you can type the:

```
899 LOAD
```

to obtain the result:

```
Sun 15 Feb 17:05:00 UTC 2026
```

THIS --- a n

Accept text from current input stream preparing the string to be sent to RPi0. Used by **ASK**.
This definition is much like **WORD** with the difference that no delimiter is given and the rest of the line is assumed as the string which address **a** and length **n** is given as result.

Rpi0-SEND a1 n1 a2 n2 --- n3

Send to RPi0 **n2** bytes of a string stored at address **a2**, the reply from RPi0 is collected to address **a1**, up to **n2** bytes, returning **n3** as the length of such a reply. It's used by **ASK**.
For example, to calculate **2^200** using Python we can do

```
NEEDS S"
: TEST
  HERE 100
  S" python -c 'print 2**200'"
  RPi0-SEND HERE SWAP TYPE
;
```

Then typing:

```
CR TEST CR
```

you'll see the result computed with Python:

```
1606938044258990275541962092341162602522202993782792835301376
```

In this case we temporarily use 100 bytes at **HERE** to receive the answer that is immediately displayed.

3.11.2 RPi0 UART low-level communication

The following constants are defined for ease of use, available inside UART-SYS.f source file:

`$5C08 CONSTANT UART-LASTK`	system variables alias which stores newly pressed key.
`$5C78 CONSTANT UART-FRAMES`	system variables alias frame counter incremented every 20ms.
`$5C6A CONSTANT UART-FLAGS2`	system variables alias (Bit 3 set when CAPS SHIFT or CAPS LOCK is on)
`$143B CONSTANT UART-RX-PORT`	hardware receive-data I/O port
`$133B CONSTANT UART-TX-PORT`	hardware transmit-data I/O port
`$153B CONSTANT UART-CT-PORT`	hardware control-data I/O port

Here is a list of low-level definitions.

?UART-BYTE-READY --- f

Check if UART has received a byte: true flag when byte ready, false elsewhere

?UART-BUSY-TX --- f

Check if UART is busy sending a byte, non-zero when transmitter is busy sending a byte. There is no transmit buffer so program must make sure the last transmission is complete before sending another byte.

RPI0 ---

Main definition to connect to Raspberry Pi Zero. It performs **RPi0-INIT** to setup I/O and Baud-Rate, **TERM-INIT** to reduce font-size to let 85 columns display. A this point, the main loop is controlled by **TERM**.

TERM ---

Resume communication with RPi0 entering the main loop in which it

1. shows cursor and accept a key-stroke from keyboard if available.
2. if RPi0 sent a byte accepts and displays it.
3. if a key-stroke was available, sends it to RPi0, $0D triggers the command computation on RPi0 end.
4. handles # key-stroke, recording the string command to be sent to Forth prompt.

RPi0-INIT ---

This definition should be invoked whenever you need to setup RPi0 UART communications, that is at startup or after you use Jary Komppa "sync" utility.

The following setup steps are performed:

1. Run at maximum speed: `3 7 REG!`
2. Select RPi0 UART sending a suitable value to the control port: `$40 $153B P!`
3. Hardware flow-control, 8-bit-frame, no-parity-check, one-stop-bit: `$40 $163B P!`
4. Enable peripheal: `$30 $A0 REG!  \ PI Peripheal enable`
5. Ask the hardware NextREG 17 ($11) "Video Timing" to determine the actual CPU speed, then compute a 14 bits *prescalar value* dividing the actual MHz clock speed by the 115,200 baud rate to have a 14 bits value to be sent to UART receive port split in two parts:

```
DUP      $7F AND $143B P!     \ send low 7 bits of 14 bits
7 RSHIFT $80 OR  $143B P!     \ send high 7 bits of 14 bits
```

RPi0-SELECT ---

Select Raspberry PI Zero UART and set Baudrate as shown in **RPi0-INIT**.

UART-1ST-TIMEOUT --- a

Variable used by `UART-RX-BURST` to timeout the first byte. At startup it's defaulted at 50.000 that is 200 ms.

UART-2ND-TIMEOUT --- a

Variable used by `UART-RX-BURST` to timeout any subsequent byte after the first. At startup it's defaulted at 10.000 that is 40 ms.

UART-RX-BYTE --- b | 0

Accept a byte if available, 0x00 if no byte is available.

UART-RX-TIMEOUT n --- c | 0

Wait for a byte within timeout expressed in ms. If no byte is available, 0 is returned.

UART-RX-BURST a n1 --- n2

Accept from UART up to `n1` bytes and store them at address `a` allowing for a fixed timeout: Timeout to receive the first byte is set at 200 ms, timeouts for the next bytes is set at 20 ms, this means we assume RPi0 to reply within 200 ms to any command we issue and any subsequent delay greater than 20 ms ends data collection.
At 115.200 Baud, one bit duration is 8.68 microseconds a 512 bytes-buffer is filled in about 4.4 ms but a 512 bytes burst-read takes 200 T per byte (102400) plus enter-exit time that at 28 MHz is about 3.7 ms so we should keep draining the buffer faster than it fills up.

UART-SET-BAUDRATE d ---
Set baud rate to `d`.

UART-SET-PRESCALAR n ---
Send 14 bits *prescalar* `n` to UART.

UART-TX-BYTE b ---
There is no transmit buffer so program must take care the last transmission is complete before sending another byte. This definition waits until transmission is possible or Break is pressed. There are a few short-cut to easily send some control characters, such as `UART-SEND-EOT`, `UART-SEND-ETX`, `UART-SEND-CR`, `UART-SEND-LF`.

UART-SEND-TEXT a n ---
Send a string of text to RPi0 by repeatedly invoking `UART-TX-BYTE`.

UART-WAIT-B b ---
wait for a specific byte (or [BREAK] key to bailout).

UART-WAIT-CR-LF ---
Wait for a CR-LF sequence.

UART-WAIT-STR a n ---
Wait for a specific **`n`** bytes-long string stored at address **`a`**. It uses `UART-WAIT-B`.

UART-WAIT-PROMPT ---
Wait for a specific string "SUP> " to be received. There is no timeout: the only way to stop the wait is pressing [BREAK]

3.12 Tilemap Hardware

The ZX Spectrum Next provides a hardware Tilemap layer that can display text or graphics using a grid of tiles, each referring to a pre-defined character or bitmap pattern stored in RAM.

Here is an experimental way to exploit this feature and have 80x32 text screen:

`NEEDS TILE80` — full Tilemap text driver, 80 columns 32 rows (lib/TILE80.f)

Utility script lib/TILE80-setup.f, which has to be run once with the non-dot version of vForth, reads the standard ROM character set and saves a binary file ./lib/TILE80-charset.bin used later at runtime by `TILE80`.

3.12.1 Hardware overview

The Tilemap is a hardware display layer independent of the ULA and Layer 2. It is controlled primarily through two Next REG registers:

Register	Address	Purpose
`Tilemap Control`	`$6B (107)`	Enable tilemap, select resolution, text mode, palette, 512-tile mode
`Tilemap Attr Ctrl`	`$6C (108)`	Palette offset, X/Y mirror, 90° rotation, tile index bit 8
`Tilemap Base Addr`	`$6E (110)`	Base address of the tile grid in RAM (unit: $200 = 512 bytes)
`Tile Def Base Addr`	`$6F (111)`	Base address of tile definitions in RAM (unit: $200)
`ULA Control`	`$68 (104)`	Bit 7: disable ULA output (set to $80 to suppress ULA)
`Palette Index`	`$40 (64)`	Select palette entry for read/write
`Palette Data`	`$41 (65)`	8-bit palette colour value

The Tilemap Control register ($6B) bit layout is as follows:

```
bit 7 = 1  enable the tilemap
bit 6 = 0  40×32 columns;  1 = 80×32 columns
bit 5 = 1  no attribute byte in tilemap (saves space)
bit 4 = 0  first palette;  1 = second palette
bit 3 = 1  text mode (1-bit B&W bitmaps)
bit 2 = 0  reserved, must be zero
bit 1 = 1  512-tile mode
bit 0 = 1  force tilemap on top of ULA
```

Two tile-grid resolutions are supported:

40×32 columns — 4-bit colours per pixel, tile grid 2560 bytes ($0A00), tile definitions from $4A00, up to 136 character definitions. Each tile cell requires two bytes (tile index + attribute).

80×32 columns — 1-bit colour per pixel (80-mode), tile grid 5120 bytes ($1400) or just 2560 bytes when used without attribute byte (i.e. TXT-MODE), both 80x32 modes tile keep definitions from $5400, up to 224 character definitions. Each tile cell requires two bytes or one byte depending on the attribute mode with or without attribute byte.

3.12.2 TILE80 — 80-column text driver

The **TILE80** library (lib/TILE80.f) implements a complete text-output driver for the 80×32 Tilemap text mode. It redirects the **EMIT**, **EMITC** and **CLS** vectors so that all Forth text output is routed to the Tilemap display. It requires the binary charset file ./lib/TILE80-charset.bin to be present on the SD card (this file is generated once by executing ./lib/TILE80-setup.f). The tile grid is located at $4000 (DISPLAY-FILE). Tile definitions occupy $5400–$5AFF (1792 bytes, 224 definitions of 8 bytes each). The grid size is 80×32 = 2560 bytes in no-attribute text mode.

TILE80 **---**

Activate the 80-column Tilemap text driver. Clears the screen, loads the character definitions from ./lib/TILE80-charset.bin into $5400, sets the palette, configures base addresses, enables **TILE-TXT** mode, and patches **EMIT**, **EMITC** and **CLS** definitions toward **T-EMIT** and **T-CLS** respectively.

NO-TILE **---**

Deactivate this Tilemap text driver, restoring the original **EMIT**, **EMITC** and **CLS** vectors, disabling the tilemap via **TILE-OFF**, and clearing the screen.

T-CLS **---**

Clear the tile grid by filling it with space characters, then call **T-HOME**.

T-HOME **---**

Reset cursor to position 0 and – just to be sure – reload the character definitions from ./lib/TILE80-charset.bin at address $5400.

T-EMITC **c** **---**

Emit character **c** to the tile grid at the current cursor position, advancing the cursor. Control characters $08 (backspace), $0A/$0D (newline), and $16 (AT) are handled. Scrolls one row when the bottom is reached. Returns 0 (used internally as a vector replacement for **EMITC**).

T-EMIT **c** **---** **0**

Emit character **c** to the tile grid at the current cursor position, advancing the cursor. Control characters $08 (backspace), $0A/$0D (newline), and $16 (AT) are handled. Scrolls one row when the bottom is reached. Returns 0 (used internally as a vector replacement for **EMIT**).

T-SCROLL **---**

Scroll the tile grid one row upward: copies rows 1–31 over rows 0–30 using CMOVE, blanks the last row, and adjusts **T-POS**.

T-CR **---**

Move the cursor to the beginning of the next row, scrolling if necessary.

T-POS **---**

VARIABLE. Holds the current cursor position as a byte offset into the tile grid (0 = top-left, 2559 = bottom-right).

T-WIDTH **---**

CONSTANT 80. Width of the tile grid in characters.

T-HEIGTH **---**

CONSTANT 32. Height of the tile grid in rows.

T-SIZE **---**

CONSTANT 2560 (**T-WIDTH** × **T-HEIGHT**). Total size of the tile grid in bytes.

DISPLAY-FILE **---**

CONSTANT $4000. Base address of the tile grid in RAM.

T-XY **--- a**

Return the address of the current cursor position in the tile grid and advance **T-POS** by one.

3.12.3 LAYER3 — low-level definitions

The following definitions are available after **NEEDS LAYER3** (or equivalently via the demo files that include them inline). They configure the hardware registers directly.

A few demos are avaliable to demonstrate the capabilities of each of three modes.

```
demo/Layer3-demo1.f
demo/Layer3-demo2s.f
demo/Layer3-demo3.f
```

TILE-OFF **---**

Disable the Tilemap layer and re-enable ULA output. Writes $00 to NextREG $6B and $00 to NextREG $68.

TILE-40 **---**

Enable Tilemap in 40×32 column mode with attribute byte. Writes %10000001 to NextREG $6B and $80 to NextREG $68 (ULA suppressed).
Set the base addresses for 40-column mode: Tile Definitions Base at $4A00 ($06 to NextREG $6F), Tilemap Base at $4000 ($00 to NextREG $6E).
Tilemap palette couple (index-color) definitions are compiled in PFA in the subsequent memory.

TILE-80 **---**

Enable Tilemap in 80×32 column mode with attribute byte. Writes %11001001 to NextREG $6B and $80 to NextREG $68.
Set the base addresses for 80-column mode: Tile Definitions Base at $5400 ($13 to NextREG $6F), Tilemap Base at $4000 ($00 to NextREG $6E).

Configure the first Tilemap palette with four foreground/background colour pairs suitable for 80-column display: yellow on blue, white on blue, red on blue, magenta on bllue.

TILE-TXT **---**

Enable Tilemap in 80×32 text mode without attribute byte (available in TILE80.f). Writes %11101001 to NextREG $6B and $80 to NextREG $68. Set the base addresses for 80-column mode: Tile Definitions Base at $5400 ($13 to NextREG $6F), Tilemap Base at $4000 ($00 to NextREG $6E). In this mode each grid cell is a single byte (tile index only), halving the memory used by the tile grid to 2560 bytes.

SET-PAL **---**

Write colour value `c` to palette entry `n`. Sends `n` to NextREG $40 (Palette Index) and `c` to NextREG $41 (Palette Data).

GET-PAL **---**

Read colour value from palette entry `n`. Sends `n` to NextREG $40, reads result from NextREG $41.

3.12.4 TILE80-setup — charset preparation

The file lib/TILE80-setup.f is a one-time setup script that must be run with the non-dot vForth version. It reads the standard ZX Spectrum ROM character set (96 printable characters from $5C36+256) and saves the tile definition bitmap file ./lib/TILE80-charset.bin to the SD card. The file is 1792 bytes long (224 definitions × 8 bytes each, 1-bit per pixel).

In addition, 32 UDG (User Defined Graphic) entries are appended: 16 half-block patterns plus their complements, providing a basic block-graphics character set usable within the Tilemap text mode.

After running the setup, **TILE80** can be activated at any time with no further preparation required:

```
NEEDS TILE80
TILE80
\ All subsequent Forth output goes to the 80-column tilemap display
NO-TILE   \ restore normal ULA output
```

3.12.5 Notes

Memory layout. Because vForth's code resides at $6363, the lower part of the display file ($4000–$635F) is available for the tile grid and definitions. The demo Layer3-demo1.f explicitly exploits this overlap, showing how the display file is filled with tile data before the layer is activated.

TILE80 vs LAYER3. **LAYER3** provides only the low-level register words and palette helpers. **TILE80** builds a complete text system on top of them, including scrolling, cursor management, and EMIT/CLS redirection. For custom tile-based graphics (e.g. 40-column 4-bit colour mode), use the **LAYER3** primitives directly as shown in demo/Layer3-demo1.f and demo/Layer3-demo2.f.

TILE-TXT-PALETTE. TILE80 installs a minimal single-entry palette (light yellow on dark blue, NextREG $6C cleared to palette offset 0). For richer colour, call SET-PAL after TILE80 to populate additional palette entries.

TILE-80 vs TILE-TXT. **TILE-80** enables 80-column mode with attribute byte (two bytes per cell, grid size 5120 bytes). **TILE-TXT** enables 80-column mode without attribute byte (one byte per cell, grid size 2560 bytes). TILE80.f uses **TILE-TXT** to minimise memory usage.

3.13 Beeper

The following definitions are available after you type **NEEDS BLEEP**. This small library was born as a demo in its own right, and grew into a handy way to make sound on the classic Spectrum beeper by toggling bit 4 of port $FE through the standard ROM. Loading it also pulls in **SPEED@** and **SPEED!**, which **BEEP** needs to drive the ROM routine at the correct clock rate.

The Spectrum ROM beeper routine is timed for a 3.5 MHz CPU. On a ZX Spectrum Next running faster, a raw beeper call would play at the wrong pitch and duration; this is why BEEP temporarily forces 3.5 MHz and restores the previous speed afterwards. If you call the lower-level **BLEEP** directly, you are responsible for setting 3.5 MHz yourself (**SPEED@ >R 0 SPEED! ... R> SPEED!**).

BLEEP n1 n2 ---
Raw beeper call into the standard ROM routine at $03B5. Both parameters must be pre-computed: n1 is the pitch parameter (3.5MHz / Hz - 241) / 8, and n2 is the duration seconds * Hz. The CPU must already be at 3.5 MHz. The library compiles the ROM call in two forms depending on the build: a RST $18 when running as a dot-command, or a plain CALL otherwise. For example, central A (220 Hz) for one second:

```
HEX 07A7 00DC BLEEP ( n1=1959 n2=220 )
```

BLEEP-CALC n1 n2 --- n3 n4
Convenience converter: turns a friendly pair (ms 8*Hz) -- a duration in milliseconds and a frequency expressed as Hz multiplied by 8 -- into the (n3 n4) pair that **BLEEP** expects. For example, 500 ms at 440 Hz:

```
500 3520 BLEEP-CALC BLEEP ( 3520 = 440 * 8 )
```

BEEP-PITCH n --- n2
Convert a chromatic pitch number n into a frequency multiplied by 8, ready for **BLEEP-CALC**. Pitch 0 is middle C, 9 is concert A (440 Hz), 12 is one octave above middle C, and negative values go below it. It is a pure function and can be tested:

```
9 BEEP-PITCH . ( prints 3520, i.e. 440 * 8 )
```

BEEP n1 n2 ---
High-level BASIC-style note, taking a duration n1 in milliseconds and a chromatic pitch n2 (see **BEEP-PITCH**). It chains **BEEP-PITCH** -> **BLEEP-CALC**, saves the current CPU speed, drops to 3.5 MHz, calls **BLEEP**, then restores the speed. This is the word for everyday use:

```
300 0 BEEP ( middle C for 300 ms )
1000 9 BEEP ( concert A 440 Hz for one second )
```

3.14 AY sound chip facility (Turbosound Next)

The following definitions are available after you type **NEEDS AY.** To forget this library from the dictionary you can type **NO-AY** (a **MARKER** placed at its head). Loading it also pulls in SPLIT, used to break a 16-bit tone period into its low byte

and high nibble. The ZX Spectrum Next carries three AY-3-8910 (Turbosound Next) sound chips, named AY1, AY2 and AY3. Each chip offers three tone channels (A, B, C), one noise generator and one envelope generator, controlled through 14 registers. Communication uses two I/O ports: AY-register-port ($FFFD) selects the active chip or the register number, and AY-data-port ($BFFD) writes the data byte to the selected register. The library also names the relevant NextRegs as Peripheal-2-register (6), Peripheal-3-register (8) and Peripheal-4-register (9).

Per-chip register map:

0 Channel A tone, low byte (12-bit period = reg0 + reg1*256)
1 Channel A tone, high 4 bits
2 Channel B tone, low byte
3 Channel B tone, high 4 bits
4 Channel C tone, low byte
5 Channel C tone, high 4 bits
6 Noise period 0-31
7 Mixer: 0 0 noise[C B A] tone[C B A] (0 = enabled)
8 Channel A volume 0-15 (bit 4 = use envelope)
9 Channel B volume 0-15
10 Channel C volume 0-15
11 Envelope period, fine
12 Envelope period, coarse
13 Envelope shape 0 0 0 0 [C At Al H]

The tone period for a frequency, at the 1.75 MHz chip clock, is period = 1750000 / (16 * freq_Hz); for example 440 Hz gives a period of about 248.

AYSELECT n ---

Select which chip subsequent writes address: 1 = AY1, 2 = AY2, 3 = AY3. The value is masked to the range 0-3; 0 does not change the chip (it only records the selection in the variable AY), so always pass 1, 2 or 3 to be explicit.

AY! b ayreg ---

Write byte b to register number ayreg (0-13) of the currently selected chip. For example, set channel A to maximum volume: **`15 8 AY!`**.

AY!! n ayreg ---

Write a 16-bit tone period n to a register pair: the low byte goes to ayreg and the high 4 bits to ayreg+1. Use ayreg = 0, 2 or 4 for channels A, B or C. For example, a ~440 Hz tone on channel A: **`248 0 AY!!`**.

SHH ---

Silence the currently selected chip by writing $FF to the mixer register 7, which disables all tones and noise on that chip.

ENABLE-TURBOSOUND ---

Set bit 1 of NextReg 8, enabling Turbosound so that AY2 and AY3 become usable in addition to AY1.

ENABLE-MONO ---

DISABLE-MONO ---

Set (respectively clear) bits 7-5 of NextReg 9, routing all channels of the current chip to a single mono output (or restoring stereo).

AYSETUP **---**

One-shot initialisation: for each of the three chips (AY3, AY2, AY1 in turn) it silences the chip with SHH and calls **ENABLE-MONO**, leaving AY1 selected and ready. Call it once before producing any sound.

A minimal tone on channel A of AY1:

```
AYSETUP                 ( silence + setup all three chips )
1 AYSELECT              ( AY1 )
15 8  AY!               ( channel A: max volume )
248 0 AY!!              ( ~440 Hz tone period )
%00111110 7 AY!         ( mixer: enable channel A tone only )
...
SHH                     ( stop )
```

For real music rather than raw register poking, the project includes an interrupt-driven frame player, **AFXFRAME** (see the related tutorial and the block-based AY demo), which feeds the AY chips from compiled frame data on every video interrupt.

3.15 Keyboard input

Reading the keyboard is split between words that are always present in the core and a couple of small helpers loaded on demand. **KEY**, **?TERMINAL** and **UPPER** are core words, available immediately; **WAIT-KEY** and **?ESCAPE** are pulled in with **NEEDS WAIT-KEY** and **NEEDS ?ESCAPE**. The two families differ in how they read the keyboard: **KEY** and **WAIT-KEY** block, suspending execution until a key is pressed, while **?TERMINAL** and **?ESCAPE** poll, testing the current state of the keyboard and returning at once. Use a blocking word when you want to read a character, and a polling word when you want a running loop to be interruptible without stopping for input.

KEY **--- c**

Suspend execution until the user presses a key, then return its code c. Most keys map to standard 7-bit ASCII: letters A-Z give $41-$5A, digits 0-9 give $30-$39, ENTER gives $0D, SPACE gives $20 and DELETE (CAPS SHIFT + 0) gives $0C. **KEY** reads through the normal input channel, so it honours CAPS LOCK and the SYMBOL/extended shift modes. For example, **KEY** . waits for a key and prints its code, and **KEY EMIT** echoes the character pressed.

WAIT-KEY **---**

Wait for a clean keypress by polling port $FE directly, bypassing the ROM keyboard scanner and the Forth input stream. It first waits for every key to be released (bits 0-4 of the port all 1), then waits for any key to be pressed (one bit goes 0). It does not modify the stack and does not tell you which key was pressed; its purpose is a debounced "press any key to continue" pause with guaranteed no key-repeat bounce.

UPPER **c1 --- c2**

Force the character c1 to its uppercase form c2; codes that are not lowercase letters pass through unchanged. It is the convenient way to make keyboard tests case-insensitive without comparing against both letter cases: **KEY UPPER [CHAR] Y =** is true whether the user typed Y or y.

?TERMINAL **--- f**

Non-blocking test of the BREAK key. Return a true flag if BREAK (CAPS SHIFT + SPACE) is being held at the moment of the call, false otherwise; it never waits. This is the standard "stop now" signal: long-running loops and the drawing primitives

poll **?TERMINAL** so the user can abort them, e.g. BEGIN (...work...) **?TERMINAL UNTIL**.

?ESCAPE **--- f**

Non-blocking test of the [EDIT] combination. It reads port $FE twice, selecting two keyboard half-rows through the high byte of the address bus ($FE for the row with CAPS SHIFT, $F7 for the row with key 1), and returns a true flag when both CAPS SHIFT and 1 are held at the same time. Like **?TERMINAL** it returns immediately. It is typically used to throttle a long scrolling listing -- **BEGIN ?ESCAPE NOT UNTIL** pauses until the user holds [EDIT] -- as done by **DIR**.

3.15.1 Direct keyboard-matrix scanning

All the words above read the keyboard one logical character or one fixed key at a time, through the ROM and the input channel. **NEEDS KEYBOARD** pulls in a small layer that instead scans the raw 8x5 key matrix directly on port $FE, the classic ZX Spectrum technique. It buys three things the words above cannot: it can report several keys held at once, it polls without blocking, and it is fast enough to read the controls once per video frame -- which is what an arcade-style game needs. The cost is that you work with raw matrix positions, not ASCII: CAPS LOCK, the shift folding and the input channel are all bypassed.

Each of the 40 keys carries a fixed index 0..39, assigned row by row. The high byte of the port address selects one of eight half-rows; the five low bits it returns are that row's five keys, active low (a 0 bit = key down):

port	bit0	bit1	bit2	bit3	bit4	indices
$FEFE	CAPS	Z	X	C	V	0.. 4
$FDFE	A	S	D	F	G	5.. 9
$FBFE	Q	W	E	R	T	10..14
$F7FE	1	2	3	4	5	15..19
$EFFE	0	9	8	7	6	20..24
$DFFE	P	O	I	U	Y	25..29
$BFFE	ENTER	L	K	J	H	30..34
$7FFE	SPACE	SYMB	M	N	B	35..39

The two shift keys are index 0 (CAPS SHIFT) and 36 (SYMBOL SHIFT).

KEYBOARD **--- c**

Return the address of the 8-byte table of half-row select bytes ($FE, $FD,... $7F), you seldom use it directly: It is the word **NEEDS KEYBOARD** tests for, and the building block the index-to-port words read.

NO-KEY **--- n**

A constant, value 255, returned by **KEY-SCAN** when nothing at all is pressed. Compare a scan result against **NO-KEY** to decide whether any key is down.

KEY#>PORT **n --- port**

Map a key index 0..39 to the 16-bit port that selects its half-row: e.g. **12 KEY#>PORT** gives $FBFE, the Q-W-E-R-T half-row. Reading that port returns the five keys of the row in its low bits.

KEY#>MASK **n --- mask**

Map a key index 0..39 to the single-bit column mask of that key within its row ($01, $02, $04, $08, $10 for columns 0..4). **KEY#>PORT** and **KEY#>MASK** together are all you need to test one key.

KEY#>RM n --- port mask

Freeze a key index into the (port mask) pair used for fast polling -- it is **KEY#>PORT** and **KEY#>MASK** applied to the same index. Store the pair in two variables at setup time and a game loop can then test the key every frame with a single port read, never re-deriving it. This is how user-redefinable controls are built.

KEY-PRESSED? port mask --- f

Read the given port and return a true flag if the key picked out by mask is down at this instant; it never waits. This is the per-frame test that consumes a stored (port mask) pair produced by **KEY#>RM**.

KEY-DOWN? n --- f

Non-blocking test of a single key by its index: true while index n is held, false otherwise. It is **KEY#>RM** followed by **KEY-PRESSED?**. Because each call asks about one key, several calls detect several keys held at the same time -- something **KEY** cannot do.

KEY-SCAN --- n

Scan the whole matrix and return the index of the lowest-numbered key held, or **NO-KEY** (255) if none is; it never waits. Use it for "which key did the user tap". Note that when two keys are down it reports only the lower index, so for independent simultaneous keys test each one with **KEY-DOWN?**.

WAIT-NOKEY ---

Block until the entire keyboard is released (every key up). Used as a debounce step so a key still held from a previous action cannot leak into the next read. It does not touch the stack.

WAIT-NEWKEY --- n

Block until some key goes down, then return its index. It is the blocking counterpart of **KEY-SCAN**.

GET-KEY --- n

A debounced blocking read: **WAIT-NOKEY** then **WAIT-NEWKEY**. It first waits for the keyboard to be clear, then for one fresh press, and returns that key's index. This is the matrix equivalent of a clean "press a key" prompt, free of key-repeat bounce, and the natural input for index-driven menus.

KEY-NAMES --- a

Return the address of the 40-byte table of printable labels, one per key index: the key's character for ordinary keys, and the codes $00, $01, $0D,$20 standing for CAPS, SYMBOL SHIFT, ENTER and SPACE. **.KEY** reads this table; you seldom use it directly.

.KEY n ---

Print the label of key index n: the character for an ordinary key, or the words CAPS, SYMB, ENTER, SPACE for the four special keys. Handy for echoing or debugging a scan, e.g. **GET-KEY DUP . .KEY**.

3.16 Direct Memory Access (DMA)

The following definitions are available after you type **NEEDS DMA**. To forget this library from the dictionary you can type **NO-DMA** (a **MARKER** placed at its head). The zxnDMA is a single-channel DMA controller on the ZX Spectrum Next that moves data between memory and memory, memory and an I/O port, or an I/O port and memory, at full CPU speed and

without CPU involvement in the byte-by-byte transfer. The hardware is programmed by writing seven sequential configuration bytes (registers WR0-WR6) to port $006B; the library hides this register sequencing behind four transfer words that simply take source, destination and length. This library covers only continuous-mode transfers (the whole block moves in one go); burst mode with fixed-time-transfer, used for audio streaming, is reserved for a future extension.

While a continuous-mode transfer is in progress the CPU is blocked until it completes, and level-triggered maskable interrupts cannot fire during that time -- keep this in mind if DMA is used together with the ULA frame interrupt or a CTC channel (see 3.7 Interrupt Service Routine and, for the newer hardware-vectored form, the Hardware Interrupt Mode 2 section). $E000-$FFFF is not off-limits by address: it is the MMU7 window, the single gateway through which any code -- the compiler, this library, or your own -- can reach the full expansion RAM pool (well beyond the 64K directly addressable by the Z80). Its content is simply whichever 8K page a prior **MMU7!** (or an equivalent NEXTREG $57 write) last selected, and that selection is not tied to one purpose: at compile time (**INCLUDE** / **NEEDS** / **LOAD**), dictionary search pages in the heap pages that hold the word headers, but at run time ordinary Forth execution never consults the dictionary again (it runs from already-resolved xt/cfa), so MMU7 is free for other jobs -- paging the **LAYER2** framebuffer, decoding a Heap string via **FAR** (as the **TUTORIAL** library does, keeping filenames in Heap and only an array of heap-pointers in the dictionary), or, most visibly, LED (3.1.2), which pages through as many of the available 8K slots as the file being edited needs, using MMU7 as a window onto up to about 2MB of RAM. A DMA transfer may legitimately target $E000-$FFFF, but only once you know -- and control -- which of these uses currently owns the page mapped there; write before repaging it and you overwrite whatever that owner still needs. Tutorial 057 avoids the question entirely by using **ALLOT**ed scratch buffers in ordinary dictionary space, which need no MMU7 care at all.

Authoritative reference: ZX Spectrum Next Developer's Guide & Reference Manual rev.3, section 3.3 "DMA".

DMA-COPY src dest len ---

Copy len bytes from address src to address dest at full CPU speed; both addresses increment after each byte. The CPU is blocked until the whole transfer completes. This is faster than a CMOVE loop because the hardware, not the Forth inner interpreter, drives the address and count arithmetic. For example, copy 128 bytes from TEST-SRC to TEST-DEST:

```
TEST-SRC TEST-DEST 128 DMA-COPY
```

DMA-FILL byte dest len ---

Fill len bytes starting at dest with the constant value byte. The source address stays fixed on a one-byte internal buffer while the destination address increments; this is faster than a FILL loop for large blocks. For example, fill 128 bytes at FILL-DEST with $AA:

```
$AA FILL-DEST 128 DMA-FILL
```

DMA-OUT src ioport len ---

Send len bytes from memory, starting at src (incrementing), to the fixed I/O port ioport; every byte is written to the same port in sequence. For example, stream 128 bytes to the border-color port:

```
TEST-SRC $FE 128 DMA-OUT
```

DMA-IN ioport dest len ---

Read len bytes from the fixed I/O port ioport into memory, starting at dest (incrementing); every byte comes from the same port in sequence. For example, read 16 bytes from the keyboard port into memory:

```
$FE IN-DEST 16 DMA-IN
```

DMA-STATUS --- b

Read and return the zxnDMA status byte (register WR6 read-status sequence). Bit 5 clear means the last transfer has completed; the other bits report end-of-block, match and ready conditions as documented in the Z80 DMA datasheet.

DMA-DONE? --- f

Convenience wrapper around DMA-STATUS: return a true flag once the last continuous-mode transfer has finished (bit 5 of the status byte is 0).

DMA-ENABLE / DMA-DISABLE / DMA-LOAD / DMA-CONTINUE / DMA-RESET / DMA-RESET-STATUS ---

Low-level WR6 command words, exposed individually for advanced control: enable or disable the DMA channel, (re)load the address/length registers and start a transfer, continue a paused transfer, reset the whole chip, or reset only the status flags. The four transfer words above already sequence DISABLE, the register writes, LOAD and ENABLE for you; use these directly only when composing a transfer sequence of your own.

This library was load-tested in the headless emulator (register sequencing verified byte by byte) but actual data movement on CSpect or real hardware has not yet been confirmed -- treat it as provisional until that verification is done.

See also: §3.7 Interrupt Service Routine, Hardware Interrupt Mode 2, §3.7 Filesystem (DIR), §3.9.1 LED, the Heap Memory Facility,

4 Tecnical specifications

4.1 Z80 CPU Registers

Z80 CPU Registers are used in the in the following way:

AF – Available for normal operations.
BC – *Forth Instruction Pointer:* should be preserved during ROM/OS calls or machine code routines.
DE – *Forth Return Stack Pointer*: should be preserved during ROM/OS calls or machine code routines.
HL – Available as *Work Register*

SP – *Forth Data Stack Pointer*

AF'– Available, sometime used for backup purpose
BC'– Available, used in I/O operations or tricky definitions
DE'– Available, often the Low part when used for 32-bit manipulations
HL'– Available, often the High part when used for 32-bit manipulations

IX – Used to point to the Forth "inner-interpreter": it should always be preserved, but for `TRACE` operations.
IY – Used by ZX System, must be preserved to let the standard keyboard to be served during Interrupts.

4.2 Single Cell 16 bits Integer Number Encoding

A 16 bits *integer* represents an integer number between –32768 and +32767 inclusive. The sign is kept in the most significant bit using the usual two-complement notation. Alternatively, the it represents an *unsigned integer* between 0 and +65535.

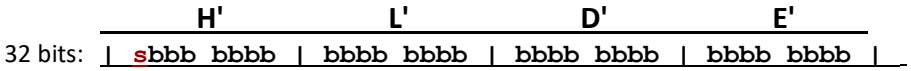
		H	L
16 bit:	HL:	s bbbb bbbb	bbbb bbbb

In the CPU registers, an *integer* is usually operated in H and L where H is the most significant part.
In memory, an *integer* is stored in two contiguous bytes in little-endian convention, that is the lower address has the least significant part, as in the L register. The byte at higher address has the most significant part, as in the H register, standard for Zilog Z80 processor.

4.3 Two cells 32 bits Integer Number Encoding

The second integer format requires two *integers* to form a 32 bits number, referred to as *double*, that allows integers between –2,147,483,648 and +2,147,483,647 where the sign is kept on the most significant bit of the first *integer*.

Imagine a *double precision integer* stored in CPU register in this way:



using register H', L', D' and E', with the most significant part in H', and the least in E'.

Then, on the Calculator Stack the double-integer integer requires four contiguous bytes split into two integers with the most significant integer (HL') on top of Calculator Stack (i.e. in the lower addresses), and the least significant integer (DE') the second element from top, in the higher address, that is the second element from the top. So, it appears as L' H' E' D',

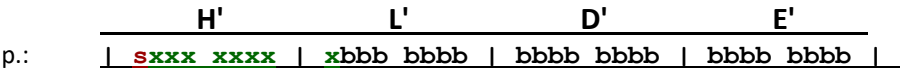
CPU	Calculator Stack
D'	SP + 3
E'	SP + 2
H'	SP + 1
L'	SP + 0 (Top Of Stack)

To adhere to the Standard, in RAM it is kept as L' H' E' D'. See how **2@** and **2!** are defined to understand this fact.

CPU	2VARIABLE
D'	Address + 3
E'	Address + 2
H'	Address + 1
L'	Address + 0

4.4 Double Cell Floating-Point Number Encoding

There is another optional format that use 32 bits as a *double-precision integer*, but all bits are used in a different way to allows representation a *floating point number* approximately between ±0.3E-38 and ±1.7E+38 with 6-7 precision digits. The sign is kept in the most significant bit, the same way as a *double integer*; then eight bits follow as the exponential part, then 23 bits of mantissa or significand. The sign in this position allowing most semantics of *double-precision integers* to deal with the sign of Floating Point numbers too.



See Floating-Point Option section for more details.

4.5 Single Cell 16 bits Heap Pointer Address Encoding

This is Spectrum Next's peculiar 16 bits *Heap Pointer* Address Encoding that leverages on MMU7 i.e. Z80 memory space addresses between $E000 and $FFFF. The three most significant bits represent an 8kibyte-page between 32 and 39 ($20 and $27), lower bits are taken as offset from $E000. A specific definition `>FAR` takes care of converting an heap-pointer address to an $E000 offset and paging to MMU7 the correct 8kibyte of physical RAM. Any NextZXOS call and most of I/O operations restore page 1 at MMU7 , so in most cases data stored in Heap has to be moved to PAD before being used elsewhere.

```
                          H          L
16 bit:      HL:     | pppb bbbb | bbbb bbbb |

          |    Page    |         Offset         |
          | 0010 0ppp  | 111b bbbb | bbbb bbbb |
```

4.6 Dictionary memory structure

From version 1.7, a dictionary definition storage is split into two parts, the name-space in `HEAP` (see §**6.4**), and the code-space in main memory. The name-space of a new definition begins at *heap-pointer address* (§**4.5**) given by **`HP`** (the current Heap Pointer), and the code-space part of it begins at **`HERE`** as usual, but all data previously contained in NFA, LFA and CFA are replaced by a single "backward" heap-pointer to point back to name-space location, as a "mirror".

The linked-list formed by each LFA uses *heap-pointers* to be decoded to actual RAM pointer via `>FAR`. To fully understand an *heap-pointer address* you have to first read § **6.4**: in short, address $0309 can be found at $E309 proved that you fit 8K page #40 to MMU7.

For example the two contiguous definitions **`SWAP`** and **`DUP`** appears in memory as follow (as per build **2026-11-07**, since it's perfectly possible that a different build shows different addresses).

```
Heap memory:
NFA   0305  | 84 | 53 57 41 D0 |     | len  | 'S'  'W'  'A'  'P'  |
LFA   030A  | FC 02 |               | 02FC |  heap-pointer to previous NFA definition
CFA   030C  | 15 69 |               | 6915 |  xt memory-address of this definition

Main memory:
Mirror 6913 | 0C 03  |              | 030C |  backward-heap-pointer to SWAP's CFA
  xt   6915 |   E1   |     E3       |   E5    |  DD  E9  |
            | pop hl | ex (sp),hl | push hl |  jp (ix) |

Heap memory:
NFA   030E  | 83 | 44 55 D0 |       | len  | 'D'  'U'  'P'  |
LFA   0312  | 05 03 |                | 0305 |  heap-pointer to previous NFA definition SWAP
CFA   0314  | 1C 69 |                | 691C |  xt memory-address of this definition

Main memory:
Mirror 691A | 14 03  |              | 0314 |  backward-heap-pointer to DUP's CFA
  xt   691C |   E1   |   E5    |   E5    |  DD E9   |
            | pop hl | push hl | push hl |  jp (ix) |
```

You can verify yourself all of that by typing some commands.

```
SEE SWAP
Nfa: E305 84
Lfa: E30A 02FC TUCK
Cfa: 6915 E5E3
```

```
6915  E1 E3 E5 DD  E9 14 03 E1  ace]i  a
691D  E5 E5 DD E9  1C 03 F1 E1  ee]i  qa
$E305 9 DUMP
E305  84 53 57 41  D0 FC 02 15    SWAP|
E30D  69 83 44 55  D0 05 03 1C   i DUP
$6913 2 DUP
6913  0C 03 E1 E3  E5 DD E9 14     ace]i

SEE DUP
Nfa: E30E 83
Lfa: E312 0305 SWAP
Cfa: 691C E5E5
691C  E1 E5 E5 DD  E9 1C 03 F1  aee]i  q
6924  E1 E3 F5 E5  DD E9 25 03  acue]i%
$E30E 8 DUMP
E30E  83 44 55 D0  05 03 1C 69    DUP   i
$691A 2 DUP
691A  14 03 E1 E5  E5 DD E9 1C     aee]i
```

4.6.1 Heap-pointers and resolved addresses

Since names live in the heap, two kinds of 16-bit value describe a dictionary entry. A heap-pointer (`ha`) uses bits 15-13 to select one of the eight 8K heap pages and bits 12-0 as the offset from $E000; a resolved address lies in the MMU7 window $E000-$FFFF and is meaningful only while the right page is fitted there. **FAR** turns the first into the second, fitting the page as a side effect. Both share the same cell type, so the system tells them apart by position, never by value.

The rule is simple. Every cell stored inside the dictionary structure holds a heap-pointer: the link field (LFA), the vocabulary cell reached through **CONTEXT @** and **CURRENT @**, the mirror cell just before each CFA, **HP** itself and the cells saved by **MARKER**. Every nfa passed on the stack is a resolved address: what **LATEST**, **NFA** and **<NAME** return, and what **ID.** and **PFA** expect. Following a link therefore always takes an explicit **FAR**: **' DUP >BODY LFA @ FAR ID.** prints "**SWAP**", the word defined just before **DUP.**

A resolved nfa is only as good as the page currently in MMU7: anything that remaps it in the meantime (another **FAR**, **HEAP**, an explicit **MMU7!**, a call into NextZXOS) leaves it pointing somewhere else. Keep it on the stack for as short a time as possible. Printing is safe: since build 2026-06-12 (**EMITC**) preserves the MMU7 page across its ROM call.

Up to build 2026-09-25 the core did not follow this rule strictly: **ID.**, **PFA** and **<NAME** accepted either form and guessed which one they had received with a numeric threshold. **?HEAP_PTR** took any non-zero value below $6300 for a heap-pointer, **?>HEAP** applied **FAR** accordingly, and the machine code of **(FIND)** carried its own copy of the test, below $6000. The guess dates back to the move from version 1.6, which kept names in the dictionary, to version 1.7, the first to keep them in the heap: during that transition the same core had to read both layouts, and the threshold simply meant "just below the dictionary origin". Once every header lived in the heap the guess no longer had anything to tell apart, but it still capped the heap: a definition whose name was laid down with HP at $6000 or above made the whole dictionary unsearchable. Since build 2026-09-26 the threshold is gone, together with **?HEAP_PTR**, **?>HEAP** and **?IN_MMU7**, and names can use the whole heap.

Code written for earlier builds that passed a link straight to **ID.** or **PFA** (for example **LFA @ ID.**) must now resolve it first with **FAR**. The library has been adjusted accordingly (**SEE**, **HIDE-WORD**, **.VOCAB**), and **:NONAME** now compiles the mirror cell, so **<NAME** works on an anonymous xt as well.

5 Error messages

Error message strings are stored in Screens 4 to 8, which are therefore reserved: message #n is line n MOD 16 of Screen 4 + n/16, so Screen 5 begins with #16, Screen 6 with #32, Screen 7 with #48 and Screen 8 with #64. The first line of Screens 5 to 8 is a header comment, so #16, #32, #48 and #64 are not messages; Screen 4 is the exception, since its first line has to be message #0, and it carries no header. Screens 2 and 3 hold the negative messages -32 to -1, aligned with the standard **THROW** codes (see the second table below). **9 LOAD** prints the whole list page by page. See **MESSAGE** (§ 6.1) and tutorial 062.

Messages marked as free below have no text yet. A library module that needs a message takes one of them and reserves it in a comment at the top of its source, as lib/LOCALS.f does for #57 to #60.

Code	Message	Meaning
#0	is undefined.	Definition not found or number conversion invalid with this **BASE**
#1	Stack is empty.	Returning to “ok” prompt, the interpreter detects this situation
#2	Dictionary full.	
#3	No such a line.	Line number should be between 0 and 15
#4	has already been defined.	Creating a definition whose name already exists.
#5	Invalid stream.	Used by Microdrive version
#6	No such a block.	Block number should be between 1 and **#SEC**
#7	Stack full.	The interpreter detects this situation
#8	Old dictionary is full.	
#9	Tape error.	Not used anymore
#10	Wrong array index.	
#11	Invalid floating point.	Issued during floating invalid operation
#12	Heap full.	
#13	Division by zero.	Issued during floating invalid operation
#14	Patching the wrong definition.	
#15	[BREAK] was pressed.	Issued when BREAK stops an operation, e.g. PLAY
#17	Can't be executed.	
#18	Can't be compiled.	
#19	Syntax error.	Mismatch in a structure.
#20	Bad definition end.	Mismatch in **DO-LOOP** structure
#21	is a protected definition.	Trying to forget to an address below **FENCE**
#22	Aren't loading now.	Issued by a misplaced **-->**.
#23	Forget across vocabularies.	You have to use **DEFINITIONS** beforehand
#24	RS loading error.	
#25	Cannot open stream.	
#26	Error at postit time.	Issued by **ASSEMBLER** dictionary operation
#27	Inconsistent fixup.	Issued by **ASSEMBLER** dictionary operation
#28	Unexpected fixup/commaer.	Issued by **ASSEMBLER** dictionary operation
#29	Commaer data error.	Issued by **ASSEMBLER** dictionary operation
#30	Commaer wrong order.	Issued by **ASSEMBLER** dictionary operation
#31	Programming error.	Issued by **ASSEMBLER** dictionary operation
#33	Programming error.	Issued by **ASSEMBLER** dictionary operation
#34	Checksum error.	Issued by **ASSEMBLER** dictionary operation
#35	F_GETLINE pos error.	Issued by **INCLUDE** and **NEEDS**: **F_GETLINE** cannot read the file position or the next line
#36	F_GETLINE seek error.	Issued by **INCLUDE** and **NEEDS**: **F_GETLINE** cannot move to the beginning of the next line
#37	Directory not found.	Issued by **CD**
#38	Not a BMP file.	
#39	NextZXOS Opendir error.	
#40	NextZXOS Out of memory.	
#41	NextZXOS Open error.	Issued to signal **F_OPEN** fails to open a file
#42	NextZXOS Close error.	Issued to signal **F_CLOSE** fails to close a file
#43	File not found.	Issued by **INCLUDE** or **NEEDS**
#44	NextZXOS DOS call error.	Issued to signal a generic NextZXOS DOS call error
#45	NextZXOS Pos error.	Issued to signal **F_SEEK** fails to change current file position
#46	NextZXOS Read error.	Issued to signal **F_READ** fails to read from file
#47	NextZXOS Write error.	Issued to signal **F_WRITE** fails to write to file
#49	Test suite errors.	Heading of the **TESTING** suite messages
#50	Incorrect result.	Used by **TESTING** suite
#51	Wrong number of results.	Used by **TESTING** suite

#52	Cell number before '->' does not match ...}T spec.	Used by **TESTING** suite
#53	Cell number before AND after '->' does not match.	Used by **TESTING** suite
#54	Cell number after '->' below …}T does not match.	Used by **TESTING** suite
#55	(free)	
#56	Real-Time-Clock module not present.	Issued when **M_GETDATE** fails (see tutorial 072)
#57	LOCALS: bad count.	Issued by **LOCALS-FOR** and { }: no locals, or more than 8
#58	LOCALS: no scope declared.	Issued by **LOCALS** without a **LOCALS-FOR** before the definition
#59	LOCALS: scope not adjacent.	Issued by **LOCALS** when its **LOCALS-FOR** is not the one right before the definition
#60	LOCALS: misplaced { or }.	Issued for a **}** without **{**, a **{** without **}**, or a second **--** inside **{ }**
#61	(free)	
#62	(free)	
#63	Abort.	Not issued by the core or the library at present
#65	Microdrive full.	Issued by the Microdrive cartridge facility (**NEEDS MDR**)
#66	Not a program.	Issued by the Microdrive cartridge facility when the file to run is not a program
#67	defined after NEEDS ZAP.	Issued by **ZAP** and **ZAP"**: the definition would not be in the standalone image
#68 to #78	(free)	
#79	msg #79	Placeholder that marks the end of the table

Negative messages, -32 to -1, are kept in Screens 2 and 3. They follow the standard THROW codes of Forth-2012, so that an uncaught n THROW prints a meaningful text; nothing in the vForth core issues them by itself. Where a standard code names a condition vForth already has, the text is the same as the positive one (-3 as #7, -4 as #1, -8 as #2, -10 as #13, -13 as #0). The first line of each Screen is a header comment, so -32 and -16 are not messages.

Code	Message	Meaning
#-1	Aborted.	
#-2	Aborted with message.	
#-3	Stack full.	
#-4	Stack is empty.	
#-5	Return stack full.	
#-6	Return stack is empty.	
#-7	DO loops nested too deeply.	
#-8	Dictionary full.	
#-9	Invalid memory address.	
#-10	Division by zero.	
#-11	Result out of range.	
#-12	Argument type mismatch.	
#-13	is undefined.	
#-14	is a compile-only word.	
#-15	Invalid FORGET.	
#-17	Pictured numeric output string overflow.	
#-18	Parsed string overflow.	
#-19	Definition name too long.	
#-20	Write to a read-only location.	
#-21	Unsupported operation.	
#-22	Control structure mismatch.	
#-23	Address alignment exception.	
#-24	Invalid numeric argument.	
#-25	Return stack imbalance.	
#-26	Loop parameters unavailable.	
#-27	Invalid recursion.	
#-28	User interrupt.	
#-29	Compiler nesting.	
#-30	Obsolescent feature.	
#-31	was not defined by CREATE.	

6 The Dictionary

Despite its title, this dictionary documents two kinds of definitions. Core definitions are compiled into forth18e.bin and are always present at startup; they need no loading at all. Extension definitions live as source files in the inc/ and lib/ directories of the SD card and enter the dictionary only on request, via NEEDS: their entry ends with the sentence "This definition is available after NEEDS cccc". Every entry without that closing sentence describes a core definition. The interactive HELP (sec. 2.2.1) follows the same convention, with the reminder "Available after NEEDS cccc" on the last line of its pages.

This list is not, however, the only place extension words are documented, and most of them are not here at all. Whenever a library module is large or coherent enough to deserve its own narrative — Graphics mode and Layer facility, Mouse, Floating-Point Option, the ASSEMBLER vocabulary, AY sound chip facility, Keyboard input, the Heap Memory Facility — it gets a dedicated section of its own earlier (or, for Heap and Testing, as a titled subsection later) in this manual, with its own worked examples. Such a word is documented once, in that section, and is not repeated here. This alphabetical list is reserved for the core vocabulary plus the smaller extension words that do not warrant a section of their own (TO, VALUE, WITHIN, THROW, ENUMERATED, THRU, ...). When looking for a word and not finding it here, check the Table of Contents for a section named after the feature it belongs to before concluding it does not exist.

6.1 The “core” dictionary

'*0x00*' --- (immediate)

This is a “ghost” definition executed by INTERPRET to go back to the caller once the text to be interpreted ends. This definition allows you to use a **0x00** (NUL ASCII) as the end-of-text indicator in the input text stream.

! n a ---

stores an integer n in the memory cell at address a and a + 1. Pronounced “store”.
Zilog Z80 microprocessor is a little-endian CPU that holds lower byte at lower address and higher byte in the higher address.

!CSP ---

saves the value of SP register in CSP user variable. It is used by : and ; for syntax checking. ~~Also, CASE use it for the same purpose.~~

d1 --- d2

From a double-precision integer d1 it produces the next ASCII character to be put in an output string using HOLD. The number d2 is d1 divided by the current BASE and is kept for subsequent elaborations. This definition is used between <# and #> .
See also #S.

#> d --- a u

terminates a numeric conversion started by <# . This definition removes **d** and leaves the values suitable for TYPE.
See also # and #S.

#BUFF --- n

Constant, the number of available buffers. Buffers are located at address between FIRST @ and LIMIT @.

#S d1 --- d2

This definition is equivalent of a series of # that is repeated until d2 becomes zero. It is used between <# and #>.

#SEC **---** **n**

This is a constant that gives the number maximum number of a BLOCK, so that half #SEC is the maximum number of a Screen. This system comes with a file "!Blocks-64.bin" that keeps persistency of all BLOCKs. Try #SEC LIST.

' **---** **xt**

Pronounced "tick". Used in the form

' cccc

this definition leaves the **cfa** of definition cccc, that is its **xt** or the value to be compiled or passed to EXECUTE. If the definition cccc is not found after the CURRENT and CONTEXT search phases, then an error **#0** is raised, that is the message "cccc is undefined". In a previous version of this Forth, this definition returned **pfa**: we changed this previous standard to return **cfa**.

(**---** **(immediate)**

Enclose a comment. Used in the form

(cccc)

ignores what is between brackets. The space after (is not considered in cccc. The comment must be delimited in the same row with a closing) followed by a space or the end of line.

(+LOOP) **n** **---**

This is the primitive low-level definition compiled by +LOOP.

(.") **---**

This is the primitive definition compiled by ." and .(It executes TYPE.

(;CODE) **---**

This is the primitive definition compiled by ;CODE. It over-writes the **cfa** of LATEST definition to make it point to the machine code starting from the following address.

(?DO) **---**

This is the primitive definition compiled by ?DO.
At compile-time it compiles the **cfa** of (?DO) followed by an offset as for BRANCH used to jump after the whole ?DO ... LOOP structure in case the limit equals the initial index, otherwise it is equivalent to (DO).

(?EMIT) **c1** **---** **c2**

Decodes the character c1 using the following table. It is used internally by EMIT.

$06 → print-comma
$07 → bell rings
$08 → back-space
$09 → tabulator
$0D → carriage return

$0A → new line (emitted as a $0D on the fly)
For not listed character, c2 is equal to c1.

(ABORT) **---**

Definition executed in case of error issued by ERROR when WARNING contains a negative number. This definition usually executes ABORT but can be patched with some user defined one at the PFA of (ABORT).

(COMPARE) **a1 a2 n -- b**

This definition performs a lexicographic compare of n bytes of text at address a1 with n bytes of text at address a2. The compare is case-sensitive or case-insensitive based on the last execution of CASEON and CASEOFF.
When executed, this definition returns a numeric value

0	if strings are equal
+1	if string at a1 greater than string at a2
-1	if string at a1 less than string at a2

Up to build 2026-10-20 it was the core primitive COMPARE was built on; since build 2026-10-28 the roles are reversed: COMPARE is in the core and (COMPARE) is kept for compatibility as : (COMPARE) TUCK COMPARE ;
Prefer COMPARE in new code. This definition is available after NEEDS (COMPARE).

(DO) **---**

This is the primitive compiled by DO.

(FIND) **a1 a2 --- cfa b tf**
--- ff

Searches in the dictionary starting from address a2 a definition which text name is kept at address a1; returns a cfa, the first byte b of nfa and a tf on a successful search; elsewhere a ff only.
The search is case-sensitive or case-insensitive based on the last execution of CASEON and CASEOFF.
Address a2 must be the nfa of the first definition involved in the search in the vocabulary.
Byte b keeps the length of the found definition in the least significant 5 bits, bit 6 is the IMMEDIATE flag. Bit 5 is the SMUDGE bit. Bit 7 is always set to mark the beginning or end of the nfa.
See also CASEON and CASEOFF.

(LEAVE) **---**

Direct procedure compiled by LEAVE that discards the current DO-LOOP frame and executes an unconditional jump. The memory cell following (LEAVE) contains the offset to be relatively added to the Instruction Pointer to jump after the corresponding (LOOP) or (+LOOP).

(LINE) **n1 n2 --- a b**

Retrieves Line n1 of Screen n2 and send it to buffer. It returns the address a within the buffer and a counter b that is C/L (=64) meaning a whole line.

(LOOP) **---**

This is the primitive compiled by LOOP. See also DO and +LOOP.

(MAP) a2 a1 n c1 --- c2

Translate character c1 using mapping string a1 and a2. If c1 is present within string a1 then the corresponding position within string a2 is taken as translation. If c1 is not present within string a1, then it is not translated, and c2 remains equal to c1. n is the length of strings a1 and a2.

For example, the following code is used to translate the characters **: ? / * | \ < > "** – that are illegal when used in a filename – to the corresponding legal characters chosen among **% ^ % & $ _ { } ~** .

```
here to ndom^     (   : ? / * | \ < > "   )
  char :  c,   char ?  c,   char /  c,   char *  c,
  char |  c,   char \  c,   char <  c,   char >  c,   char "  c,

here to ncdm^     (   % ^ % & $ _ { } ~   )
  char _  c,   char ^  c,   char %  c,   char &  c,
  char $  c,   char _  c,   char {  c,   char }  c,   char ~  c,

: needs-ch ( a -- ) \ Replace illegal characters in filename counted-zstring
  count bounds
  Do
      [ ncdm^ ] Literal
      [ ndom^ ] Literal
      9 i c@ (map) i c!
  Loop
;
```

(NEXT) --- a

Constant. It is the address of "next" entry point for the **Inner Interpreter**. When creating a new definition using machine code, the last op-code should be an unconditional jump to this address. If the newly created definition wants to return an *integer* value on TOS, it has to do it beforehand.

This Forth implementation *always* keeps (NEXT) value in **IX register.** For example, to create two definitions that disables and enables interrupts, without an ASSEMBLER, you could use the following snippet:

```
CODE   ISR-DI   HEX
  F3 C,          \  di
  DD C, E9 C,    \  jp (ix)
  SMUDGE         \  now a dictionary search will find this definition

CODE   ISR-EI   HEX
  FB C,          \  ei
  DD C, E9 C,    \  jp (ix)
  SMUDGE         \  now a dictionary search will find this definition
```

(NUMBER) d a --- d2 a2

Converts the ASCII text at address a+1 in a double-precision integer using the current BASE. Number d2 is left on top of stack for any subsequent elaborations, a2 is the address of the first non-converted character.
Used by NUMBER and (EXP) in the Floating-Point Option.

(PREFIX) a --- a2

During a NUMBER interpretation, if the character at address a+1 is a "$" or a "%" modifies BASE accordingly and increments a: "$" for hexadecimal (base 16), "%" for binary (base 2).

(SGN) a --- a2 f

Determines if the character at address `a+1` is a sign (+ o −) and if found increments `a`. The flag `f` indicates the sign: `ff` for a positive sign `+` or no sign at all, `tf` for a negative sign `-`. If a is incremented then variable `DPL` is incremented as well. Used by da `NUMBER` and `(EXP)` in the Floating-Point Option.

*** n1 n2 --- n3**

Computes the product of two integers.

***/ n1 n2 n3 --- n4**

Compute (`n1` · `n2`) / `n3` using a double-precision integer for the intermediate value to avoid precision loss.

***/MOD n1 n2 n3 --- n4 n5**

Leaves the quotient `n5` and the reminder `n4` of the operation (`n1`· `n2`) / `n3` using a double-precision integer for the intermediate to avoid precision loss.

+ n1 n2 --- n3

Leaves the sum of two integer.

+! n a ---

Adds to the cell at address `a` the number `n`. It is the same as the sequence `a @ n + a !`

+- n1 n2 --- n3

Computes `n3` as `n1` with the sign of `n2`. If `n2` is zero, it means positive.

+BUF a1 --- a2 f

Advances the address of the buffer from `a1` to `a2`, that is the next buffer. The flag `f` is false if `a2` is the buffer pointed by `PREV`.

+LOOP n1 --- (run time)
a n2 --- (compile time)

Used in colon definition in the form

DO ... n1 +LOOP

At run-time `+LOOP` checks the return to the corresponding DO, `n1` is added to the index and if the index did not cross the boundary between the loop limit minus one and the loop limit, the execution jumps back to the beginning of the loop. Otherwise the execution leaves the loop. On leaving the loop, the parameters are discarded and the execution continues with the following definition.
At compile-time `+LOOP` compiles `(+LOOP)` and a jump is calculated from `HERE` to `a` which is the address left on the stack by `DO`. The value `n2` is used internally for syntax checking.

+ORIGIN n --- a

Returns the address n bytes after the “origin”. In this build the origin is 6400h. Used rarely to modify the boot-up

parameters in the origin area.

+TO **n** **---** **cccc**
Used in the form

n +TO cccc

If not compiling, add the value `n` to `cccc` . At compile-time it compiles a literal pointer to `cccc`'s PFA followed by a plus-store-command (`+!`) so that later, at run-time the literal is used by the `!` definition to alter `cccc` value.
Definition `cccc` was created via `VALUE`. See also `TO`. This definition is available after `NEEDS +TO`.

, **n** **---**
It puts `n` in the following cell of the dictionary and increments `DP` (dictionary pointer) of two locations.

," **---**
Compile a "Counted-ZString". It calls `WORD` to read characters from the current input stream up to a delimiter `"` and stores such a string at `HERE`. In a "Counted-ZString" the length of the string is stored as the first byte and the string itself ends with a NUL character (0x00). For example

```
," Hello"
```

compiles: **05 48 65 6C 6C 6F 00**
where 05 is the length of "Hello" string which is followed by a 00 'nul' character.

- **n1 n2** **---** **n3**
Computes `n3` = `n1` – `n2` as the difference from the penultimate and the last number on the stack.

--> **---**
Continues the interpretation in the next Screen during a `LOAD` .

-1 **---** **n**
This is the constant value `-1` that in this implementation is 0FFFFh. Compiling a constant result in a faster execution than a literal.

-?EXECUTE **---**
Simple conditional execution semantics, used in the form

-?EXECUTE cccc

it invokes `-FIND` to search for **cccc** definition. If found **cccc** is executed. If not found **cccc** is ignored.
You have to import this definition via **NEEDS -?EXECUTE.**

-DUP **n** **---** **n n (non zero)**
n **---** **n (zero)**
Duplicates `n` if it is non zero. This definition is available only for backward compatibility. See also `?DUP`.

-FIND **--- xt b tf (ok)**
--- ff (ko)

Used in the form `-FIND cccc`.
Parse a word (a non-space character sequence delimited by spaces) from the current input stream, storing it at address `HERE 1+` and its length at `HERE`. Then, it run a search for it in the `CONTEXT` vocabulary first, then in the `CURRENT` vocabulary. If such a definition is found, it leaves the `cfa` of the definition, its length-byte `b` and a `tf`. Otherwise only a `ff`.

-TRAILING a n1 --- a n2

It can be used to trim trailing spaces from a counted string described by `a`, `n1`. This definition assumes that a string `n1` characters long is already stored at address `a` containing a word i.e. a characters sequence delimited by spaces. It determines `n2` as the position of the first delimiter after the word.

. n ---

Emits the integer `n` followed by a space.

." **--- (immediate)**

Used in the form

```
." cccc"
```

At compile-time, within a colon-definition, it accepts text from the input sources until a quote character (") is encountered, then it compiles the primitive to output the text followed by the string `cccc` . The text `cccc` is prepended by a length-counter that `TYPE` will use at run-time.
When interpreted, i.e. outside a colon-definition, immediately emits the text to output.

.(**--- (immediate)**

Used in the form

```
.( cccc)
```

acting as `." cccc "` but the string is delimited by a closed-parenthesis.

.C c --- (immediate)

Used in the form

```
c .C xxxx C
```

acting as `." xxxx"` but the string is delimited by character `c`. It is a more generic form of `.(` and `."` that, in fact, use this definition as their primitive.

.LINE n1 n2 ---

Sends line `n1` of **BLOCK** `n2` to the current peripheral ignoring the trailing spaces.

.R n1 n2 ---

Prints a number `n1` right aligned in a field `n2` character long, with no following spaces. If the number needs more than

n2 characters, the excess protrudes to the right.

/ n1 n2 --- n3

Computes n3 = n1/ n2, the quotient of the integer division. This system uses floored division via M/MOD and implements UM/MOD in machine-code, FM/REM and SM/MOD as derived definitions. See M/MOD for details.

/MOD n1 n2 --- n3 n4

Computes the quotient n4 and the reminder n3 of the integer division n1/ n2. The reminder has the sign of n1. This system uses floored division via M/MOD and implements UM/MOD in machine-code, FM/REM and SM/MOD as derived definitions. See M/MOD for details.

0 --- n

This is a constant value zero. Compiling a constant results in a faster execution than a literal.

0< n --- f

Leaves a tf if n is less than zero, ff otherwise.

0= n --- f

Leaves a tf if n is not zero, ff otherwise. It is like a NOT n.

0> n --- f

Leaves a tf if n is greater than zero, ff otherwise.

0BRANCH f ---

Direct procedure that executes a conditional jump. If f is zero the offset in the cell following 0BRANCH is added to the Instruction Pointer to jump forward of backward.
It is compiled by IF, UNTIL and WHILE.

1 --- n

Constant value 1. Compiling a constant results in a faster execution than a literal.

1+ n1 --- n2

Increments by one the number on TOS.

1- n1 --- n2

Decrements by one the number on TOS.

2 --- n

Constant value 2. Compiling a constant results in a faster execution than a literal.

2! **d a** **---**
n-lo n-hi a **---**
Stores the double-precision integer held on TOS to address a.

2* **n1** **---** **n2**
Doubles the number on TOS.

2+ **n1** **---** **n2**
Increments by two the number on TOS.

2- **n1** **---** **n2**
Decrements by two the number on TOS.

2/ **n1** **---** **n2**
Halves the number on TOS.

2@ **a** **---** **d**
a **---** **n-lo n-hi**
Fetches the double-precision integer at address `a` to TOS.

2CONSTANT **d** **---** **(immediate)** **(compile time)**
--- **d** **(run time)**
This is a defining definition that creates a double-precision constant. Used in the form

d 2CONSTANT cccc

it creates the new definition `cccc` which `pfa` holds the number **d**. When `cccc` is later executed it put **d** on TOS.
This definition is not available at startup, it has to be loaded via `NEEDS 2CONSTANT`.

2DROP **d** **---**
n1 n2 **---**
Discards a double-precision integer from the TOS, i.e. discards the top two integer.

2DUP **d** **---** **d d**
n1 n2 **---** **n1 n2 n1 n2**
Duplicates the double-precision integer on TOS, i.e. duplicates in order the two top integer.

2OVER **d1 d2** **---** **d1 d2 d1**
n1 n2 n3 n4 --- **n1 n2 n3 n4 n1 n2**
Copies to TOS the second double-precision integer from top.
This definition isn't available at startup and must be included via `NEEDS 2OVER`.

2ROT **d1 d2 d3 --- d2 d3 d1**
n1 n2 n3 n4 n5 n6 --- n3 n4 n5 n6 n1 n2

Rotates the three top double-precision integers, taking the third and putting it on top. The other two double-precision integers are pushed down from top by one place. This definition isn't available at startup and must be included via NEEDS 2ROT.

2SWAP **d1 d2 --- d2 d1**

Swaps the two double-precision integers on TOS.

2VARIABLE **--- (immediate) (compile time)**
--- a (run time)

This is a defining definition used in the form:

```
2VARIABLE cccc
```

creates the new definition cccc with the pfa capable to hold a double-integer. When cccc is executed, it puts on TOS the pfa of cccc that is the address that holds the double-integer value.
When used in the form

```
cccc 2@
```

the content of the double-precision variable cccc is left on TOS.
When used in the form

```
d cccc 2!
```

the double-precision value on TOS is stored to the double-precision variable cccc.
This definition is not available at startup, it must be loaded via NEEDS 2CONSTANT.

3 **--- n**

Constant value 3. Compiling a constant results in a faster execution than a literal.

3DUP **n1 n2 n3 --- n1 n2 n3 n1 n2 n3**

Duplicates the three top integer on Stack. This definition is not available at startup, it must be loaded via NEEDS 3DUP.

: **---**

This is a defining definition that creates and begins a colon-definition. Used in the form

```
: cccc ... ;
```

creates in the dictionary a new definition cccc so that, when later invoked, it executes the sequence of already existing definitions '...'. The CONTEXT vocabulary is set to be the CURRENT and compilation continues while the value of STATE is non-zero. Definitions having the bit 6 of its length-byte set are immediately executed instead of being compiled.

:NONAME **--- xt**

This is a defining definition that creates and begins a name-less colon-definition. It returns the xt of the word being defined. Such xt should be kept in some way, for example as a CONSTANT. Used in the form, for example

```
:NONAME   ...   ;
CONSTANT cccc
```

This definition is not available at startup, it must be loaded via NEEDS :NONAME but the source-file that contains this definition is named %noname.f

; --- (immediate)

Ends a colon-definition and stops compilation.. It compiles EXIT and executes SMUDGE to make the definition accessible to subsequent search.

;CODE --- (immediate)

Used in the form

```
: cccc ... ;CODE
```

terminates a colon definition stopping compilation of definition cccc and compiling (;CODE).
Usually ;CODE is followed by a suitable machine-code sequence. It is used to define a creator of new kinds of definitions.

;S ---

This definition is obsolete and kept for backward compatibility reasons only. It is usually the last xt compiled in a colon definition by ; and it does the action of returning to the calling definition. It is used to force the immediate end of a loading session started by LOAD.
Obsolete, prefer EXIT. This definition isn't available at startup and must be included via NEEDS ;S sequence.

< n1 n2 --- f

Leaves a tf if n1 is less than n2, ff otherwise.

<# ---

Sets HLD to the value of PAD. It is used to format numbers using #, #S, SIGN and #>. The conversion is performed using a double-precision integer, and the formatted text is kept in PAD.

<> n1 n2 --- f

Leaves a tf if n1 isn't equal to n2, ff otherwise. This definition isn't available at startup and must be included via NEEDS <> sequence and the file loaded is {}.f but usually I prefer simpy using the - (difference operator).

<BUILDS ---

Deprecated, the standard is the sequence **CREATE** ... **DOES>** instead, but **<BUILDS** is kept for backward compatibility, but it's simply a synonym for **CREATE**. This definition is used in a colon definition in the form

```
: cccc ... <BUILDS ... DOES> ... ;
```

or the standard:

```
: cccc ...  CREATE ... DOES> ... ;
```

Subsequent execution of **cccc** in the form

cccc nnnn

creates a new definition **nnnn** with an high-level procedure that at run-time calls the part of **cccc** after **DOES>** . When **nnnn** is later executed, the PFA of **nnnn** is put on TOS and the part after **DOES>** is executed.
<BUILD and **DOES>** allow writing high-level procedures instead of using machine code such as **;CODE** would require.

Obvious uses are new vocabularies (**ASSEMBLER**), multidimensional arrays, closures, and other compiling operations. The "Floating Point Option Library" available via **NEEDS FLOATING** provides a good example of use of this structure.
~~**<BUILD** differs from **CREATE** because it allocate the pointer to the **DOES>** part of the defining word~~.

<NAME xt --- nfa
Converts a `xt` or `cfa` in its `nfa`. It is the same as `>BODY NFA` sequence. Reads the heap-pointer stored just before the CFA and resolves it with FAR, so MMU7 is left on the page holding the name.
See also: `CFA`, `LFA`, `NFA`, `PFA`, `>BODY`.

= n1 n2 --- f
Leaves a `tf` if `n1` equals to `n2`, `ff` otherwise.

> n1 n2 --- f
Leaves a `tf` if `n1` is greater than `n2`, `ff` otherwise.

>BODY xt --- pfa
Converts a `cfa` in its `pfa`.
See also: `CFA`, `LFA`, `NFA`, `PFA`, `<NAME`.

>IN --- a
User variable that keeps track of text position within an input buffer. `WORD` uses and modifies the value of `>IN` that is incremented when consuming input buffer.

>NUMBER d a u --- d2 a2 u2
This is the standard numeric conversion routine available for completeness only after `NEEDS >NUMBER`. This definition converts digits from the string `a`, `u` accumulating digits in number `d`. Conversion stops when any character that is not a legal digit is encountered returning the result `d2` and the string parameters `a2` and `n2` for the remaining characters in the string. For historical reasons, this system doesn't use `>NUMBER`, instead it uses a non-standard version definition `(NUMBER)`.

>R n ---
Takes an integer from TOS and puts it on top of the Return Stack. It should be used only within a colon-definition *and* the use of `>R` should be balanced with a corresponding `R>` within the same colon-definition.

? a ---
Prints the content of cell at address `a`. It is the same as the sequence: `a @` .

?COMP **---**

Raises an error message #17 if the current STATE is not compiling state.

?CSP **---**

Raises an error message #20 if the value of CSP is different from the current value of SP register. It is used to check the compilation in a colon definition.

?DO **n1 n2** **---** **(immediate)** **(run time)**
--- **a n** **(compile time)**

Used in a colon definition in the form

```
?DO ... LOOP
?DO ... n3 +LOOP
```

It is used as DO to put in place a loop structure, but at run-time it first checks if n1 = n2 and in that case the loop is skipped. At run-time ?DO starts a sequence of definitions that will be repeated under control of an initial-index n2 and a limit n1. ?DO consumes these two value from stack and the corresponding LOOP increments the index. If the index did not cross the boundary between the loop limit minus one and the loop limit, then the executions returns to the corresponding ?DO, otherwise the two parameters are discarded and the execution continues after the LOOP.
The limit n1 and the initial value n2 are determined during the execution and can be the result of other previous operations. Inside a loop the definition I copies to TOS the current value of the index.
Se also: I, DO, LOOP, +LOOP, LEAVE. In particular LEAVE allows leaving the loop immediately.
At compile-time ?DO compiles (?DO) followed by an offset like BRANCH and leaves the address of the following location and the number n to syntax-check

?DO- **[a1 n1] a n** **---**

This is a peculiar definition equivalent to BACK but fitted for ?DO. It computes and compiles a relative offset from a to HERE and, in the case, it completes the BRANCH part previously compiled by ?DO that left a1 and n1. It is used by LOOP, +LOOP. If the loop begins with DO then a1 and n1 won't be there and no BRANCH will be compiled.

?DUP **n** **---** **n n (non zero)**
n **---** **n (zero)**

Duplicates the value on TOS if it is not qual to zero. This is the same as –DUP.

?ERROR **f n** **---**

Raises an error message # n if f is true.

?EXEC **---**

Raises an error message #18 if we aren't compiling.

?LOADING **---**

Raises an error message #22 if we aren't loading. It show the illegal use of --> .

?PAIRS **n1 n2** **---**

Raises an error message #19 if n1 is different from n2. It is used for syntax checking by the definitions that completes the

construction of structures DO, BEGIN, IF, CASE.

?STACK **---**

Raises an error message #1 if the stack is empty and we tried to consume an element from the calculator stack.
On the other hand, an error message #7 if the stack is full.

?TERMINAL **---** **f**

Tests the keyboard for a [BREAK] key-press. Leaves a tf if the [BREAK] key is pressed, ff otherwise. Useful to stop an indefinite loop, for example:

```
BEGIN ... ?TERMINAL UNTIL
```

@ **a** **---** **n**

Reads cell at address a and put an integer on TOS.

ABORT **---**

Clears the stack and pass to prompt command, ~~prints the copyright message~~ and returns the control to the human operator executing QUIT.

ABORT" **f** **---** **(run time)**

Used in a colon definition in the form

```
... ABORT" cccc" ...
```

At run-time if TOS is non-zero, prints the message cccc and performs **ABORT** to return to command-prompt. If TOS is zero does nothing and passes to the instruction after the message.

ABS **n** **---** **u**

Leaves the absolute value of n.

ACCEPT **a n1** **---** **n2**

Transfers characters from the input terminal to the address a for n1 location or until receiving a 0x13 “CR” character. A 0x00 “null” character is added. It leaves on TOS n2 as the actual length of the received string. More, n2 is also copied in SPAN user variable. See also QUERY.

AGAIN **---** **(immediate)** **(run time)**
a n **---** **(compile time)**

Used in colon definition in the form

```
BEGIN ... AGAIN
```

At run-time AGAIN forces the jump to the corresponding BEGIN and has no effect on the calculator stack. The execution cannot leave the loop (at least until a R> is executed at a lower level).
At compile-time AGAIN compiles BRANCH with an offset from HERE to a. The number n is used for syntax-check.

ALLOT n ---

This definition is used to reserve some space in the dictionary or to free memory. It adds the signed integer `n` to `DP` (Dictionary Pointer) user variable.

ALIGN ---

force `HERE` to an even address. This definition is available after `NEEDS ALIGN`.

ALIGNED a1 --- a2

force `a1` to an even address. This definition is available after `NEEDS ALIGNED`.

AND n1 n2 --- n3

It executes an bitwise AND operation between the two integers. The operation is performed bit by bit.

AUTOEXEC ---

This definition is executed the first time the Forth system boots and **loads Screen# 11**. Once called, it patches `ABORT` definition to prevent any further executions at startup. Anyway, you can still invoke it directly.
This allows you to perform some automatic action at startup.

B/BUF --- n

Constant. Number of bytes per buffer. In this implementation is 512.

B/SCR --- n

Constant that indicates the number of Blocks per Screen. In Next version is 2, that means a Screen is 1024 byte long. In Microdrive version it was 1…

BACK a ---

Computes and compiles a relative offset from `a` to `HERE`. Used by `AGAIN`, `UNTIL`, `LOOP`, `+LOOP`.

BASE --- a

User variable that indicates the current numbering base used in input/output conversions. It is changed by `DECIMAL` that put ten, HEX that put sixteen, and with some extensions `BINARY` that put two and `OCTAL` that put eight.

BASIC u ---

Quits Forth and returns to Basic returning to the caller USR the unsigned integer on TOS.

BETWEEN n1 n2 n3 --- f

Available after `NEEDS BETWEEN` returns a true flag if `n2` <= `n1` <=`n3` else a false flag.

BEGIN --- (immediate) (run time)
--- a n (compile time)

Used in colon definition in one of the following forms

```
BEGIN ...   AGAIN
BEGIN ... f UNTIL
BEGIN ... f WHILE ... REPEAT
BEGIN ... f END
```

At compile-time, it starts one of these structures.
At run-time `BEGIN` marks the beginning of a definitions sequence to be repeatedly executed and indicates the jump point for the corresponding `AGAIN`, `REPEAT`, `UNTIL` or `END`.
With `UNTIL`, the jump to the corresponding BEGIN happens if on TOS there is a `ff`, otherwise it quits the loop.
With `AGAIN` and `REPEAT`, the jump to the corresponding BEGIN always happens.
The `WHILE` part is executed if and only if on TOS there is a `tf`, otherwise it quits the loop.

BINARY **---**
Sets `BASE` to 2, that is the binary base. This definition is available after `NEEDS BINARY`.

BL **---** **c**
Constant for "Blank". This implementation uses ASCII and `BL` is 32.

BLANK **a** **n** **---**
Fills with "Blank" `n` location starting from address `a`.

BLK **---** **a**
User variable that indicates the current `BLOCK` to be interpreted. If zero then the input is taken from the terminal buffer `TIB`.

BLK-FH **---** **a**
Variable containing file-handle to block's file !Blocks-64.bin.

BLK-FNAME **---** **a**
Variable containing the counted-zstring "`!Block-64.bin`". as produced by `,"` definition.
See also `,"` definition.

BLK-INIT **---**
Initialize BLOCK system. It opens for update (read/write) file "`!Block-64.bin`" .

BLK-READ **a** **n** **---**
Read block n to address a. `See also F_READ.`

BLK-SEEK **n** **---**
Seek block n within **!Blocks-64.bin** file. `See also F_SEEK.`

BLK-WRITE **a** **n** **---**
Take text content at address a to disk block `n`. `See also F_WRITE.`

BLOCK n --- a

Leaves the address of the buffer that contains the block `n`. If the block isn't already there, it is fetched from disk. If in the buffer there was another buffer and it was modified, then it is re-written to disk before reading the block n.
See also `BUFFER`, `R/W`, `UPDATE`, `FLUSH`.

BOUNDS a n --- a+n a

Given an address and a length (a n) calculate the bound addresses suitable for `DO ... LOOP`.
It is used by `TYPE`.

BRANCH ---

Direct procedure that executes an unconditional jump. The memory cell following `BRANCH` has the offset to be relatively added to the Instruction Pointer to jump forward or backward. It is compiled by `AGAIN`, `ELSE`, `REPEAT`.

BUFFER n --- a

Makes the next buffer available assigning it the block number `n`. If the buffer was marked as modified (by `UPDATE`), such buffer is re-written to disk. The block is not read from disk. The address point to the first character of the buffer. The buffer holding BLOCK 1 is never reassigned: BLOCK 1 is the line buffer used by `INCLUDE`, `NEEDS` and `EVALUATE`, and its content cannot be re-read from disk.

BYE ---

Executes `FLUSH` and `EMPTY-BUFFERS`, then quits Forth and returns to Basic returning to the caller USR the value of `0 +ORIGIN`. See also `BASIC`.

C! b a ---

Stores a byte `b` to address `a`.

C, b ---

Puts a byte `b` in the next location available in the dictionary and increments `DP` (dictionary pointer) by 1.

C/L --- c

Constant that indicate the number of characters per screen line. In this implementation it is 32.

C@ a --- b

Puts on TOS the byte at address `a`.

CALL# n1 a --- n2

Performs a CALL to the routine at address `a`. First argument `n1` is passed via bc register *and* a register. The routine can return bc register which is pushed on TOS. This definition is useful to call normal ZX Spectrum ROM routines.
This definition is available after `NEEDS CALL#`.

CASEOFF **---**

Sets case-sensitive search OFF. It changes the system behavior so that `(FIND)` can search the dictionary ignoring case, and `COMPARE` compares two strings ignoring case. This is the start-up setting.

CASEON **---**

Sets case-sensitive search ON. It changes the system behavior so that `(FIND)` searches the dictionary case sensitive, and `COMPARE` compares two strings case sensitive, as the Standard wants.

CATCH **xt** **---** **0 | n**

In conjunction with **THROW**, this definitions put in place the error-handling facility. Used in colon-definition in the form

```
... ['] wwww CATCH …
```

this definition saves the current status of data-stack and return-stack, and performs **xt** via `EXECUTE`. If the execution of **xt** completes normally, i.e. with a `0 THROW` execution or with no `THROW` execution at all, it restores such saved status and leaves `0` on top of stack to signal no error occurred. Otherwise, `THROW` restores the most recent data-stack and return-stack status – saved by **CATCH** – so that the execution continues. See also **THROW**. This definition is available after `NEEDS CATCH`.

CELL **---** **2**

In this implementation a cell is two bytes. This definition is available after `NEEDS CELL`.

CELL+ **n1** **---** **n2**

Increments `n1` by 1 “cell”, that is two units. In this implementation a cell is two bytes.

CELL- **n1** **---** **n2**

Decrements `n1` by 1 “cell”, that is two units. In this implementation a cell is two bytes.

CELLS **n1** **---** **n2**

Doubles the number `n1` on TOS giving the number of bytes equialent to `n1` “cells”. In this implementation a cell is two bytes.

CFA **pfa** **---** **xt**

Converts a `pfa` in its `cfa`. See also `LFA`, `NFA`, `PFA`, `>BODY`, `<NAME`.

CHAR **---** **c**

Used in the form

```
CHAR c
```

determines the first character of the next definition in the input stream.

CLS **---**

Clears the screen using the ZX Spectrum ROM routine 0DAFh.

CMOVE a1 a2 n ---

It copies the content of memory from address a1 for n consecutive bytes, storing them to address a2 onward, proceeding one byte at a time. The content of address a1 is moved first. If n is zero, no byte is copied. See also CMOVE>.

CMOVE> a1 a2 n ---

Same as CMOVE but the copy process starts copying location a1+n-1 first, and location a1 last. This is important when the two memory areas overlap, and CMOVE would corrupt the result.

CODE ---

Defining definition used in the form

```
CODE cccc
```

it creates a new dictionary entry for the definition cccc with the cfa ~~of such a definition pointing to its pfa~~ that is empty for the moment, HERE points that location; then some machine-code instruction should be added using C, that will be compiled from HERE on-wards. The new definition is created in the CURRENT vocabulary but won't be found by (FIND) because it has the SMUDGE bit set. Once the definition construction is complete, it is programmer's responsibility to execute SMUDGE to make visible.
This definition is redefined / overridden by ASSEMBLER vocabulary available after LOAD-ing Screens 100-165, this allows the programmer to use a pseudo-standard Z80 notation to create a new low-level definition using assembler directly.

Here is an example that creates a definition SYNC-FRAME to wait for the next interrupt:

```
CODE SYNC-FRAME HEX
      76 C,        \ halt       ; wait for interrupt or reset
      DD C, E9 C, \ jp (ix)     ; jump to the inner interpreter
      SMUDGE
```

COMPARE a1 n1 a2 n2 --- b

Given two string descriptors, that is address and length, (a1, n1) and (a2, n2), this definition compares the two strings byte by byte and returns:

0	if they are equal
1	if String1 is greater than String2
-1	if String1 is less than String2

When one string is the beginning of the other, the shorter one is the lesser: "zoo" comes before "zoology". The compare is case-sensitive or case-insensitive based on the last execution of **CASEON** and **CASEOFF**; at start-up it is case-insensitive. Note that the Standard wants **COMPARE** case-sensitive: in vForth this is obtained with **CASEON.**

For example:

```
CREATE S1 ," Hello world!"
CREATE S2 ," Hello world?"
S1 COUNT S2 COUNT COMPARE .
```

will print -1 since the two strings differ only for the last character and the ASCII code of ! comes before the code of ? , so the string comparison S1 < S2 is true. Compare the result of the following two rows:

```
S2 COUNT S1 COUNT COMPARE .
```

```
S1 COUNT S1 COUNT COMPARE .
```

Up to build 2026-10-20 this definition was available only after NEEDS COMPARE; on such an older core it still is. See also CASEON and CASEOFF.

COLD **---**

This definition executes the Cold Start procedure that restore the system at its startup state.
It sets `DP` to the minimum standard and executes `ABORT`.

COMPILE **---**

Used in the form

```
COMPILE cccc
```

At compile-time, it determines the xt of the definition that follows `COMPILE` and compile it in the next dictionary cell.

COMPILE, xt ---

Used within a colon-definition to put `xt` in the following cell of the dictionary. The dictionary pointer (**DP**) is incremented by two locations.

CONSTANT n --- (immediate) (compile time)
--- n (run time)

Defining definition that creates a constant. Used in the form

```
n CONSTANT cccc
```

it creates the definition `cccc` and `pfa` holds the number n. When `cccc` is later executed it put n on TOS.

CONTEXT --- a

User variable that points to the vocabulary address where a definition search begins.

COUNT a1 --- a2 b

Leaves the address of text `a2` and a length `b`. It expects that the byte at address `a1` to be the length-counter and the text begins to the next location.

CR **---**

Transmits a 0x0D to the current output peripheral.

CREATE --- (compile time)
--- a (run time)

Defining definition used in the form

```
CREATE cccc
```

it creates a new dictionary entry for the definition **cccc** with the PFA still empty.
When `cccc` is executed, it puts on TOS the PFA of **cccc**
Often used with **ALLOT** to reserve space in the dictionary to be later used, for instance as an array.

The sequence **CREATE** … **DOES>** is used in a colon definition in the form

```
: cccc ...  CREATE ... DOES> ... ;
```

Subsequent execution of **cccc** in the form

```
cccc  nnnn
```

creates a new definition **nnnn** with an high-level procedure that at run-time calls the part of **cccc** after **DOES>** . When **nnnn** is later executed, the PFA of **nnnn** is put on TOS and the part after **DOES>** is executed.
<BUILD and **DOES>** allow writing high-level procedures instead of using machine code such as **;CODE** would require.

Obvious uses are new vocabularies (**ASSEMBLER**), multidimensional arrays, closures, and other compiling operations.
The "Floating Point Option Library" available via **NEEDS FLOATING** provides a good example of use of this structure
See also **VARIABLE** and **DOES>**.

CSP --- a

User variable that temporarily holds the value of SP register during a compilation syntax error check.

CURRENT --- a

User variable that points to the address in the Forth vocabulary where a search continues after a failing search executed in the CONTEXT vocabulary. See also LATEST.

CURS ---

Shows a (flashing) cursor on current video position and wait for a key-press.
Depending on CAPS-LOCK state, the faces of flashing cursor are different depending on the content of a few bytes in ORIGIN area:

```
HEX
026 +ORIGIN C@ . → 8F        Full square graphic character
027 +ORIGIN C@ . → 8C        Lower-half square graphic character
028 +ORIGIN C@ . → 5F        Underscore character
```

When CAPS-LOCK is On the cursor switches between ▉ (8F) and _ (5F)
When CAPS-LOCK is Off the cursor switches between ▉ (8F) and ▄ (8C)
You can modify this behavior putting some suitable values on these three bytes. For example you can make disappear the flashing cursor using the following patch:

```
HEX
BL 026 +ORIGIN C!
BL 027 +ORIGIN C!
BL 028 +ORIGIN C!
```

D+ d1 d2 --- d3

Computes d3 as the sum of d1 and d2. This is a 32 bits sum.

D+- d1 n --- d2

Forces the double-precision integer d1 to have the the sign of n.
It is used by DABS .

D- **d1 d2 --- d3**

Subtract d2 from d1. This is a 32 bits subtraction. Available after `NEEDS D-`.

D. **d --- **
n-lo n-hi ---

Emits a double-precision integer followed by a space. The double-precision integer is kept on stack in the format `n-lo n-hi` and the integer on TOS is the most significant.

D.R **d n ---**

Emits a double-precision integer rigth aligned in a field `n` character wide. No space follows. If the field is not large enough, then the excess protrudes to the right.

D0= **d --- f**

True if `d1` = 0. This is a 32 bits comaprison. Available after `NEEDS D0=`.

D< **d1 d2 --- f**

True if `d1` < `d2`. This is a 32 bits comaprison. Available after `NEEDS D<`.

D= **d1 d2 --- f**

True if `d1` equals `d2`. This is a 32 bits comaprison. Available after `NEEDS D=`.

DABS **d --- ud**

Leaves the absolute value of a double-precision integer.

DECIMAL **---**

Sets `BASE` to 10, that is the decimal base.

DEFINITIONS **---**

To be used in the form

cccc DEFINITIONS

it sets the `CURRENT` vocabulary to be the `CONTEXT` vocabulary and this allows adding new definitions to `cccc` vocabulary.
For example: `FORTH DEFINITIONS` or `ASSEMBLER DEFINITIONS`.
In this implementation, a Forth oriented `ASSEMBLER` vocabulary is available as an extra-option that can be imported via `NEEDS ASSEMBLER` or LOADed from Screens 100 -160 as a legacy.

DEVICE **--- a**

Variable that holds the number of current channel: 2 for video, 3 for printer, and any number between 4 and 15 to refer to a Basic OPEN# channel.

DIGIT **c n --- u tf (ok)**
c n --- ff (ko)

Converts the ASCII character `c` in the equivalent number using the base `n`, followed by a a `tf`. If the conversion fails it leaves a `ff` only.

DLITERAL **d --- d (immediate) (run time)**
d --- (compile time)

Same as `LITERAL` but a 32 bits number is compiled. `DLITERAL` is an immediate definition that is executed and not compiled.

DMAX **d1 d2 --- d3**

Leaves the maximum between `d1` and `d2`. Available after `NEEDS DMAX`.

DMIN **d1 d2 --- d3**

Leaves the minimum between `d1` and `d2`. Available after `NEEDS DMIN`.

DNEGATE **d1 --- d2**

Computes the opposite double-precision number.

DO **n1 n2 --- (immediate) (run time)**
--- a n (compile time)

Used in colon definition in the form

DO ... LOOP or
DO ... n +LOOP

It is used to put in place a loop structure: The execution of `DO` starts a sequence of definitions that will be repeated, under control of an initial-index `n2` and a limit `n1`. `DO` drops these two value from stack and the corresponding `LOOP` increments the index. If the index did not cross the boundary between the loop limit minus one and the loop limit, then the executions returns to the corresponding `DO`, otherwise the two parameters are discarded and the execution continues after the `LOOP`.
The limit `n1` and the initial value `n2` are determined during the execution and can be the result of other previous operations. Inside a loop the definition `I` copies to TOS the current value of the index.
See also: `I`, DO, `LOOP`, `+LOOP`, `LEAVE`. In particular LEAVE allows leaving the loop immediately.
At compile-time `DO` compiles `(DO)` and leaves the address of the following location and the number `n` to syntax-check.

DOES> **---**

Definition that defines the execution action of a high-level creative definition. **DOES>** changes the PFA of the definition being created to point the sequence compiled after **DOES>**. It is used in conjunction with **CREATE** (or **<BUILDS** but is deprecated). When the part after of **DOES>** is executed, it leaves on TOS the PFA of the new definition, and this allows the interpreter to use this area. Obvious uses are new vocabularies (**ASSEMBLER**), multidimensional arrays, closures, and other compiling operations.

The "Floating Point Option Library" available via **NEEDS FLOATING** provides a good example of use of this structure. See **CREATE** and **<BUILDS**.

DP **---** **a**

User variable (Dictionary Pointer) that holds the address of next available memory location in the dictionary. It is read by HERE and modified by ALLOT.

DPL **---** **a**

User variable that holds the number of digits after the decimal point during the interpretation of double-precision integer. It can be used to keep track of the column of the decimal point during a number format output. For 16 bit integer it defaults to –1. It takes into account the exponential part and its sign for floating point numbers.

DROP **n** **---**

Drops the value on TOS. See also OVER, NIP, TUCK, SWAP, DUP, ROT.

DUP **n** **---** **n n**

Duplicates the value on TOS. See also OVER, DROP, NIP, TUCK, SWAP, ROT.

DUP>R **n** **---** **n**

Copy TOS to the Return Stack. See also DUP, >R, R@.

DU< **ud1 ud2** **---** **f**

True if ud1 < ud2. This is a 32 bits comaprison. Available after NEEDS D<.

ELSE **a1 n1** **---** **a2 n2** **(immediate)** **(compile time)**
--- **(run time)**

Used in colon definition in the form

```
IF ... ELSE ... ENDIF
IF ... ELSE ... THEN
```

At run-time ELSE forces the execution of the false part of an IF-ELSE-ENDIF structure. It has no effects on the stack.
At compile-time ELSE compiles BRANCH and prepares the following cell for the relative offset, stores at a1 the previous offset from HERE; then it leaves a2 and n2 for syntax checking.

EMIT **c** **---**

Sends a printable ASCII character to the current output peripheal. OUT is incremented. 7 EMIT activates an acoustic signal. The 'null' 0x00 ASCII character is not transmitted.

EMITC **b** **---**

Sends a byte b character to the current output peripheal selected with SELECT. See also DEVICE.

EMPTY-BUFFERS **---**

Erases all buffers. Any data stored to buffers after the last FLUSH is lost.

ENCLOSE **a c --- a n1 n2 n3**

Starting from address `a`, and using a delimiter character `c`, it determines the offset `n1` of the first non-delimiter character, `n2` of the first delimiter after the text, `n3` of first character non enclosed.
This definition doesn't go beyond a 'null' ASCII that represent a unconditional delimiter. For example:

```
1:    c  c  x  x  x  c  x      →    2  5  6
2:    c  c  x  x  x  'null'   →    2  5  5
3:    c  c  'null'             →    2  3  2
```

END **a n --- (immediate) (compile time)**
f --- (run time)

Synonym of `UNTIL`.

ENDIF **a n --- (immediate) (compile time)**

At run-time, `ENDIF` indicates the destination of the forward jump from `IF` or `ELSE`. It marks the end of a conditional structure. `THEN` is a synonym of `ENDIF`.
At compile-time `ENDIF` calculates the forward jump offset from `a` to `HERE` and store it at `a`. The number `n` is used for syntax checking.

ENUMERATED **n --- (immediate) (compile time)**

Simple enumeration utility available after `NEEDS ENUMERATED`. Used in the form
Used in the form

n ENUMERATED name_0 name_1 ... name_n

For example:

8 ENUMERATED _black _blue _red _magenta _green _cyan _yellow _white

This definition has a limitation due to the way **LOAD** is implemented in that it cannot cope with **BLOCK**s boundaries or – for source-file – must be written in a single row that cannot be longer than 510 characters.

ERASE **a n ---**

Erases `n` memory location starting from `a`, filling them with 'null' characters (0x00).

ERROR **b --- n1 n2**
--- ff

Notifies an error **b** and resets the system to command prompt. First of all, the user variable **WARNING** is examined:
If **WARNING** is 0 then the offending definition is printed followed by a "?" character and a short message "MSG#n".
If **WARNING** is 1, instead of the short message, the text available on line b of **BLOCK** 4 is displayed. Such a number can be positive or negative and lay beyond **BLOCK** 4. Then, if **BLK** is non zero, then **ERROR** leaves on the stack **n1** that is the value of **>IN** and **n2** that is the value of **BLK** at the error moment. These numbers can then be used by **WHERE** to determine and show the exact error position. Otherwise, if **BLK** is zero, then only a `ff` is left on TOS. In both cases, the final action is **QUIT** .
If **WARNING** is –1 then **ABORT** is executed, which resets the system to command prompt. The user can (with care) modify this behavior patching **(ABORT)** definition by putting a specific **xt** at its **>BODY**.

EVALUATE **a u ---**

This definition is available after **NEEDS EVALUATE**. Save the current input source specification. Store minus-one (-1) in **SOURCE-ID**. Make the string described by `a` and `u` the input source, set **>IN** to zero, and invoke **INTERPRET**. When the

parse area is empty, restore the prior input source specification. Other stack effects are due to the words **EVALUATE**d. This definition assumes the string **a** has been created in **HEAP** via **S"**, and MMU7 is correctly in place, as usually is.

EXEC: **n** **---**

Vectorised fast case structure. Used in colon-definition in the form

```
: MY_ACTION_LIST ( n -- )
  EXEC:
      word0  \ executed when n = 0
      word1  \ executed when n = 1
      word2  \ executed when n = 2
    ...
;
```

to execute the definition indexed by **n** .
Warning: there is no run-time checking on **n** and if **n** is out of range, then a crash is likely to happen.
This definition is not available at startup, it must be loaded via **NEEDS EXEC:** . The source file is EXEC_.f

EXECUTE **cfa** **---**

Executes the definition which `cfa` is held on TOS.

EXIT **---**

This is (usually) the last definition compiled in a colon definition by **;** doing the action of returning to the calling definition. It is used to force the immediate end of a loading session started by `LOAD`.

EXP **---** **a**

User variable that holds the exponent in a floating-point conversion. It is not used until the *Floating Point Option* is enabled via `NEEDS FLOATING`.

EXPECT **a** **n** **---**

Transfers characters from the input terminal to the address `a` for `n` location or until receiving a 0x13 “CR” character. A 0x00 “null” character is added in the following location. The actual length of the received string is kept in `SPAN` user variable. See also `ACCEPT`. This definition isn't available at startup, it must be loaded using `NEEDS EXPECT`.

FENCE **---** **a**

User variable that holds the (minimum) address to where `FORGET` can act.

FILL **a** **n** **b** **---**

Fills `n` memory location starting from address `a` with the value of `b`.

FIRST **---** **a**

User variable that holds the address of the first buffer. See also `LIMIT`.

FLD **---** **a**

User variable that holds the width of output field.

FLIP **n1 --- n2**

Exchange high and lower byte of n1. Available after NEEDS FLIP.

FLUSH **---**

Executes SAVE-BUFFERS. It saves to disk the buffers marked "modified" by UPDATE.
Since BLOCK 1 is never reassigned, FLUSH does not write it back to disk even if it was marked by UPDATE, and it can safely be used inside a file being INCLUDEd. To rewrite BLOCK 1 on purpose use 1 BLOCK 1 0 R/W .

FM/MOD **d n1 --- n2 n3**

Floored Division. Leaves the quotient n3 and the reminder n2 of the integer division of d /n1.
This system has only UM/MOD coded in machine-code.

Dividend	Divisor	Remainder	Quotient
10	7	3	1
-10	7	4	-2
10	-7	-4	-2
-10	-7	-3	1

FORGET **---**

Used in the form

```
FORGET cccc
```

removes from the dictionary the definition cccc and all the preceding definitions. Care must be put when more than one VOCABULARY is involved. Use MARKER instead.
See also DP.

FORTH **--- (immediate)**

This is the name of the first vocabulary. Executing FORTH sets this to be the CONTEXT vocabulary. As soon as no new vocabulary is defined, all new colon definitions became part of FORTH vocabulary. FORTH is immediate, so it is executed during the creation of a colon definition to select the needed vocabulary. See also ASSEMBLER (optional vocabulary).

HANDLER **--- a**

User variable that holds the current error-handler. See CATCH and THROW.

HELP **--- cccc**

Display brief help of cccc definition. Available after NEEDS HELP.

HERE **--- a**

Leaves the address of next location available on the dictionary.

HEX **---**

Changes the base to hexadecimal, setting BASE to 16.

HLD **---** **a**

User variable that holds the address of last character used in a numeric conversion output.

HOLD **c** **---**

Used between <# and #> to put a ASCII character during a numeric format.

HP **---** **a**

User variable that holds the heap-address of the first free byte on Heap. See HEAP section.

HP, **n** **---**

Puts n in the next location available in the heap and increments HP (heap pointer) by 2.

I **---** **n**

Used between DO and LOOP (or DO and +LOOP, ?DO and LOOP, ?DO and +LOOP) to put on TOS the current value of the loop index. In this implementation this definition has the same effect as R@ i.e. it pick the first element on top of the Return-stack.

I' **---** **n**

Used between DO and LOOP (or DO and +LOOP, ?DO and LOOP, ?DO and +LOOP). It puts on TOS the *limit* of the loop. In this implementation I' picks the second element from the top of the Return-stack.

ID. **nfa** **---**

It prints the definition name whose nfa is on TOS.

IF **f** **---** **(immediate)** **(run time)**
--- **a** **n** **(compile time)**

Used in colon definition in the form

```
IF ...  ENDIF
IF ... ELSE ... ENDIF
```

At run-time IF selects which definitions sequence to execute based on the flag on TOS:
If f is true, the execution continues with the instruction that follows IF ("true" part).
If f is false, the execution continues after the ELSE ("false" part).
At the end of the two parts, the executions always continues after ENDIF.
ELSE and its "false" part are optional and if omitted no "false part" will be executed and execution continues after ENDIF.
At compile time IF compiles 0BRANCH reserving a cell for an offset to the point after the corresponding ELSE or ENDIF .
The integer n is used for syntax checking.

IMMEDIATE **---**

Marks the latest definition such that at compile-time it is always executed instead of being compiled. The bit 6 of the length byte of the definition is set. This allows such a definition to handle complex compilation situation instead of burdening the main compiler.
The user can force the compilation of an immediate definition prepending a [COMPILE] to it.

INCLUDE **---**

It is used in in the form:

```
INCLUDE cccc
```

starts interpretation of text read from file cccc.
See also NEEDS and LOAD

INDEX **n1 n2** **---**

Prints the first line of all screens between n1 and n2. Useful to quick check the content of a series of screens.

INKEY **---** **b**

Reads the next character available from current stream and previously selected with SELECT leving it on TOS. It is the opposite of EMITC.

INTERPRET **---**

This is the text interpreter. It executes or compiles, depending on the value of STATE, text from input buffer a word at a time. It first searches on CONTEXT and CURRENT vocabularies; if they fail, the text is interpreted as a numeric value, converted using the current BASE, and put on TOS. If that numeric conversion fails too, an error is notified with the symbol "?" followed by the word that caused the error. INTERPRET executes NUMBER and the presence of a decimal point "." indicates that the number is assumed as double-precision integer instead of a simple integer.
After execution of the found definition, the control is given back to the caller procedure.

INVERT **n1** **---** **n2**

Inverts all bits. This definition is available after NEEDS INVERT.

INVV **---**

"Inverse video". It enables Inverse-Video attribute mode. See also TRUV.
This definition isn't available at startup and must be included via NEEDS INVV.

J **---** **n**

Used inside a DO-LOOP gives the index of the *first* outer loop. See also I.
This definition isn't available at startup and must be included via NEEDS J.
E.g.

```
DO .. DO .. J .. LOOP .. LOOP
```

In this case **J** is used to get the index of the outer DO-LOOP while **I** gives the index of the inner DO-LOOP.

K **---** **n**

Used inside a DO-LOOP gives the index of the *second* outer loop. See also **I**.
This definition isn't available at startup and must be included via NEEDS K.
E.g.

```
DO .. DO .. DO .. K .. LOOP .. LOOP .. LOOP
```

Anyway, in Forth, it isn't a good programming technique nesting loop, better split the definition.

KEY **---** **b**

Waits for a key-press, without showing a flashing cursor. To show a cursor you have to use CURS just before KEY. It leaves

the ASCII code b of the character read from keyboard without printing it to video. In this implementation some **SYMBOL-SHIFT** key combinations are decoded as follow:

E2	STOP	→	7E	~
C3	NOT	→	7C	|
CD	STEP	→	5C	\
CC	TO	→	7B	{
CB	THEN	→	7D	}
C6	AND	→	5B	[
C5	OR	→	5D	]
AC	AT	→	7F	©
C7	<=	→	20	same as SHIFT-1 [EDIT]
C9	<>	→	06	same as CAPS-SHIFT + 2 and toggles CAPS-LOCK On and Off
C8	>=	→	20	same as SHIFT-0 [BACKSPACE]

L/SCR --- n

Constant that indicates the number of lines per Screen. In this implementation is 16.

LATEST --- nfa

Leaves the nfa of the latest definition in CURRENT vocabulary.

LEAVE ---

Forces the conclusion of a DO ... LOOP by compiling (LEAVE) followed by an offset to the first instruction after the corresponding LOOP or +LOOP.

LFA pfa --- lfa

Converts a pfa in its lfa. See also CFA, NFA, PFA, >BODY, <NAME.

LIMIT --- a

User variable that points to the first location above the last buffer. Normally it is the top of RAM, but not always. In this implementation, it set at $E000 to allow MMU7 as a general purpose 8K RAM bank. See also: FIRST.

LIST n ---

Prints screen number n and sets SCR to n.

LIT --- n

Puts on TOS the value hold in the following location. It is automatically compiled a before each literal number.

LITERAL n --- n (immediate) (run time)
n --- (compile time)

Compile-time, LITERAL compiles LIT followed by the value n in the following cell. This is an immediate definition and, even within a colon definition, it will be executed.
It is used in the form

```
: cccc ... [ calculations ] LITERAL ... ;
```

the compilation is suspended during the calculations and, when compilation resumes, LITERAL compiles the value put on TOS during the previous calculations.

LOAD **n** **---**

Start interpretation of Screen n. The loading phase ends at the end of the screen or at the first occurrence of EXIT.
~~If n is negative, instead of loading from Screen# n, it loads text directly from stream n as previously OPEN# from Basic.~~
See also --->

LOOP **a n ---** **(immediate)** **(run time)**
n --- **(compile time)**

Used in colon definition in the form

```
DO ... LOOP
?DO ... LOOP
```

At run-time LOOP checks the jump to the corresponding DO. The index is incremented and the total compared with the limit; the jump back happens if the index did not cross the boundary between the loop limit minus one and the loop limit. Otherwise the execution leaves the loop. On loop leaving, the parameters are discarded and the execution continues with the following definition. At compile-time LOOP compiles (LOOP) and the jump is calculated from HERE to a which is the address left by DO on the stack. The value n2 is used internally for syntax checking.

LP **---** **a**

User variable for printer purpose. In this Forth implementation it is used during compilation phase by CASE.

LSHIFT **n1 u** **---** **n2**

Shifts left an integer n1 by u bit.

M* **n1 n2** **---** **d**

Mixed operation. It leaves the product of n1 and n2 ad a double-precision integer.

M*/ **d1 n1 n2** **---** **d2**

Mixed operation. Compute (d · n1) / n2 using a "triple precision integer" as the intermediate value to avoid precision loss. This definition isn't available at startup and must be included via NEEDS M*/. The source file is M&%.f

M+ **d u** **---** **d2**

Mixed operation. It leaves the sum of d and unsigned u as a double-precision integer d2.
This definition is available after NEEDS M+

M/ **d n1** **---** **n2**

Mixed operation. It leaves the quotient n2 of the integer division of a double-precision integer d by the divisor n1.

M/MOD **d1 n1** **---** **n2 n3**

Mixed operation. It leaves the remainder n2 and the quotient n3 of the integer division of a double-precision integer d by the divisor n1. The sign of the reminder is the same as d. This system uses floored division via M/MOD and implements

UM/MOD in machine-code, FM/REM and SM/MOD as derived definitions.

MARK **a n** **---**

TYPE in inverse video. This definition is not available at startup, it has to be loaded via NEEDS MARK.

MARKER **---** **(immediate)** **(run time)**

Used outside a colon defintion in the form

MARKER cccc

this creates a new definition cccc that once executed restores the dictionary to the status before cccc was created. This removes cccc and all subsequent definitions. This definition allows forgetting across vocabularies since it keeps track of VOC-LINK, CURRENT, CONTEXT values.

MAX **n1 n2** **---** **n3**

Leaves the maximum between n1 and n2.

MESSAGE **n** **---**

Prints to the current device the error message identified by n. If WARNING is zero, a short MSG#n is printed. If WARNING is non zero 1, line n of screen 4 (of drive 0) is displayed. Such a number can be positive or negative and lay beyond block 4. See also ERROR.

MIN **n1 n2** **---** **n3**

Leaves the minimum between n1 and n2.

MMU7! **n** **---**

This definition accepts n between 0 and 223 and map the corresponding 8K-page at E000-FFFh addresses. It is coded in Assembler and uses NEXTREG A,n Next's peculiar op-code (ED 92). See MMU7@.

MMU7@ **---** **n**

This definition returns a number n between 0 and 223 by asking the hardware which 8K-page is currently fitted in MMU7. See MMU7!

MOD **n1 n2** **---** **n3**

Divides n1 by n2 and leaves the remainder n3. The sign is the same as n1.

MS **u** **---**

Waits at least u milliseconds.
At 28 MHz, u must be < 8192.
At 14 MHz, u must be < 16384.
At 7 MHz, u must be < 32768.
At 3.5 MHz, u must be < 65536.
This definition isn't available at startup and must be included via **NEEDS MS**.

N.B. Interrupts aren't disabled during execution.

M_GETDATE --- date time hs f

Returns the current date and time from the Real Time Clock, if present: date and time in MS-DOS format, and hs a packed cell of seconds (high byte) and 100ths of a second (low byte) -- decode with SPLIT. Flag f is 0 for OK (RTC present and valid); on error date and time are both 0. This is just a wrapper around NextZXOS API via RST 8 call followed by $8E service number. This definition is available after NEEDS M_GETDATE.

M_GETSETDRV n --- n2 f

Gets (`n`=0) or sets (`n`<>0, drive letter in bits 7..3, i.e. 0=A .. 15=P) the default drive used by NextZXOS/esxDOS API calls; does not affect NextBASIC's own LOAD/SAVE drives. Returns the resulting default drive `n2` and flag `f`, 0 for OK. This is just a wrapper around NextZXOS API via RST 8 call followed by $89 service number. This definition is available after NEEDS M_GETSETDRV.

M_SETCAPS a --- prev f

Sets NextZXOS capability flags `a` (bit 7 = skip zero-filling new space in `F_TRUNCATE`/`F_FTRUNCATE`); returns the previous capabilities `prev`, to be restored later with another `M_SETCAPS` call. Flag `f` is 0 for OK (available from NextZXOS v1.98M+). This is just a wrapper around NextZXOS API via RST 8 call followed by $91 service number. This definition is available after NEEDS M_SETCAPS.

M_P3DOS n1 n2 n3 n4 a --- n4 n5 n6 n7 f

This is the NEXTZXOS call wrapper. Parameters passed on stack are used as follow:

`n1` = input parameter value for **hl** registers pair
`n2` = input parameter value for **de** registers pair
`n3` = input parameter value for **bc** registers pair
`n4` = **a** register input parameter value
`a` = service routine address
`n5` = **hl** returned value
`n6` = **de** returned value
`n7` = **bc** returned value
`n8` = **a** register
`f` = 0 for OK, non zero for KO.

This definition calls uses RST 08 followed by $94 to call the specified routine.
Value returned on register IX is also stored at HEX 2A `+ORIGIN` before IX is restored to its fixed value.

Some NEXTZXOS primitives are coded by their own definition (e.g `F_OPEN`, `F_OPENDIR`, etc), but most of them aren't. For example all `LAYER` definitions use `IDE_MODE!` which in turn uses `M_P3DOS`.

NEEDS ---

Used in in the form:

```
NEEDS cccc
```

if the definition cccc is not present in dictionary, then it starts interpretation of text read from file **inc/*cccc*.f** and, if this is not found, gives a second chance from file **lib/*cccc*.f**

Some characters are illegal for filename: noticeably the “double-quote” character (") is among them. In such case, these characters are converted into “tilde” (~) and *that* file is then searched for.
For example:

NEEDS S" searches the file S~.f instead of an illegal filename S".f

Here is the complete map:

```
: ? / * | \ < > "
_ ^ % & $ _ { } ~
```

At the moment we are writing, this NEEDS definition has a flaw: in case of interpretation/compilation error, the file/handle remains open and you have to close it manually using something like 2 F_CLOSE DROP,

This definition differs from **INCLUDE** because **NEEDS cccc** refers to a dictionary entry whilst INCLUDE cccc refers to a full-path filename with explicit extension.

This definition is defined as follow: Since any given Screen # n occupies **BLOCK**s n and n+1, NEEDS exploits BLOCK number 1 – which normally isn't reachable – and uses it as a temporary buffer for each line read from file, for this reason a text source line cannot exceed 510 bytes, but for ZX Spectrum's standards is a lot more than anybody would need. Since build 2026-09-25 the buffer holding BLOCK 1 is never recycled, so a source file can read any number of other blocks while it is being included without losing its own current line.

NEGATE **n** **---** **-n**
Changes the sing of n1

NFA **pfa** **---** **nfa**
Converts a pfa to nfa. Reads the heap-pointer stored just before the CFA and resolves it with FAR, so MMU7 is left on the page holding the name. See also CFA, LFA, PFA, >BODY, <NAME.

NIP **n1 n2** **---** **n2**
Removes the second element from TOS. See also: OVER, DROP, TUCK, SWAP, DUP, ROT.

NMODE **---** **a**
User variable that indicates how double-precision numbers are interpreted. During the input, numbers can be read as double-precision integer numbers or floating-point numbers. This variable is modified by the optional definition INTEGER that sets it to 0 and FLOATING that sets it to 1.

NOOP **---**
This token does nothing. Useful as a placeholder or to prevent crashes in INTERPRET.

NOT **---**
Equivalent to 0=

NUMBER **a** **---** **d**
a **---** **fp** **(floating-point)**
Converts a counted string at address a with a in a double-precision number. If NMODE is 0, the string is converted to double-precision integer. Position of the last decimal point encountered is kept in DPL.
If **NMODE** is 1, a floating-point number conversion is tried instead of an simple double-precision integer conversion.
If no conversion can be done, and error #0 is raised. See § 2.9 for more details.

OCTAL **---**

Changes the base to octal, setting BASE to 8. To use this definition you have to type NEEDS OCTAL.

OFFSET **--- a**

Current edit position within current screen. Used by Line-Editor.

OPEN< **--- fh**

Used in the form

```
OPEN< cccc
```

this definition invokes **F_OPEN** NextZXOS and opens a file cccc. It returns file-handle number fh. This definition is used by INCLUDE. At the moment, this definition cannot be compiled and should be used only in interpretation phase.

OR **n1 n2 --- n3**

Executes a bitwise OR operation between the two integers. The operation is performed bit by bit.

OUT **--- a**

User variable incremented by EMIT. The user can examine and alter OUT to control the video formatting.

OVER **n1 n2 --- n1 n2 n1**

Copies the second number from TOS and put it on the top. See also DROP, NIP, TUCK, SWAP, DUP, ROT.

P! **b u ---**

Sends to port u a byte b. Note: u is a 16 bit port address and an OUT (C) op-code is internally executed.

P@ **u --- b**

Accepts the byte b from port u. Note: u is a 16 bit port address and an IN(C) op-code is internally executed.

PAD **---**

Leaves on TOS the address of text output buffer. It is at a fixed distance of 68 byte over HERE.

PFA **nfa --- pfa**

Converts a definition's nfa to its pfa. See also CFA, LFA, NFA, >BODY, <NAME.

PICK **n --- pfa**

Picks the **n-th** element from TOS. This means:

0 PICK is the same as DUP
1 PICK is the same as OVER

PLACE **--- a**

User variable that holds the number of places after the decimal point to be shown during a numeric output conversion.

See also PLACES.

POSTPONE **---**

Available after **NEEDS POSTOPNE** and used in colon defintion in the form:

```
: cccc ... POSTPONE wwww ... ;
```

It blends the functionality of [COMPILE] and COMPILE appending to the current definition the correct wwww *compilation behavior*. This is part of the modern standard, but the Author prefers the old-fashion [COMPILE] and COMPILE.

PREV **--- a**

User variable that points to the last referred buffer. UPDATE marks that buffer so that it is later written to disk.

QUERY **---**

Awaits from terminal up to 80 characters or until a CR is received. The text is stored in TIB. User variable IN is set to zero.

QUIT **---**

Clears the Return-Stack, stops any compilations and return the control to the operator terminal. No message is issued.

R@ **--- n**

Copies to TOS the value on top of Return Stack without alter it.

R# **--- a**

User variable that holds the position of the editing cursor or other function relative to files.

R/W **a n f ---**

Standard FIG-FORTH read-write facility. Address a specifies the buffer used as source or destination; n is the sequential number of the block; f is a flag, 0 to Write, 1 to Read. R/W determines the location on mass storage, performs the transfer and error checking.

R0 **--- a**

User variable that holds the initial value of the Return Stack Pointer. See also RP! and RP@.

R> **--- n**

Removes the top value from Return Stack and put it on TOS. See also >R, R@ and RP!.

R>DROP **---**

Removes the top value from Return Stack. See also >R, R@ and DROP. Available after NEEDS R>DROP.

RECURSE **---**

Used only at compile-time inside a colon-definition, it compiles the definition being created to put in place a recursion call. This definition is available after a `NEEDS RECURSE`.

REG! **b n ---**

Writes value `b` to Next REGister `n`.

REG@ **n --- b**

Reads Next REGister `n` giving byte `b`.

REMOUNT **---**

This definition is available only after `NEEDS REMOUNT`.
Enter the unmount/mount routine. Interactively the user is asked for a Y key-stroke, and the system waits for that key-stroke allowing the manipulation of the SD.

RENAME **---**

Used in the form:

RENAME cccc xxxx

it searches the definition `cccc` in the `CONTEXT` vocabulary and changes its name to `xxxx`. The two definition names `cccc` and `xxxx` **must have the same length**. This definition is available after `NEEDS RENAME`.

REPEAT a1 n1 a2 n2 --- (immediate) (compile time)
--- (run time)

Used in colon definition in the form:

BEGIN ... WHILE ... REPEAT

At run-time `REPEAT` does an unconditional jump to the corresponding BEGIN.
At compile-time `REPEAT` compiles `BRANCH` and the offset from `HERE` to `a1` and resolves the offset from `a1` to the location after the loop; `n1` and `n1` are used for syntax check.

ROT n1 n2 n3 --- n2 n3 n1

Rotates the three top elements, taking the third and putting it on top. The other two elements are pushed down from top by one place. See also `OVER`, `DROP`, `NIP`, `TUCK`, `SWAP`, `DUP`.

ROLL n1 ... k --- n2 ... n1

Rotates the `k` top elements, taking the `k`-th and putting it on top. The other k – 1 elements are pushed down from top by one place. The index `k` is zero based, so that `0 ROLL` does nothing, `1 ROLL` is `SWAP` and `2 ROLL` is `ROT`.
See also `ROT`. This definition isn't available at startup, it needs to be imported via `NEEDS ROLL` .

RP! **a ---**

System procedure to initialize the Return Stack Pointer to the value passed on TOS that should be the address held in `R0` user variable.

RP@ **---** **a**

Leaves the current value of Return Stack Pointer.

RSHIFT **n1 u** **---** **n2**

Shifts right an integer `n1` by `u` bit.

S0 **---** **a**

User variable that holds the initial value of che SP register. See also: `SP!` and `SP@`.

S>D **n** **---** **d**

Converts a 16 bit integer into a 32 bit double-precision integer, sign is preserved. An obsolete version `S->D` is still available via `NEEDS`.

SCR **---** **a**

User variable that hold the number of the last screen retrieved with `LIST`.

SELECT **n** **---**

Selects the current channel. As usual for ZX Spectrum, `n` is 0 and 1 for lower part of screen, 2 for the upper part, 3 for printer, 4 for "!Blocks.bin" stream. Note: `KEY` always select chanle 2 to display the (flashing) cursor.

SIGN **n** **---**

If `n` is negative, it puts an ASCII "-" at the beginning of the numeric string converted in the text buffer. Then, `n` is discarded while d is kept unchanged. Used between `<#` and `#>`.

SM/REM **d n1** **---** **n2 n3**

Symmetric Division. Leaves the quotient `n3` and the reminder `n2` of the integer division of `d` / `n1`.
This system has only UM/MOD coded in machine-code.

Dividend	Divisor	Remainder	Quotient
10	7	3	1
-10	7	-3	-1
10	-7	3	-1
-10	-7	-3	1

SMUDGE **---**

Used by the creation definition `:` during the definition of a new definition; it toggles the smudge-bit of the first byte in the nfa of the `LATEST` definition. When a definition's smudge-bit is set, it prevents the compiler to find it. This is typical for uncomplete or a not correctly ended definition.
It is also used to remove a malformed incomplete definition via

```
SMUDGE FORGET cccc
```

SOURCE-ID **---** **a**

User variable that keeps the file-handle used during `INCLUDE` or `NEEDS`.

SP! **a** **---**

System procedure to initialize the SP register to the address a that should be the address hold in `S0` user variable.

SP@ **---** **a**

Returns the content of SP register before `SP@` was executed.

SPACE **---**

Emits a space to the current output peripheal, usually the video. See also `SELECT`.

SPACES **n** **---**

Emits `n` spaces.

SPAN **---** **a**

User variable that holds the number of characters got from the last `EXPECT`.

SPLASH **---**

Shows splash screen build date-number.

SPLIT **n1** **---** **n2 n3**

Split the two bytes of n1 into two separate numbers, n2 low byte, n3 high byte. Available after `NEEDS SPLIT`.

STATE **---** **a**

User variable that holds the compilator status. A non-zero value indicates a compilation in progress.

SWAP **n1 n2** **---** **n2 n1**

Swaps the two top element at the TOS. See also `OVER`, `DROP`, `NIP`, `TUCK`, `DUP`, `ROT`.

THEN **a n** **---** **(immediate)**
--- **(compile time)**

Synonym of `ENDIF`.

THROW **... n** **---** **...** **| ... n**

In conjunction with **CATCH**, this definitions put in place the error-handling facility. If n is zero, just discard n. If n is non-zero, it restores the most recent data-stack and return-stack status – saved by the corresponding **CATCH** – so that the execution continues just after that point. See also **CATCH**. This definition is available after `NEEDS THROW`.

THRU **scr1 scr2 ---**
Load (interpret) Screens scr1 through scr2 inclusive. Do not mix with **-->** chaining: screens already chained-in would be loaded twice. Available after **NEEDS THRU**.

TIB **--- a**
User variable that holds the address of the Terminal Input Buffer.

TO **n ---**
Used in the form:

TO cccc

It assigns the value `n` to the variable `cccc` previously defined via `VALUE`. This definition available after `NEEDS TO`.

TOGGLE **a b ---**
The byte at location address `a` is XOR-ed with the model `b`.

TRAVERSE **a1 n --- a2**
Spans through the name-field of a definition depending on the value of `n`.
If `n` = 1, then `a1` must be the beginning of the name-field, i.e. nfa itself; `a2` is the address of the last byte of the name field.
If `n` = –1, then `a1` must be the last byte of name-field and `a2` will be the nfa.
Used by da `NFA` and `PFA`.

TRUV **---**
"True video". It disables Inverse-Video attribute mode. See also `INVV`.
This definition isn't available at startup and must be included via `NEEDS TRUV`.

TUCK **n1 n2 --- n2 n1 n2**
Takes the top element of calculator stack and copies after the second. See also `OVER`, `DROP`, `NIP`, `SWAP`, `DUP`, `ROT`.

TYPE **a n ---**
Sends to the current output peripheal `n` characters starting from address `a`.

U. **u ---**
Emits an unsigned integer followed by a space.

U< **u1 u2 --- f**
Leaves a `tf` if `u1` is less than `u2`, a `ff` otherwise.

UM* **u1 u2 --- ud**
Unsigned product of the two integers `u1` and `u2`. The result is a double-precision integer.

UM/MOD ud u1 --- u2 u3

Leaves the quotient `u3` and the reminder `u2` of the integer division of `ud` / `u1`. All values and arithmetic are unsigned. An ambiguous condition exists if `u1` is zero or if the quotient lies outside the range of a single-cell unsigned integer.

UNLOOP ---

This definition is specified by the "standard". It discard the loop-control parameters for the current nesting level. This definition isn't available at startup and must be included via `NEEDS UNLOOP`.

UNTIL a n --- (immediate) (compile time)
f --- (run time)

Used in colon definition in the forms

```
BEGIN ... UNTIL
```

At run-time `UNTIL` controls a conditional jump to the corresponding `BEGIN` when `f` is false; the exit from the loop happens if `f` is true.
At compile-time `UNTIL` compiles `0BRANCH` and an offset from `HERE` to a; n is used for syntax checking.

UPDATE ---

Marks as modified the latest used buffer, the one pointed by `PREV`. The block contained in the buffer will be transferred to disk when that buffer is requested for another block.

UPPER c1 --- c2

This definition converts given character `c1` to upper-case. If `c1` is not between "a" and "z", then `c1` is left unchanged.

USE ---

Used outside colon-definitions in the forms

```
USE filename
```

It tries to open `filename`, and if it succeeds, it closes the current `BLOCK`s file and uses the one just opened instead. To restore the standard/default file, you must use `BLK-INIT` which uses the filename given by `BLK-FNAME`.
You cannot `USE` the same file you're currently using, so one reason of failure is when you try to directly USE the default file !Blocks-64.bin, you need first to close it.
For example, the following sequence should work:

```
BLK-FH @ F_CLOSE USE !Blocks-64.bin
```

USED --- a

User variable that holds the buffer address of the block to be read from disk or that has just been written to.

USER n ---

Defining definition used in the form

```
n USER cccc
```

creates an user variable 'cccc'. The first byte of pfa of cccc is a fixed offset for the User Pointer, that is the pointer for the user area. In this implementation there is only one User Area and a fixed User Pointer.
When cccc is later executed, it put on TOS the sum of offset and User Pointer, sum to be used ad the address for that specific user variable. The user variable are: TIB, WIDTH, WARNING, FENCE, DP, VOC-LINK, FIRST, LIMIT, EXP, NMODE, BLK, >IN, OUT, SCR, OFFSET, CONTEXT, CURRENT, STATE, BASE, DPL, FLD, CSP, R#, HLD, USE, PREV, LP, PLACE, SOURCE-ID, SPAN, HANDLER, HP.

VALUE n ---

Defining definition used in the form:

```
n VALUE cccc
```

Creates the definition cccc that acts as a variable but which syntax behaves as a constant . To store a value in such a variable you have to use TO. When cccc is later executed it directly returns the value of the variable without the need to access its address using @. This definition is available after **NEEDS VALUE**.

VARIABLE ---

Defining definition used in the form:

```
VARIABLE cccc
```

creates the definition cccc with the pfa containing the initial value 0. When cccc is executed, it puts on TOS the pfa of cccc that is the address that holds the value.
When used in the form

```
cccc @
```

the content of the variable cccc is left on TOS.
When used in the form

```
n cccc !
```

the value on TOS is stored to the variable cccc.

VIDEO ---

Sets DEVICE 2 to select the video as current output peripheral. See SELECT and DEVICE.

VOC-LINK --- a

User variable that holds the address of a field in the definition of the last vocabulary. Each vocabulary is part of a linked-list that uses that field, in each vocabulary definition, as pointer-chain.

VOCABULARY ---

Defining definition used in the form

```
VOCABULARY cccc
```

creates the definition cccc that gives the name of a new vocabulary.
Later execution of

```
cccc
```

makes such vocabulary the CONTEXT vocabulary, so that it is possible to search for definitions in this vocabulary first and execute them.
Used in the form

cccc DEFINITIONS

makes such vocabulary the CURRENT vocabulary, so that it is possible to insert new definitions in it.

WARM **---**

Executes a warm system restart. It closes and reopens Block/Screen file then does ABORT.
It does not EMPTY-BUFFERS and you should be able to recover any transient work.

WARNING **---** **a**

User variable that determines the way an error message is reported. If zero, only a short "MSG#n" is reported. If non zero, a long message is reported. See also ERROR.

WHILE **f** **---** **(immediate)** **(run time)**
a n --- **a1 n1 a2 n2** **(compile time)**

Used in colon defintion in the form:

BEGIN ... WHILE ... REPEAT

At run-time WHILE does a conditional execution based on f. If f is true, the execution continues to a REPEAT which will jump to the corresponding BEGIN. If f is false, the execution continues after the REPEAT quitting the loop.
At compile-time WHILE compiles 0BRANCH leaving a2 for the offset; a2 will be comsumed by a REPEAT. The address a1 and the number n1 was left by a BEGIN.

WIDTH **---** **a**

User variable that indicates the maximum number of significant characters of a definition during compilation. It must be between 1 and 31.

WITHIN **n1 n2 n3** **---** **f**

Perform a comparison of a test value n1 against a lower-limit n2 and a upper-limit n3.
When n2 < n3, return a true-flag if n2 <= n1 < n3, a false-flag otherwise.
When n2 > n3, return a true-flag if n2 <= n1 OR n1 < n3, a false-flag otherwise.
This definition is available only after NEEDS WITHIN.

WORD **c** **---** **a**

Reads one or more characters from the current input stream up to a delimiter c and stores such string at HERE that is left on TOS. WORD leaves at HERE the length of the string as the first byte and ends everything with at least two spaces.
Further occurrences of c will be ignored.
If BLK is zero, the text is taken from the terminal input buffer TIB. Otherwise the text is taken from the disk block held in BLK. User variable >IN is added with the number of character read, the number ENCLOSE returns.

WORDS **---**

Shows a list of definitions of CONTEXT vocabulary. [Break] stops.

XOR **n1 n2 --- n3**

Executes a bitwise XOR operation between the two integers. The operation is performed bit by bit.

ZAP **---**

Used in the form

```
ZAP cccc
```

it produces a standalone executable of the definition **cccc**, writing in the current directory the following binary files:

cccc-core.bin
cccc-user.bin
cccc-heap.bin

and, if **HP** has gone beyond the first 16K bank of the heap, one more file for each further bank: **cccc-heap1.bin**, **cccc-heap2.bin**, **cccc-heap3.bin**.
These files contain the whole vForth system as it was just before **NEEDS ZAP**, so **NEEDS ZAP** must be the last thing loaded: a definition **cccc** defined after it gives message #67. In the image **COLD** runs **cccc** and then **BYE, ERROR** and **ABORT** end in **BYE** too, and **WARNING** is 0. The current session is left unchanged.
To load and start them, make a copy of the **BASIC** loader "Standard-Loader.bas" with **cccc** in its **DATA** line 7990, as described in 2.8.
A heap bank holding data beyond HP can be saved after **ZAP** with **n SAVE-BANK** , n being the BASIC bank number 16 to 19; the loader then needs a **LOAD ... BANK n** line for it.
This definition is available after NEEDS ZAP.

ZAP" **xt ---**

Used in the form

```
' cccc ZAP" filename.nex"
```

it produces a standalone executable of the definition **cccc** as a single **.nex** file, to be started from the NextZXOS Browser. The extension is not added automatically, and a filename longer than 47 characters gives error -18.
The image is prepared as **ZAP** does: it is the dictionary as it was just before **NEEDS ZAP"**, it runs **cccc** at **COLD** and ends in **BYE,** also on any error, and **WARNING** is 0 in it. The file holds banks 5, 2 and 0 and the heap banks **HP** has reached; any other bank can be added before **ZAP"** with `n NEX-BANK` , n being 0 to 111. Since there is no **BASIC** to go back to, **BYE** is meant to end in a soft reset of the machine.
This definition is available after **NEEDS ZAP"**.

[**--- (immediate)**

Used in colon defintion in the form:

```
: cccc [ ... ] ... ;
```

it suspends compilation. The definitions that follow [will be executed instead of being compiled. This allows to perform some calculations or start other compilers before resuming the original compilation with]. See also LITERAL.

['] --- **(immediate)** **(compile time)**

It is the same as the sequence `[ ' wwww ] LITERAL`.
It is used in colon-defintion in the form:

```
: cccc ... ['] wwww  ... ;
```

At compile time, `[']` compiles `LIT` and **xt** of `wwww` definition in the following cell.

[CHAR] --- **(immediate)** **(compile time)**

It is the same as the sequence `[ CHAR c ] LITERAL`.
It is used in colon defintion in the form:

```
: cccc ... [CHAR] c  ... ;
```

At compile time, `[CHAR]` compiles `LIT` and the numeric value of ASCII character `c` in the following cell.

[COMPILE] --- **(immediate)**

Used in colon defintion in the form:

```
: cccc ... [COMPILE] wwww ... ;
```

`[COMPILE]` forces the compilation of a definition `wwww` that is immediate. Normally an immediate definition isn't compiled but executed and to compile an immediate definition it isn't possible to use the sequence `COMPILE wwww` but it is necessary using the sequence `[COMPILE] wwww`.
For example, to create an alias `ENDIF` for `THEN` you can type:

```
: ENDIF [COMPILE] THEN ;
```

**** ---

Used in the from:

```
\ ...
```

Any character that follows `\` until the end of line are treated as a comment.

] ---

Resumes the compilation suspended by `[` so it is possible to continue the definition.

6.2 Case -Of structure

The following definitions aren't available at startup, it must be loaded via `NEEDS CASE`

`CASE` **`n0`** **`---`** **`(immediate)`** **`(run time)`**
`---` **`a n`** **`(compile time)`**

Used in colon definition in the form

```
n0 CASE
    n1  OF  ... ENDOF
       ...
    nz  OF  ... ENDOF
    ... ( else )
ENDCASE
```

The `CASE` definition marks the beginning of Case-Of structure i.e. a set of branches where only one is performed based on the value of `n0`. If none of the "OF clause" values matches, the `ELSE` part is performed, if any.
At compile time `CASE` leaves the previous `CSP` address `a` and a number `n` for syntax checking.
`CASE` has to be balanced by a corresponding `ENDCASE`.

`OF` **`n0 nk`** **`---`** **`(immediate)`** **`(run time)`**
`n1` **`---`** **`a n2`** **`(compile time)`**

This definition is used in colon-definition as parto of a Case-Of structure.
At run-time it compares the matching value `nk` with the matching value `n0` that was on TOS before the beginning of the Case-Of structure.
At compile-time, it compiles `(OF)` that does a `0BRANCH`. The numbers `n1` and `n2` are used for syntax checking and an address `a` is left and used by `ENDCASE` to resolve the branch.
See also `CASE`.

`ENDOF` **`---`** **`(immediate)`** **`(run time)`**
`a1 n1` **`---`** **`a n2`** **`(compile time)`**

This definition ends an "Of-EndOf" clause started with `OF`.
At compile-time it acts like a `THEN`, first compiling a BRANCH to be later resolved by `ENDCASE` to skip any subsequent "Of-End-Of" clauses and resolving here the `0BRANCH` compiled by the corresponding previous `OF` to continue the Case-Of structure.
See also `CASE`.

`ENDCASE` **`---`** **`(immediate)`** **`(run time)`**
`a a1 … az` **`---`** **`(compile time)`**

This definition ends a Case-Of structure started with `CASE`.
At compile-time it compiles a `DROP` to discard the value `n0` put on TOS before `CASE` and resolves all OF-ENDOF clauses to jump after the `ENDCASE`. Finally, it restores previous content of `CSP`.
See also `CASE`.

`(OF)` **`n0 nk`** **`---`** **`(run time)`**

This definition is the run-time semantic compiled by `OF` definition. At run-time, it compares the value now on TOS `nk` with the value `n0` that was on TOS just before the beginning of the Case-Of structure and leave a flag to be used by the following 0BRANCH (that was compiled by `OF`). When `n0` equals `nk`, the definitions between `OF` and `ENDOF` will be executed, otherwise a jump to the def after `ENDOF` is performed.

6.3 Named Local Variables (LOCALS)

The following definitions aren't available at startup, they must be loaded via **NEEDS LOCAL**.

{ ccc1 … ccn [-- ccn+1 …] --- (immediate)

Compile-only, must be the first word of the definition being compiled. Used in the form

```
: cccc ( … -- … ) { name1 … namen } … ;
```

The word **{** parses names up to a closing **}** all on the same source line as **{** , and declares them as local, **VALUE**-like words visible only inside this definition. At run time, when **cccc** executes, the names are bound to the caller's arguments, deepest first: name1 receives the deepest item the caller pushed.

A local behaves like a **VALUE** : writing its bare name pushes its current value; **TO name** stores a new one.

An optional **--** inside the braces marks off a second group: output locals. These are NOT bound from the caller's stack – each starts at 0 on every entry – and every exit path (the closing ; or an early EXIT) automatically pushes their current value, in declaration order, before returning to the caller. The form is:

```
: cccc ( n -- sum ) { N -- ACC } ... ;
```

The local names live in their own vocabulary and are invisible outside the definition that declared them, so two different definitions may freely reuse the same names. Locals are re-entrant: a word using them may **RECURSE**, and nested or recursive calls each see their own values. Up to 8 locals total (input and output combined) may be declared per **{}**.

Note that **{** must be balanced by a corresponding **}** .

} --- (immediate)

Closes the local-names declaration opened by **{**. See also **{**.

LOCALS-FOR n ---

Older, two-part form that **{** is built on; still supported. Used in interpretation state, on the line immediately BEFORE the colon definition, in the form:

```
n LOCALS-FOR cccc name1 ... namen
: cccc LOCALS … ;
```

n is the count of locals, **cccc** is the name of the definition that follows – everything on ONE line. There is no **--** in this form: every declared local is bound from the stack. See also **{**, **LOCALS**.

LOCALS --- (immediate)

Companion to **LOCALS-FOR**: the first word of the colon definition it names, making the names visible and binding them to the caller's arguments. Used in the form

```
: cccc ( … -- … ) LOCALS … ;
```

See also **LOCALS-FOR**.

6.4 Heap Memory Facility

Among ZX Spectrum Next new features is the huge amount of RAM. Strings are dictionary expensive, so it would be useful storing them in heap as constant-strings and fetch them at need. The question is how to leverage all that memory in Forth. More, 8K of room is a good place to store an array of strings, or even numeric array and implement some matrices algebra. The definitions that handle The Heap are available after loading via **NEEDS HEAP**, but all low-level definitions are available in the core dictionary.

6.4.1 Heap-Pointer encoding and decoding

The Heap Memory Facility introduces a new kind of Pointer, the Heap-Pointer that represents the offset inside a virtual memory area called "heap". A Heap-Pointer is not an usual address pointer and we need a way to encode both **page number** and **address offset** in a usual Z80 16-bits integer.

Two definitions are available to perform these coding and decoding operations: **>FAR** and **<FAR** called "to-far" and "from-far" respectively.

Given a page number **n** and an address **a** (to be intended as an offset of addresses between $E000 and $FFFF, but only the lower part bits are used) **<FAR** definition decodes the page number in the most significant bits of **ha** and the offset in the remaining least significant bits. The opposite function is performed by **>FAR** that splits a 16-bit Heap-Pointer into two numbers again, the page part **n** and the offset part **ha** between $0000 and $1FFF.

In the following paragraphs a couple of possible implementations are described in detail.

6.4.2 Heap structure

The Heap can be seen as a "double linked-list" starting at 8K page $20 offset $0002. The User variable `HP` keeps the "heap-pointer" to the next available location on Heap. Heap space is shared with name-space dictionary.

A Heap memory allocation reserves the requested number of bytes and advances **HP** to point to the next available location on Heap. The previous value of **HP** is also stored at the location that was available *before* the memory allocation was requested to put in place a "linked-list".

In steps:
1. **HP** is advanced by one cell (2 bytes) to make room for the forward linked-list pointer.
2. Current **HP** value returned by the memory allocation (memory is not initialized and its content is then undefined)
3. **HP** is advanced to the number of bytes requested (plus 2 to ensure room for two trailing 0x00 character).
4. **HP** is advanced of one cell (2 bytes) to make room for the backward linked-list pointer.

Here is a real case example:

Suppose **HP** contains **$0F80** and the Heap memory looks as follows (Location is expressed in the form "$page.$offset")

```
Page.Address  Content

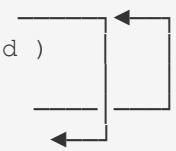
 20:0F80      .... free memory pointed by HP so that HEX HP ? gives F80
```

Then, we want to reserve a 7-bytes chunk of memory and we can type **7 HEAP** that returns $0F82 as “Heap-Pointer” to that new area of memory and **HP** will be advanced to $0F8D.

After the execution the memory will look like this:

```
Page.Address  Content

 20:0F80      0F8D   ( forward pointer )
 20:0F82      00 00 00 00 00 00 00     ( 7 bytes just allocated )
 20:0F89      00 00                    ( plus trailing 0x00 )
 20:0F8B      0F80   ( backward pointer )
 20:0F8D      ....  free heap memory pointed by HP
```



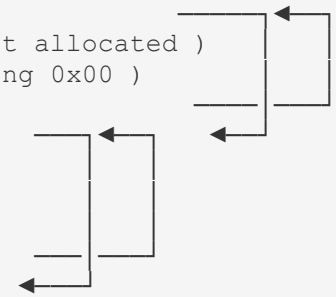
Then we can create a new string with Heap-Storage typing **S" 123"** which returns on TOS the actual real address $EF90 and the length $0003 that can directly used with **DUP TYPE** this way the Heap-Pointer is not lost, usually you want to keep it in a safe place such as a **CONSTANT,** or it should be saved beforehand via **HP@ CELL+**.

This allocation requires ten bytes on Heap: the forward pointer, the byte of length, the string itself 3+1 byte long, a two trailing 0x00, the backward pointer.

After the execution, the memory will look like this:

```
Page.Address  Content

 20:0F80      0F8D   ( forward pointer )
 20:0F82      00 00 00 00 00 00 00     ( 7 bytes just allocated )
 20:0F89      00 00                    ( plus trailing 0x00 )
 20:0F8B      0F80   ( backward pointer )
 20:0F8D      0F97   ( forward pointer )
 20:0F8F      03     ( string-length byte )
 20:0F90      31 32 33  ( the string “123” )
 20:0F93      00 00     ( plus trailing 0x00 )
 20:0F95      0F8D   ( backward pointer )
 20:0F97      ....  free memory pointed by HP
```

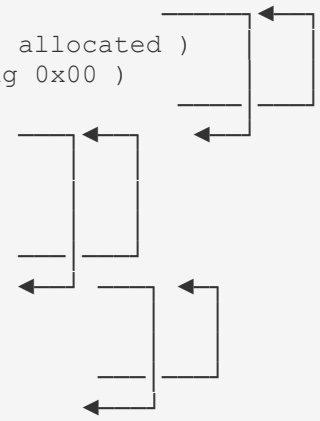


Then, we want to reserve another 5-bytes chunk and we can type **5 HEAP** that returns $0F99 as “Heap-Pointer” to that new area of memory and HP will be advanced to $0FA2.

After the execution the memory will look like this:

```
Location      Content

 20:0F80      0F8D    ( forward pointer )
 20:0F82      00 00 00 00 00 00 00     ( 7 bytes just allocated )
 20:0F89      00 00                    ( plus trailing 0x00 )
 20:0F8B      0F80    ( backward pointer )
 20:0F8D      0F97    ( forward pointer )
 20:0F8F      03      ( length byte )
 20:0F90      31 32 33  ( the string "123" )
 20:0F93      00 00     ( plus trailing 0x00 )
 20:0F95      0F8D    ( backward pointer )
 20:0F97      0FA2    ( forward pointer )
 20:0F99      00 00 00 00 00       ( 5 bytes )
 20:0F9E      00 00     ( plus trailing 0x00 )
 20:0FA0      0F97    ( backward pointer )
 20:0FA2      ....  free memory pointed by HP
```



Now, you should be able to see the Linked-List starting at $**0F80** that points to $**0F8D** that points to $**0F97** and then $**0FA2** is the current value of `HP`.

You can follow all these Forward-Pointers using the following procedure:

```
HEX
0F80 .S \  Stack is 0F80 as Heap-Pointer, that is 20:0F80, the beginning of Heap Memory.
FAR  .S \  Stack is EF80 as real Address (and page $20 is fitted in MMU7)
@    .S \  Stack is 0F8D as Heap-Pointer
FAR  .S \  Stack is EF8D as real Address (and page $20 is fitted in MMU7)
@    .S \  Stack is 0F97 as Heap Pointer
FAR  .S \  Stack is EF97 as real Address (and page $20 is fitted in MMU7)
@    .S \  Stack is 0FA2 as Heap Pointer
```

Likewise, the Backward-Pointers sequence would be:

```
HP@ .S \  Stack is 0FA2 as Heap-Pointer
CELL-  \  Stack is 0FA0 as Heap-Pointer
FAR .S \  Stack is EFA0 as real Address (and page $20 is fitted in MMU7)
@   .S \  Stack is 0F97 as Heap-Pointer
CELL-  \  Stack is 0F95 as Heap-Pointer
FAR .S \  Stack is EF95as real Address (and page $20 is fitted in MMU7)
@   .S \  Stack is 0F8D as Heap Pointer
CELL-  \  Stack is 0F8B as Heap-Pointer
FAR .S \  Stack is EF8B as real Address (and page $20 is fitted in MMU7)
@   .S \  Stack is 0F80 as Heap Pointer
```

Some low-level definitions are available to allow store and retrieve "to and from" Heap memory and how to avoid that a string isn't "paged away" in the middle of processing i.e. how to guarantee a page to stay in place across Standard-ROM calls or I/O disk operations that use page-bank C000-FFFF for their purposes:

`MMU7!` is used to fit a given 8K page number at E000h (i.e. MMU7).
`>FAR` is used to decode a "16 bit pointer" splitting it into "page & offset" as shown above.

The User Variable `HP` has been introduced to keep track of room in Heap: it's "the pointer" to the next available space on Heap.

Large part of the following definitions are available after loading via NEEDS HEAP

(H") **--- a n**

This is the run-time counterpart of **S"** that uses the Heap-Pointer in the following cell to fit the right 8K-Page in MMU7 using FAR definition and leave the real-address a and the length of the string n. See also **H" .**

+" **ha --- ha2**

Assuming ha is a Heap-Address Pointer to a "counted string" and this is the last chunk of memory of Heap, this definition accepts some text from the current input-source, parse it looking for a quote " that is the common "string terminator", and appends to the previous string on Heap. It returns the same Heap-Address Pointer to a "counted string" but the "count-byte" is incremented correctly. No page boundary check is performed.

+C **ha c --- ha**

Consume a character c from the current input source and append the string being created in Heap at ha. The heap pointer ha is returned unchanged.

>FAR **ha --- a p**

Given a Heap-encoded Pointer **ha** this definition decodes the top bits as one of the 8K-page available page **p** and the lower bits as the offset from $E000 **a** . It does not modify what MMU7 page is/has.
Since version 1.7, this definition is always available. Previous versions needs NEEDS >FAR.
See <FAR, MMU7 !

<FAR **a p --- ha**

Given an offset-address **a** (to be intended as a physical address between $E000 and $FFFF) and a page number **p** for an 8K-page this definition encodes the page number in the most significant bits of **ha** and an offset in the remaining bits. It does not modify MMU7 page.
Since version 1.7, this definition is always available. Previous versions needs NEEDS <FAR.
See >FAR, MMU7 !

FAR **ha --- a**

This definition converts a heap-pointer ha into an offset a (at $E000) and perform the correct 8K paging on MMU7. It simply calls >FAR and MMU7 !

H" **--- ha**

Accept text from the current input-source and store it to Heap. It returns a "heap-address-pointer" where a counted string is stored in a non-volatile way. The process is this: **H"** parses the string, stores it as a counted string in the heap, returns a heap-pointer (ha) that stays valid indefinitely and you decode with FAR right before use. That real address is volatile: it becomes invalid as soon as another FAR/HEAP call remaps the MMU7 page.

HEAP **n --- ha**

This definition reserves n bytes on Heap and returns the "heap-address-pointer" to retrieve them later via FAR.
This **ha** can be turned into a constant name using POINTER.

POINTER ha --- a

It works like `CONSTANT` but it returns a `"FAR`-resolved" offset-pointer that starts from/at **$E000**.
A possible use is : `H" ccc" POINTER P1`

S" --- a n

Accept text from the current input-source and store it to Heap as a “Counted-ZString”.
At compile time it compiles **`(H")`** followed by an Heap-pointer just after it, which at run-time is converted into a real-address (MMU7 being fitted correctly among the) and a counter representing the “counted-string” that can be used
If **`STATE`** is 0, i.e. we aren’t compiling, the “counted-string” address is resolved immediately, so **`S"`** behaves like **`H"`** but returns a real, already MMU7-decoded address at interpret time. Such an address is volatile, it becomes invalid as soon as another FAR/HEAP call remaps the MMU7 page. Rule of thumb: use S" in definitions, use H" when interpreting only.

SKIP-HP-PAGE n ---

Check if `n` bytes are available at the top of Heap on current 8K-page, otherwise advance `HP` to skip to the beginning of next 8K-page. It raises an “Heap Full” error if there is no more room in Heap.

6.4.3 Heap Pointer description for 64 kiBytes space

The following solution allows 64K of physical RAM Heap: Since an 8K offset requires 13 bits, the remaining 3 bits can be used to encode, say, from page 32 ($20) to page 39 ($27). For instance:

```
                    16 bit pointer
                 | pppb bbbb | bbbb bbbb |
                 |           |           |
  |    Page     |         Offset         |
  |  0010 0ppp  | 111b bbbb | bbbb bbbb |
```

The encoding/decoding definitions would be something like the following:

```
\ >far routine
\ input:  hl : heap-pointer
\ output:  a : page
\       : hl : address starting from $E000
      ld      a,h
      ex      af,af'  ; save high part
      ld      a,h
      or      $E0
      ld      h,a     ; hl = offset at $E000
      ex      af,af'
      rlca
      rlca
      rlca
      and     $07     ; so there are eight pages
      add     $20     ; this is peculiar to this example
      ret
```

```
\ <far routine
\ input :  a : page number between 32 and 39
\       : hl : address starting from $E000
\ output: hl : heap-pointer
      and     $07     ; ? questionable: it could be SUB $20
      rrca
      rrca
      rrca
      ex      af,af'  ; keep bits 765 in alternate A
      ld      a,h
      and     $1F
      ld      h,a
      ex      af,af'  ; retrieve bits 765
      or      h
      ld      h,a
      ret
```

6.5 Testing Suite

This is an adaptation of the ANS test harness based on the work originally developed by John Hayes, see https://forth-standard.org/standard/testsuite for details.

The suite is loaded using `NEEDS TESTING` and the "Core test-set" can be execute by typing

```
INCLUDE  ./test/core-tests.f
```

or

```
INCLUDE  ./test/locals-tests.f
```

to perform the LOCALS dedicated tests.

Here is the complete output of current version of core-tests collected to a file: the first part is just the echo of any definition being loaded into dictionary.

```
:NONAME 2ROT ALIGN ALIGNED CASE CHAR+ CHARS DEPTH FIND FLIP INVERT J M*/ POSTPONE
RECURSE S" H" HEAP UNLOOP ['] EVALUATE SOURCE BINARY DEFER DEFER! DEFER@ IS FALSE
TRUE VALUE TO (TO) +TO .PAD TESTING Suite .S <> COMPARE ERROR msg#4
 TESTING Test Suite
 TESTING \ F.3.1 Basic Assumptions
 TESTING Test Suite - Basic Assumptions
 TESTING F.6.1.0080 - (
 TESTING \ F.3.2 Booleans
 TESTING F.6.1.1720 - INVERT
 TESTING F.6.1.0720 - AND
 TESTING F.6.1.1980 - OR
 TESTING F.6.1.2490 - XOR
 TESTING F.6.2.2298 - TRUE
 TESTING F.6.2.1485 - FALSE
 TESTING \ F.3.3 Shifts
 TESTING F.6.1.0320 - 2*
 TESTING F.6.1.0330 - 2/
 TESTING F.6.1.1805 - LSHIFT
 TESTING F.6.1.2162 - RSHIFT
 TESTING \ F.3.4 Numeric Notation
 TESTING Numeric Notation
 TESTING \ F.3.5 Comparison
 TESTING F.6.1.0270 - 0=
 TESTING F.6.1.0530 -  =
 TESTING F.6.1.0250 - 0<
 TESTING F.6.1.0480 -  <
 TESTING F.6.1.0540 -  >
 TESTING F.6.1.2340 - U<
 TESTING F.6.1.1880 - MIN
 TESTING F.6.1.1880 - MAX
 TESTING F.6.1.0270 - NOT  0= equivalent
 TESTING F.6.1.0270 - <>  0= 0= equivalent
 TESTING \ F.3.6 Stack Operators
 TESTING F.6.1.1260 - DROP
 TESTING F.6.1.1290 - DUP
 TESTING F.6.1.1990 - OVER
 TESTING F.6.1.2160 - ROT
 TESTING F.6.1.2260 - SWAP
 TESTING F.6.1.0370 - 2DROP
 TESTING F.6.1.0380 - 2DUP
 TESTING F.6.1.0400 - 2OVER
 TESTING F.6.1.0430 - 2SWAP
 TESTING F.6.1.0630 - ?DUP
 TESTING F.6.1.0630 - -DUP   ?DUP equivalent
```

```
TESTING F.6.1.1200 - DEPTH
TESTING            - NIP
TESTING            - TUCK
TESTING            - PICK
TESTING F.6.1.2160 - ROT equivalent
TESTING            - 2ROT
TESTING \ F.3.7 Return Stack Operators
TESTING F.6.1.0580 - >R - R@ R>
TESTING \ F.3.8 Addition and Subtraction
TESTING F.6.1.0120 - +
TESTING F.6.1.0160 - -
TESTING F.6.1.0290 - 1+
TESTING F.6.1.0300 - 1-
TESTING F.6.1.0690 - ABS
TESTING F.6.1.1910 - NEGATE
TESTING \ F.3.9 Multiplication
TESTING F.6.1.2170 - S>D
TESTING F.6.1.0090 - *
TESTING F.6.1.1810 - M*
TESTING F.6.1.2360 - UM*
TESTING \ F.3.10 Division
TESTING F.6.1.2370 - UM/MOD
TESTING F.6.1.1561 - FM/MOD
TESTING F.6.1.2214 - SM/REM
TESTING F.6.1.0110 - */MOD
TESTING F.6.1.0240 - /MOD
TESTING F.6.1.0230 - /
TESTING F.6.1.1890 - MOD
TESTING F.6.1.0100 - */
TESTING 8.6.1.1820 - M*/
TESTING \ F.3.11 Memory
TESTING F.6.1.0150 - ,
TESTING F.6.1.0130 - +!
TESTING F.6.1.0890 - CELLS
TESTING F.6.1.0860 - C,
TESTING F.6.1.0898 - CHARS
TESTING F.6.1.0705 - ALIGN
TESTING F.6.1.0710 - ALLOT
TESTING \ F.3.12 Characters
TESTING F.6.1.0770 - BL
TESTING F.6.1.0895 - CHAR
TESTING F.6.1.2520 - [CHAR]
TESTING F.6.1.2500 - [
TESTING F.6.1.2165 - S"
TESTING \ F.3.13 Dictionary
TESTING F.6.1.0070 - '
TESTING F.6.1.2510 - [']
TESTING F.6.1.1550 - FIND
TESTING Custom - -FIND
TESTING F.6.1.1780 - LITERAL
TESTING F.6.1.0980 - COUNT
TESTING F.6.1.2033 - POSTPONE
TESTING F.6.1.2250 - STATE
TESTING F.6.2.2530 - [COMPILE]
TESTING F.6.2.0945 - COMPILE,
TESTING \ F.3.14 Flow Control
TESTING F.6.1.1700 - IF
TESTING F.6.1.2430 - WHILE
TESTING F.6.1.2390 - UNTIL
TESTING F.6.2.0455 - :NONAME
TESTING F.6.1.2120 - RECURSE
```

```
 TESTING F.6.2.0873 - CASE
 TESTING \ F.3.15 Counted Loops
 TESTING F.6.1.1800 - LOOP
 TESTING F.6.1.0140 - +LOOP
 TESTING F.6.1.1730 - J
 TESTING F.6.1.1760 - LEAVE
 TESTING F.6.1.2380 - UNLOOP
 TESTING F.6.2.0620 - ?DO
 TESTING Custom - I'
 TESTING \ F.3.16 Defining Words
 TESTING F.6.1.0450 - :
GDX has already been defined.
 TESTING It's correct seeing this message:
 TESTING GDX has already been defined.
 TESTING It's correct seeing this message.
 TESTING F.6.1.0950 - CONSTANT
 TESTING F.6.1.2410 - VARIABLE
 TESTING F.6.1.0550 - >BODY
 TESTING F.6.1.1250 - DOES>
 TESTING F.6.2.2405 - VALUE
 TESTING F.6.2.1725 - IS
 TESTING F.6.2.1173 - DEFER
 TESTING F.6.2.1175 - DEFER!
 TESTING F.6.2.1177 - DEFER@
 TESTING \ F.3.17 Evaluate
 TESTING F.6.1.1360 - EVALUATE
 TESTING \ F.3.18 Parser Input Source Control
 TESTING F.6.1.2216 - SOURCE
 TESTING F.6.1.0560 - >IN
 TESTING F.6.1.2450 - WORD
 TESTING \ F.3.19 Number Patterns
 TESTING F.6.1.1670 - HOLD
 TESTING F.6.1.2210 - SIGN
 TESTING F.6.1.0030 - #
 TESTING F.6.1.0050 - #S
 TESTING F.6.1.0750 - BASE
 TESTING F.6.1.1675 - HOLDS
 TESTING F.6.1.0570 - >NUMBER
0
FFFF
FFFFFFFF
0
1EKF
1Z141Z3
 TESTING \ F.3.20 Memory movement
 TESTING F.6.1.1540 - FILL
MOVE  TESTING F.6.1.1900 - MOVE
 TESTING \ F.3.21 Output
 TESTING F.6.1.1320  -  EMIT
YOU SHOULD SEE THE STANDARD GRAPHIC CHARACTERS:
 !"#$%&'()*+,-./0123456789:;<=>?@
ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`
abcdefghijklmnopqrstuvwxyz{|}~
YOU SHOULD SEE 0-9 SEPARATED BY A SPACE:
0 1 2 3 4 5 6 7 8 9
YOU SHOULD SEE 0-9 (WITH NO SPACES):
0123456789
YOU SHOULD SEE A-G SEPARATED BY A SPACE:
A B C D E F G
YOU SHOULD SEE 0-5 SEPARATED BY TWO SPACES:
0  1  2  3  4  5
```

```
YOU SHOULD SEE TWO SEPARATE LINES:
LINE1
LINE2
YOU SHOULD SEE THE NUMBER RANGES OF SIGNED AND UNSIGNED NUMBERS:
SIGNED: -8000 7FFF
UNSIGNED: 0 FFFF
 TESTING \ F.3.22 Input
 TESTING F.6.1.0695    -   ACCEPT
PLEASE TYPE UP TO 80 CHARACTERS:
Hello world!
RECEIVED: "Hello world!"
 TESTING \ F.3.23 Dictionary Search Rules
GDX has already been defined.  GDX has already been defined.
ok.
```

TESTING ---

This definition is much like a comment, it displays the whole source line where it is.

T{ ---

Begin a test phrase that ends with `}T`. It records pre-test stack depth to be compared later.

-> ---

Record depth and contents of stack to be copared after `}T`.

}T ---

End a test phrase begun with `T{`. It compares two stack images. Any discrepancies is shown by repeating the current test SOURCE line involved followed by one of the error
In general, a test is given in the form

```
T{  ...  ->  ...  }T
```

for example:

```
T{ 1 DUP -> 1 1 }T
```

6.6 Other Utilities

SHOW-PROGRESS n ---

Useful within long-lasting definitions to display a “rolling-bar” that show that your ZX Spectrum hasn’t hanged or crashed. This definition isn’t available at startup and must be included via `NEEDS SHOW-PROGRESS`.

?VOCAB ---

Useful to see which `VOCABULARY` is `CURRENT`, `CONTEXT` and the linked-list described by `VOC-LINK`.

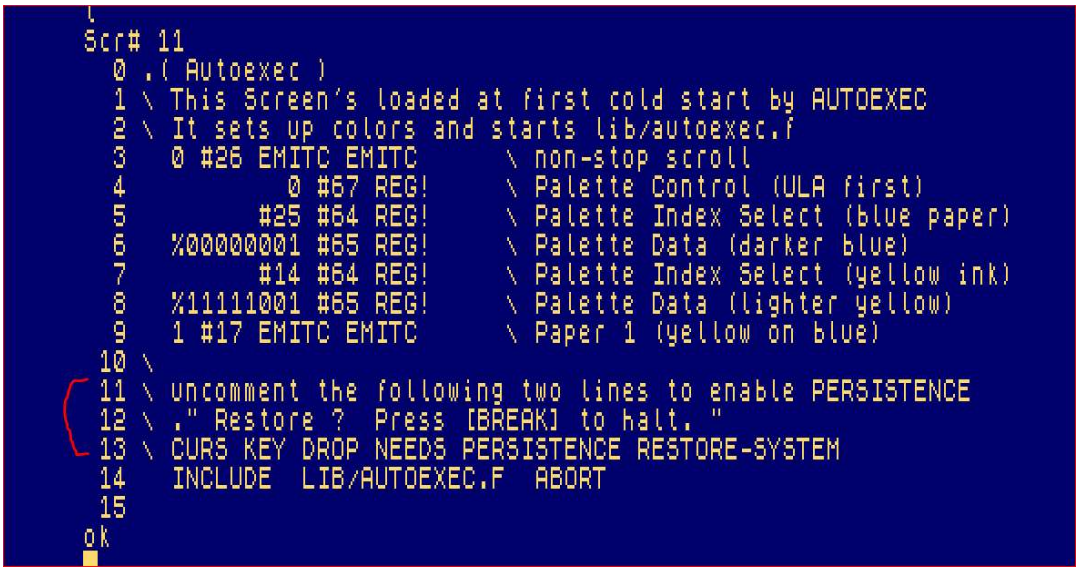
VIEW ---

Used in the form `VIEW cccc` used to display a file. Pressing [EDIT] during ouput allows you to temporarily pause the stream of output. Pressing [BREAK] will stop the output and return to prompt. Available after `NEEDS VIEW`.

6.7 Persistence across sessions

This paragraph introduces an *experimental utility* that allows the current system status to be saved to BLOCKs and that can be restored later.

To enable this facility you have to type **NEEDS PERSISTENCE** as the very first definition after a **COLD** start *and* you have to *un-comment* two lines in Screen #11 to allow **AUTOEXEC** to restore the system at next clean startup.



This way, at first **COLD** start, the system will ask to continue the "persistence course", but you can use [BREAK] to stop it. This is a safety measure, because if the binary to be restored is bugged, there should be little way to have a clean startup ever again but to tamper with the **!Block-64.bin** file directly.

Later you have to type **SAVE-SYSTEM** to save a snapshot of the system, so that at next startup will restore the system with the image it was left just before the last **SAVE-SYSTEM** or the last **BYE** was given: **BYE** is patched to perform **SAVE-SYSTEM** before exit.

To disable, you have to type **NO-PERSISTENCE** and comment those two lines.

The system status is stored in BLOCKs as follows (remember Screen# number is half a BLOCK number):

Normal version:
User data: 32000
Core data: 32001 - 32040
Heap data: 32048 - 32175

Dot-version:
User data: 32200
Core data: 32201 - 32240
Heap data: 32248 - 32375

This way, each time from BASIC you invoke `.vforth` dot-command you will restart from where you left.

To disable the feature, you have to type **NO-PERSISTENCE**. That stores a 0x0000 to the beginning of BLOCK #32000 (or #32200)

To clear all these blocks you can use **CLEAR-BLOCKS** imported via **NEEDS PERSISTENCE**.

7 The Memory Map

The memory is divided differently depending on which way vForth is started.

1. In dot-command, the core is loaded in DivMMC RAM at address $2000-$3FFF, and the dictionary continues at $8000. To preserve previous BASIC memory state, MMU4, MMU5 and MMU6 are fitted with 8k pages #40, #41 and #42 ($28-$2A); heap pages from #32 to #39 ($20-$27) are fitted in MMU7 ($E000-$FFFF) when needed. I've found experimentally that, since MMU2 and MMU3 are extensively swapped using Layer 1,2, the original content of MMU3 must be kept saved in page #43 ($2B) as backup and restored before return to BASIC. This means **twelve 8K-Pages are requested** (via NextZXOS call), from **#32** to **#43 ($20-$2B)**.

2. In standard program, the RAMTOP is lowered, the core is loaded in memory at $6366 and the dictionary continues upward in a linear way. Standard BANKS 5-2-0 configuration is used, i.e. pages $0A, $0B, $04, $05, $00, $01, with the exception that MMU7 is also used to access any 8K RAM Page other than $01. **Eight 8K-Pages are requested** via NextZXOS call, from **#32** to **#39 ($20-$27)** and fitted in MMU7 ($E000-$FFFF) when needed. Also, Layer2 BANKS are usually accessed via MMU7.

In both ways, the Name-Space part of dictionary is loaded in **BANK 16**, or to be more specific, **8K-Pages #32 ($20)** and **#33 ($21)**, then the next six Pages from **#34 to #39 ($22-$27)** are reserved as **`HEAP`** memory (§6.4) and used to keep the Name-Space along with any other **`HEAP`** data, such as long strings: this way both `FORGET` and `MARKER` free memory from HEAP and Dictionary in the same call. Also, vForth should refuse to start if these page aren't available.

```
Address     Name        Description
---------   ---------   ------------------------------
0000-3FFF               ROM of Spectrum
2000-3FFF               Dot-command memory area.
4000-57FF               Display file
5800-5AFF               Attribute file.
5B00-5BFF               System variables 128K RAM (former Printer buffer)
5C00-5CEF               System variables
            *CHANS      Stream map
            *PROG       Basic program
            *VARS       Basic variables
            *E_LINE     Line in editing
            *WORKSP     Workspace
            *STKBOT     Floating point Stack Bottom
            *STKEND     Floating point end
            *SP         Z80 Stack Pointer register in Basic
61FF        *RAMTOP     Logical RAM top (RAMTOP var is 23730)
6200-6300               IM2 ISR vector table (non dot-command version)
6301-6330               Return Stack during ISR (20 entries)
6331-6362               Stack area during OS operations
6363                    ISR entry point (JP address)
6366          ORIGIN    Forth Origin (non dot-command version)
                        FENCE @
              LATEST    CURRENT @ @
              HERE      DP @
              PAD       HERE 68 +  (+44h), length is limited by SP@
              ...       Dictionary grows upward
              ...       Dictionary free memory
              SP@       Calculator Stack grows downward
D0F4                    S0 @
D0F4          TIB @     <160 bytes it shares memory with Return-Stack.
              RP@       Return Stack grows downward it can hold 80 entries
D194                    R0 @
D194-D1E4               User variables area (40 entries, 80 bytes)
D1E4          FIRST     First buffer: There are 7 buffers (516*7 = 3612 bytes)
E000          LIMIT     First byte outside Forth.
E000-FFFF     MMU7      8K Page that can page any of the 224 banks of RAM
FFFF          P_RAMT    Physical ram-top
```

Contents

1 Foreword....2
1.1 MIT License....3
1.2 Typos, bugs and suggestions....3
1.3 Acknowledgment....3
1.4 Document structure....4
1.5 Legend....5
2 Getting started....6
2.1 Installation....6
2.1.1 dot-command version....7
2.2 Activation / Deactivation and tutorial....8
2.2.1 Online Reference – Interactive help....10
2.2.2 Tutorial....10
2.2.3 Editing with Visual Studio Code....12
2.3 Case-insensitive and Case-sensitive option....13
2.4 Block / Screen system....13
2.4.1 Block File Format....14
2.5 Upgrading keeping your own code and data....14
2.6 Character visualization size....15
2.7 Source feeding and output spooling....15
2.8 Creation of standalone executable....15
2.9 Numeric literals interpretation....16
2.10 Back-port to Microdrive and MGT and 128K....16
2.10.1 ZX Microdrive version using Fuse Emulator....17
2.10.2 DISCiPLE version using Fuse Emulator....17
2.11 128K memory and later models....17
2.12 Definitions grouped by category....20
2.12.1 Comments....20
2.12.2 Stack manipulation....20
2.12.3 Comparison....20
2.12.4 Output....21

2.12.5 Integer Arithmetics....21
2.12.6 Memory....22
2.12.7 Flow control....22
2.12.8 I/O and Hardware....22
2.12.9 Definition related....23
2.12.10 BLOCK / Screen related....23
2.12.11 Numbers & strings....23
2.12.12 Dictionary related....24
2.12.13 Editor....24
2.12.14 NextZXOS....24
2.13 Known bugs, workarounds, and improvement needed....25
3 Utilities....26
3.1 The Editors....26
3.1.1 The Full Screen Editor Utility – Screen oriented....26
EDIT --- 26
SCREEN-FROM-FILE n --- 27
SCREEN-TO-FILE n --- 28
3.1.2 LED – the Large file Editor 28
LED --- cccc 28
LED-EDIT --- 28
LED-SAVE --- 29
LED-FILE --- cccc 29
TOUCH --- cccc 29
UNLINK --- cccc 29
3.1.3 Line Editor....30
-MOVE a n ---....31
.PAD ---....31
B --- 31
D n --- 31
BCOPY n1 n2 --- 31
E n --- 31
H n --- 31

INS n --- 31
L --- 31
LINE n --- a 31
N --- 31
P n --- 31
RE n ---....32
S n --- 32
SAVE --- 32
ROOM --- 32
TEXT c ---....32
UNUSED --- n....32
WHERE n1 n2 --- 32
WIPE --- 32
3.1.4 Block Search and Locate Utility....33
LOCATE --- 33
GREP --- 33
BSEARCH n1 n2 --- 34
3.2 Graphics mode and Layer facility....35
LAYER! n ---....35
LAYER0 ---....36
LAYER10 ---....36
LAYER11 ---....36
LAYER12 ---....36
LAYER13 ---....36
LAYER2 ---....36
LAYER2+ ---....37
IDE_MODE! n ---....37
IDE_MODE@ --- hl de bc a....37
3.2.1 Low-level definitions....38
ATTRIB --- n....38
PIXELADD x y --- n....38
PIXELATT x y --- n....38

PLOT x y --- 38
POINT x y --- c....38
3.2.2 High-level definitions....39
DRAW-CIRCLE x y r ---....39
DRAW-LINE x0 y0 x1 y1 ---....39
PLOT x y ---....39
PAINT x y ---....39
3.3 Colors & Attributes....40
.BORDER b ---....40
.BRIGHT b ---....40
.FLASH b ---....40
.INK b ---....40
.INVERSE b ---....40
.OVER b ---....40
.PAPER b ---....40
3.4 LAYERxx-GRAPHICS - the modular alternative to GRAPHICS....41
3.5 Filesystem....43
3.5.1 Filesystem navigation words....43
CD --- 43
DIR --- 43
DOSVER --- 43
MAKEDIR --- 43
PWD --- 43
REMOVEDIR --- 43
WILDCARD --- 44
3.5.2 Loading and saving bytes....44
LOAD-BYTES a n --- 44
LOAD2BLOCK n --- 44
SAVE-BYTES a n --- 44
SAVE-BYTES-FORCE a n --- 44
3.5.3 Low-level definitions....45
F_CHDIR a --- f....45

F_CHMOD a1 attr mask -- f....45
F_CLOSE fh --- f....45
F_FGETPOS fh --- d f....45
F_FTRUNCATE fh d --- f....45
F_GETCWD buf --- f....45
F_GETLINE a n1 fh --- n2....46
F_INCLUDE fh --- 46
F_MKDIR a --- f....46
F_OPEN a1 a2 b --- fh f....46
F_OPENDIR a --- fh f....46
F_READ a u1 fh --- u2 f....46
F_READDIR a1 a2 fh --- u f....46
F_RENAME src dest --- f....47
F_REWINDDIR fh --- f....47
F_RMDIR a --- f....47
F_SEEKDIR hf d --- f....47
F_TELLDIR hf --- d f....47
F_TRUNCATE a d --- f....47
F_RMDIR a --- f....47
F_SEEK d fh --- f....47
F_SYNC fh --- f....47
F_UNLINK a --- f....48
M_DOSVERSION --- ver language-code....48
3.6 Mouse....49
MOUSE-XY --- n1 n2....50
?MOUSE --- f....50
MOUSE --- n....50
NO-MOUSE --- 51
BYE --- 51
3.7 Interrupt Service Routine....52
ISR-OFF --- 52
ISR-XT xt --- 52

ISR-ON --- 52
ISR-EI --- 53
ISR-DI --- 53
ISR-IM1 --- 53
INT-IM2 --- 53
ISR-SYNC --- 53
SETIREG b --- 53
ISR-RET --- 53
ISR-SUB --- 53
3.8 Debugger Utility....54
SEE --- 54
3.8.1 The Inner-interpreter....56
DUMP a u --- 57
.WORD a --- 57
.S --- 57
DEPTH --- n....57
3.9 Floating-Point Option....58
3.9.1 Floating-point option activation and deactivation....59
FLOATING --- 59
INTEGER --- 59
NO-FLOATING --- 59
3.9.2 Floating-point number conversion....59
D>F d --- fp....59
F>D fp --- d....59
FLOAT n --- fp....59
FIX fp --- n....59
3.9.3 Representation and constants....59
F>PAD fp --- u....59
F.R fp u --- 59
F. fp --- 59
1/2 --- fp....60
PI --- fp....60

3.9.4 Arithmetics....60
F- fp1 fp2 --- fp3....60
F+ fp1 fp2 --- fp3....60
F* fp1 fp2 --- fp3....60
F/ fp1 fp2 --- fp3....60
FNEGATE fp1 --- fp2....60
FSGN fp1 --- fp2....60
FABS fp1 --- fp2....60
F/MOD fp1 fp2 --- fp3 fp4....60
F** fp1 fp2 --- fp3....60
FMOD fp1 fp2 --- fp3....60
F*/ fp1 fp2 fp3 --- fp4....60
F< fp1 fp2 --- f....61
F> fp1 fp2 --- fp3....61
F0< fp1 --- f....61
F0> fp1 --- f....61
3.9.5 Log, Exp, Trig....61
FLN fp1 --- fp2....61
FEXP fp1 --- fp2....61
FINT fp1 --- fp2....61
FSQRT fp1 --- fp2....61
FSIN fp1 --- fp2....61
FCOS fp1 --- fp2....61
FTAN fp1 --- fp2....61
FASIN fp1 --- fp2....61
FACOS fp1 --- fp2....62
FATAN fp1 --- fp2....62
RAD>DEG fp1 --- fp2....62
DEG>RAD fp1 --- fp2....62
3.9.6 Low-level definitions....62
FOP n --- 62
>W fp --- 62

W> --- fp....62
3.10 ASSEMBLER vocabulary....62
3.10.1 Complete list of Assembler Op-Code....64
3.10.2 Assembler Example - Checksum....68
3.10.3 Assembler Example - WHITE-NOISE from Egghead 3....69
3.10.4 Assembler Example “ms” millisecond delay....70
HOLDPLACE --- a1....71
DISP, a1 a2 --- 71
BACK, a1 --- 71
3.10.5 Syntax check error messages....72
3.11 Raspberry Pi Zero accelerator....73
3.11.1 RPi0 high-level communication....74
ASK --- n....74
DISPLAY n --- 74
THIS --- a n....75
Rpi0-SEND a1 n1 a2 n2 --- n3....75
3.11.2 RPi0 UART low-level communication....75
?UART-BYTE-READY --- f....75
?UART-BUSY-TX --- f....75
RPI0 --- 75
TERM --- 76
RPi0-INIT --- 76
RPi0-SELECT --- 76
UART-1ST-TIMEOUT --- a....76
UART-2ND-TIMEOUT --- a....76
UART-RX-BYTE --- b | 0....76
UART-RX-TIMEOUT n --- c | 0....76
UART-RX-BURST a n1 --- n2....76
UART-SET-BAUDRATE d --- 77
UART-SET-PRESCALAR n --- 77
UART-TX-BYTE b --- 77
UART-SEND-TEXT a n --- 77

UART-WAIT-B b --- 77
UART-WAIT-CR-LF --- 77
UART-WAIT-STR a n --- 77
UART-WAIT-PROMPT --- 77
3.12 Tilemap Hardware....78
3.12.1 Hardware overview....78
3.12.2 TILE80 — 80-column text driver....79
TILE80 --- 79
NO-TILE --- 79
T-CLS --- 79
T-HOME --- 79
T-EMITC c --- 79
T-EMIT c --- 0....79
T-SCROLL --- 79
T-CR --- 79
T-POS --- 80
T-WIDTH --- 80
T-HEIGTH --- 80
T-SIZE --- 80
DISPLAY-FILE --- 80
T-XY --- a....80
3.12.3 LAYER3 — low-level definitions....80
TILE-OFF --- 80
TILE-40 --- 80
TILE-80 --- 80
TILE-TXT --- 81
SET-PAL --- 81
GET-PAL --- 81
3.12.4 TILE80-setup — charset preparation....81
3.12.5 Notes....81
3.13 Beeper....82
BLEEP n1 n2 --- 82

BLEEP-CALC n1 n2 --- n3 n4....82
BEEP-PITCH n --- n2....82
BEEP n1 n2 ---....82
3.14 AY sound chip facility (Turbosound Next)....82
AYSELECT n ---....83
AY! b ayreg ---....83
AY!! n ayreg ---....83
SHH ---....83
ENABLE-TURBOSOUND ---....83
ENABLE-MONO ---....83
DISABLE-MONO ---....83
AYSETUP ---....84
3.15 Keyboard input....84
KEY --- c....84
WAIT-KEY ---....84
UPPER c1 --- c2....84
?TERMINAL --- f....84
?ESCAPE --- f....85
3.15.1 Direct keyboard-matrix scanning....85
KEYBOARD --- c....85
NO-KEY --- n....85
KEY#>PORT n --- port....85
KEY#>MASK n --- mask....85
KEY#>RM n --- port mask....86
KEY-PRESSED? port mask --- f....86
KEY-DOWN? n --- f....86
KEY-SCAN --- n....86
WAIT-NOKEY ---....86
WAIT-NEWKEY --- n....86
GET-KEY --- n....86
KEY-NAMES --- a....86
.KEY n ---....86

3.16 Direct Memory Access (DMA).....86
DMA-COPY src dest len ---.....87
DMA-FILL byte dest len ---.....87
DMA-OUT src ioport len ---.....87
DMA-IN ioport dest len --- 87
DMA-STATUS --- b.....87
DMA-DONE? --- f.....88
DMA-ENABLE / DMA-DISABLE / DMA-LOAD / DMA-CONTINUE / DMA-RESET /.....88
DMA-RESET-STATUS ---.....88
4 Tecnical specifications.....88
4.1 Z80 CPU Registers.....88
4.2 Single Cell 16 bits Integer Number Encoding.....88
4.3 Two cells 32 bits Integer Number Encoding.....89
4.4 Double Cell Floating-Point Number Encoding.....89
4.5 Single Cell 16 bits Heap Pointer Address Encoding.....90
4.6 Dictionary memory structure.....90
4.6.1 Heap-pointers and resolved addresses.....91
5 Error messages.....91
6 The Dictionary.....93
6.1 The “core” dictionary.....93
'0x00' --- (immediate).....93
! n a ---.....93
!CSP ---.....93
d1 --- d2.....93
#> d --- a u.....93
#BUFF --- n.....93
#S d1 --- d2 93
#SEC --- n.....94
' --- xt.....94
(--- (immediate).....94
(+LOOP) n ---.....94
(.") ---.....94

(;CODE) ---.....94
(?DO) ---.....94
(?EMIT) c1 --- c2.....94
(ABORT) ---.....95
(COMPARE) a1 a2 n -- b.....95
(DO) ---.....95
(FIND) a1 a2 --- cfa b tf.....95
(LEAVE) --- 95
(LINE) n1 n2 --- a b.....95
(LOOP) ---.....95
(MAP) a2 a1 n c1 --- c2.....96
(NEXT) --- a.....96
(NUMBER) d a --- d2 a2.....96
(PREFIX) a --- a2.....96
(SGN) a --- a2 f.....97
* n1 n2 --- n3.....97
*/ n1 n2 n3 --- n4.....97
*/MOD n1 n2 n3 --- n4 n5.....97
+ n1 n2 --- n3.....97
+! n a --- 97
+- n1 n2 --- n3.....97
+BUF a1 --- a2 f.....97
+LOOP n1 --- (run time).....97
+ORIGIN n --- a.....97
+TO n --- cccc.....98
, n --- 98
," --- 98
- n1 n2 --- n3.....98
--> --- 98
-1 --- n.....98
-?EXECUTE --- 98
-DUP n --- n n (non zero).....98

-FIND --- xt b tf (ok).....99
-TRAILING a n1 --- a n2.....99
. n ---.....99
." --- (immediate).....99
.(--- (immediate).....99
.C c --- (immediate).....99
.LINE n1 n2 ---.....99
.R n1 n2 ---.....99
/ n1 n2 --- n3.....100
/MOD n1 n2 --- n3 n4.....100
0 --- n.....100
0< n --- f.....100
0= n --- f.....100
0> n --- f.....100
0BRANCH f ---.....100
1 --- n.....100
1+ n1 --- n2.....100
1- n1 --- n2.....100
2 --- n.....100
2! d a --- 101
2* n1 --- n2.....101
2+ n1 --- n2.....101
2- n1 --- n2.....101
2/ n1 --- n2.....101
2@ a --- d.....101
2CONSTANT d --- (immediate) (compile time).....101
2DROP d --- 101
2DUP d --- d d.....101
2OVER d1 d2 --- d1 d2 d1.....101
2ROT d1 d2 d3 --- d2 d3 d1.....102
2SWAP d1 d2 --- d2 d1.....102
2VARIABLE --- (immediate) (compile time).....102

3 --- n....102
3DUP n1 n2 n3 --- n1 n2 n3 n1 n2 n3....102
: --- 102
:NONAME --- xt....102
; --- (immediate)....103
;CODE --- (immediate)....103
;S --- 103
< n1 n2 --- f....103
<# ---....103
<> n1 n2 --- f....103
<BUILDS ---....103
<NAME xt --- nfa....104
= n1 n2 --- f....104
> n1 n2 --- f....104
>BODY xt --- pfa....104
>IN --- a 104
>NUMBER d a u --- d2 a2 u2 104
>R n ---....104
? a --- 104
?COMP --- 105
?CSP --- 105
?DO n1 n2 --- (immediate) (run time)....105
?DO- [a1 n1] a n --- 105
?DUP n --- n n (non zero)....105
?ERROR f n --- 105
?EXEC --- 105
?LOADING --- 105
?PAIRS n1 n2 --- 105
?STACK --- 106
?TERMINAL --- f....106
@ a --- n....106
ABORT ---....106

ABORT" f --- (run time).....106
ABS n --- u.....106
ACCEPT a n1 --- n2.....106
AGAIN --- (immediate) (run time).....106
ALLOT n --- 107
ALIGN --- 107
ALIGNED a1 --- a2.....107
AND n1 n2 --- n3.....107
AUTOEXEC ---.....107
B/BUF --- n.....107
B/SCR --- n.....107
BACK a --- 107
BASE --- a.....107
BASIC u --- 107
BETWEEN n1 n2 n3 --- f.....107
BEGIN --- (immediate) (run time).....107
BINARY --- 108
BL --- c.....108
BLANK a n ---.....108
BLK --- a.....108
BLK-FH --- a.....108
BLK-FNAME --- a.....108
BLK-INIT --- 108
BLK-READ a n --- 108
BLK-SEEK n --- 108
BLK-WRITE a n --- 108
BLOCK n --- a.....109
BOUNDS a n --- a+n a.....109
BRANCH --- 109
BUFFER n --- a.....109
BYE --- 109
C! b a ---.....109

C, b --- 109
C/L --- c....109
C@ a --- b....109
CALL# n1 a --- n2....109
CASEOFF --- 110
CASEON --- 110
CATCH xt --- 0 | n....110
CELL --- 2....110
CELL+ n1 --- n2....110
CELL- n1 --- n2....110
CELLS n1 --- n2....110
CFA pfa --- xt....110
CHAR --- c....110
CLS --- 110
CMOVE a1 a2 n --- 111
CMOVE> a1 a2 n ---....111
CODE --- 111
COMPARE a1 n1 a2 n2 --- b....111
COLD ---....112
COMPILE ---....112
COMPILE, xt ---....112
CONSTANT n --- (immediate) (compile time)....112
CONTEXT --- a....112
COUNT a1 --- a2 b....112
CR --- 112
CREATE --- (compile time)....112
CSP --- a....113
CURRENT --- a....113
CURS --- 113
D+ d1 d2 --- d3....113
D+- d1 n --- d2....113
D- d1 d2 --- d3....114

D. d --- 114
D.R d n --- 114
D0= d --- f....114
D< d1 d2 --- f....114
D= d1 d2 --- f....114
DABS d --- ud....114
DECIMAL --- 114
DEFINITIONS --- 114
DEVICE --- a....114
DIGIT c n --- u tf (ok)....115
DLITERAL d --- d (immediate) (run time)....115
DMAX d1 d2 --- d3....115
DMIN d1 d2 --- d3....115
DNEGATE d1 --- d2....115
DO n1 n2 --- (immediate) (run time)....115
DOES> --- 115
DP --- a....116
DPL --- a....116
DROP n --- 116
DUP n --- n n....116
DUP>R n --- n 116
DU< ud1 ud2 --- f....116
ELSE a1 n1 --- a2 n2 (immediate) (compile time)....116
EMIT c --- 116
EMITC b --- 116
EMPTY-BUFFERS --- 116
ENCLOSE a c --- a n1 n2 n3....117
END a n --- (immediate) (compile time)....117
ENDIF a n --- (immediate) (compile time)....117
ENUMERATED n --- (immediate) (compile time)....117
ERASE a n --- 117
ERROR b --- n1 n2....117

EVALUATE a u --- 117
EXEC: n --- 118
EXECUTE cfa --- 118
EXIT --- 118
EXP --- a....118
EXPECT a n --- 118
FENCE --- a....118
FILL a n b --- 118
FIRST --- a....118
FLD --- a....118
FLIP n1 --- n2....119
FLUSH --- 119
FM/MOD d n1 --- n2 n3....119
FORGET --- 119
FORTH --- (immediate)....119
HANDLER --- a....119
HELP --- cccc....119
HERE --- a....119
HEX --- 119
HLD --- a....120
HOLD c --- 120
HP --- a....120
HP, n --- 120
I --- n 120
I' --- n 120
ID. nfa --- 120
IF f --- (immediate) (run time)....120
IMMEDIATE --- 120
INCLUDE ---....121
INDEX n1 n2 ---....121
INKEY --- b 121
INTERPRET --- 121

INVERT n1 --- n2....121
INVV --- 121
J --- n 121
K --- n 121
KEY --- b 121
L/SCR --- n....122
LATEST --- nfa 122
LEAVE --- 122
LFA pfa --- lfa 122
LIMIT --- a....122
LIST n --- 122
LIT --- n 122
LITERAL n --- n (immediate) (run time)....122
LOAD n --- 123
LOOP a n --- (immediate) (run time)....123
LP --- a....123
LSHIFT n1 u --- n2....123
M* n1 n2 --- d....123
M*/ d1 n1 n2 --- d2....123
M+ d u --- d2....123
M/ d n1 --- n2 123
M/MOD d1 n1 --- n2 n3 123
MARK a n --- 124
MARKER --- (immediate) (run time)....124
MAX n1 n2 --- n3....124
MESSAGE n --- 124
MIN n1 n2 --- n3....124
MMU7! n --- 124
MMU7@ --- n....124
MOD n1 n2 --- n3....124
MS u --- 124
M_P3DOS n1 n2 n3 n4 a --- n4 n5 n6 n7 f....125

NEEDS ---.....125
NEGATE n --- -n.....126
NFA pfa --- nfa.....126
NIP n1 n2 --- n2.....126
NMODE --- a.....126
NOOP --- 126
NOT --- 126
NUMBER a --- d.....126
OCTAL --- 127
OFFSET --- a.....127
OPEN< --- fh.....127
OR n1 n2 --- n3.....127
OUT --- a.....127
OVER n1 n2 --- n1 n2 n1.....127
P! b u --- 127
P@ u --- b.....127
PAD --- 127
PFA nfa --- pfa.....127
PICK n --- pfa.....127
PLACE --- a.....127
POSTPONE --- 128
PREV --- a.....128
QUERY --- 128
QUIT --- 128
R@ --- n.....128
R# --- a.....128
R/W a n f --- 128
R0 --- a.....128
R> --- n.....128
R>DROP --- 128
RECURSE --- 129
REG! b n --- 129

REG@ n --- b....129
REMOUNT --- 129
RENAME ---....129
REPEAT a1 n1 a2 n2 --- (immediate) (compile time)....129
ROT n1 n2 n3 --- n2 n3 n1....129
ROLL n1 ... k --- n2 ... n1....129
RP! a --- 129
RP@ --- a....130
RSHIFT n1 u --- n2....130
S0 --- a....130
S>D n --- d....130
SCR --- a....130
SELECT n --- 130
SIGN n --- 130
SM/REM d n1 --- n2 n3....130
SMUDGE --- 130
SOURCE-ID --- a....131
SP! a --- 131
SP@ --- a....131
SPACE --- 131
SPACES n --- 131
SPAN --- a....131
SPLASH --- 131
SPLIT n1 --- n2 n3....131
STATE --- a....131
SWAP n1 n2 --- n2 n1....131
THEN a n --- (immediate)....131
THROW ... n --- ... | ... n....131
THRU scr1 scr2 --- 132
TIB --- a....132
TO n --- 132
TOGGLE a b --- 132

TRAVERSE a1 n --- a2....132
TRUV ---....132
TUCK n1 n2 --- n2 n1 n2....132
TYPE a n --- 132
U. u --- 132
U< u1 u2 --- f....132
UM* u1 u2 --- ud....132
UM/MOD ud u1 --- u2 u3....133
UNLOOP --- 133
UNTIL a n --- (immediate) (compile time)....133
UPDATE --- 133
UPPER c1 --- c2....133
USE --- 133
USED --- a....133
USER n --- 133
VALUE n --- 134
VARIABLE ---....134
VIDEO --- 134
VOC-LINK --- a....134
VOCABULARY ---....134
WARM --- 135
WARNING --- a....135
WHILE f --- (immediate) (run time)....135
WIDTH --- a....135
WITHIN n1 n2 n3 --- f....135
WORD c --- a....135
WORDS --- 135
XOR n1 n2 --- n3....136
ZAP --- 136
ZAP" xt --- 136
[--- (immediate) 136
['] --- (immediate) (compile time) 137

[CHAR] --- (immediate) (compile time) 137
[COMPILE] --- (immediate) 137
\ --- 137
] --- 137
6.2 Case -Of structure....138
CASE n0 --- (immediate) (run time)....138
OF n0 nk --- (immediate) (run time)....138
ENDOF --- (immediate) (run time)....138
ENDCASE --- (immediate) (run time)....138
(OF) n0 nk --- (run time)....138
6.3 Named Local Variables (LOCALS)....139
6.4 Heap Memory Facility....140
6.4.1 Heap-Pointer encoding and decoding....140
6.4.2 Heap structure....140
(H") --- a n....143
+" ha --- ha2....143
+C ha c --- ha....143
>FAR ha --- a p....143
<FAR a p --- ha....143
FAR ha --- a....143
H" --- ha....143
HEAP n --- ha....143
POINTER ha --- a....144
S" --- a n....144
SKIP-HP-PAGE n --- 144
6.4.3 Heap Pointer description for 64 kiBytes space....145
6.5 Testing Suite....146
TESTING --- 149
T{ --- 149
-> --- 149
}T --- 149
6.6 Other Utilities....150

SHOW-PROGRESS n --- 150
?VOCAB --- 150
VIEW --- 150
6.7 Persistence across sessions.....151
7 The Memory Map.....152
Contents.....153